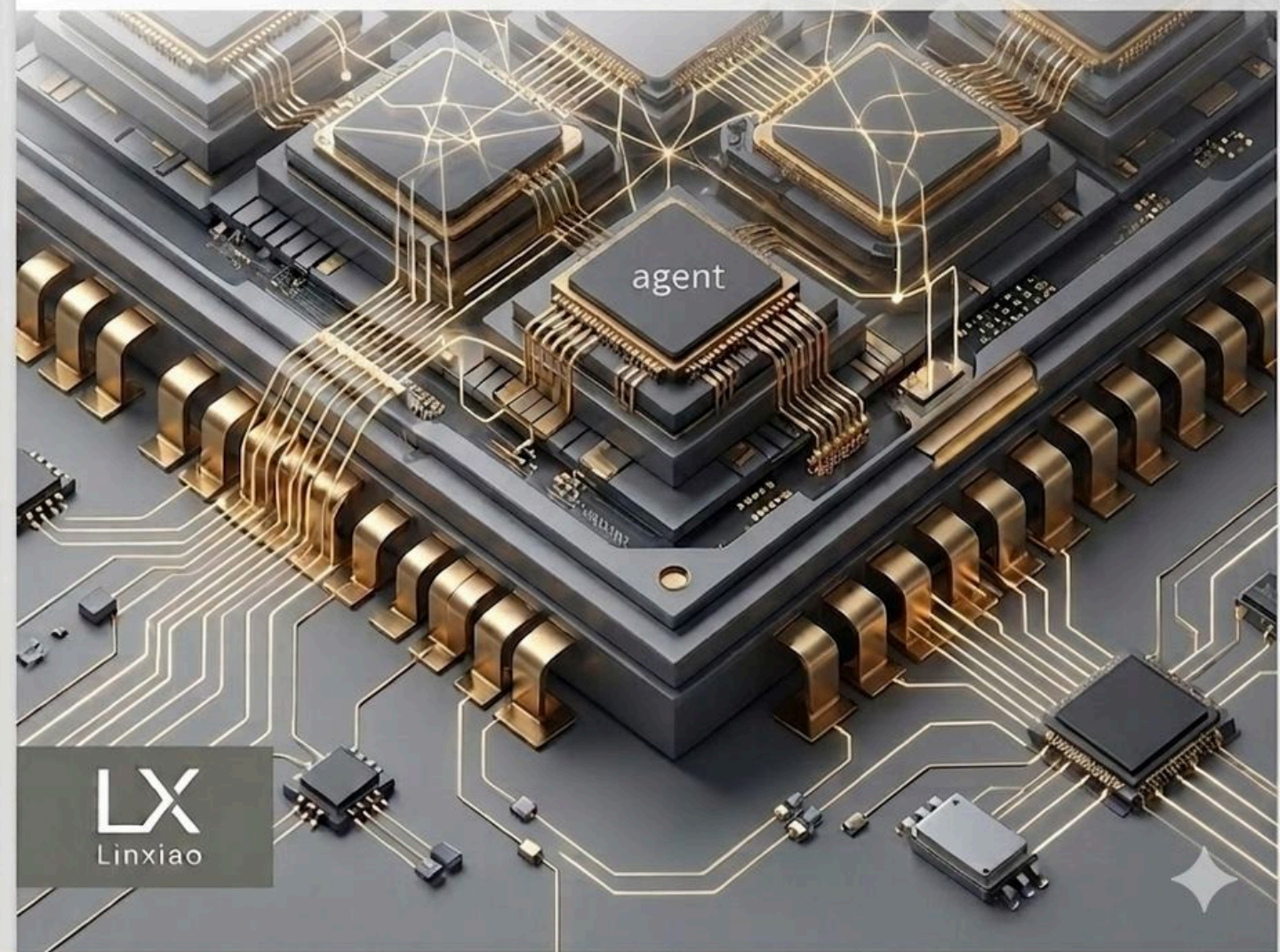


# Agent CPU 体系结构

基于多智能体方法的深度解析



# Agent CPU 体系结构

基于多智能体方法的深度解析

---

**作者** 林啸, GPT

**版本** 第一版讲义草案

**适用对象** 计算机体系结构、芯片设计、Agent 系统、编译器、操作系统、AI 基础设施方向的研究者与工程师

基于在线教材正文、插图、代码块与表格整理的正式印刷版，用于课程讲授、研究交流与内部评审。

本版针对 PDF 教材阅读进行了开本、章节分页、图表边界与正文层级重排，适合正式归档与打印。

TABLE OF CONTENTS

目录

Agent CPU 体系结构

前言：为什么需要一门新学科

学科名称与边界

本书结构

第一篇：学科基础

第 1 章 Agentic Workload 的出现

第 2 章 Agent CPU 的基本抽象

第 3 章 双域架构：Super Domain 与 Agent Domain

第 4 章 APU-IR：Agent 与硬件之间的契约

第 5 章 Memory 与 NoC：从状态到数据移动

第 6 章 任务型 Agent 的执行模型

第 7 章 长期任务记忆

第 8 章 安全与权限

第四篇：传统体系结构映射

第 9 章 从 ISA 到 APU-IR

第 10 章 从 Cache 到 Ledger-Aware Memory

第 11 章 从 ROB Commit 到 Candidate Commit

第 12 章 从 Branch Prediction 到 Task-State Recovery

第 13 章 从 SPEC 到 Agentic Benchmark

第五篇：方法与实验

|                          |       |
|--------------------------|-------|
| 第 14 章 Benchmark 方法      | ..... |
| 第 15 章 九天 APU v0.1 实验    | ..... |
| 第 16 章 从教材到研究计划          | ..... |
| 第 17 章 学科建设纲领            | ..... |
| 附录 A：术语表                 | ..... |
| 附录 B：第一版教学大纲             | ..... |
| 附录 C：与九天项目的对应关系          | ..... |
| 附录 D：课程实验手册              | ..... |
| 附录 E：APU-IR 示例库          | ..... |
| 附录 F：Benchmark 评分 Rubric | ..... |

# 前言：为什么需要一门新学科

传统计算机体系结构研究的是“程序如何在机器上高效执行”。它默认程序由人类编写，软件需要长期维护，二进制接口需要稳定，操作系统需要保护多个进程，硬件需要尽量兼容历史代码。

AI Agent 出现后，计算系统中出现了一类新的执行对象：由 Agent 围绕目标即时生成、即时验证、即时执行、即时回收的短生命周期逻辑。它们由任务流、证据链、工具调用图和状态变换共同组成。

因此，我们需要建立一门新学科：

## Agent CPU 体系结构学。

这门学科研究：

- Agent 生成的代码如何表达为可验证、可调度、可恢复的执行对象。
- CPU、APU、NoC、Memory、Runtime、Compiler、Security 如何共同服务 Agentic workload。
- 长任务记忆、工具调用、上下文投影、证据账本、恢复点如何成为硬件可感知的系统状态。
- 传统 CPU 中为人类软件服务的复杂机制，哪些应该保留在控制面，哪些应该从 Agent 执行面移除。

它是一种围绕 Agent 行为、任务流、长期记忆和显式执行状态建立的新体系结构方法。

# 学科名称与边界

## 中文名称

Agent CPU 体系结构学

## 英文名称

# Agent CPU Architecture

## 核心命题

当代码由 Agent 生成时，硬件可以把它表达为：

- 有目标的任务片段。
- 有权限边界的执行对象。
- 有输入输出和证据引用的状态变换。
- 有预算、恢复点和 trace 的可审计行为。

## 与既有学科的区别

| 学科        | 研究对象           | Agent CPU 体系结构的不同点                            |
|-----------|----------------|-----------------------------------------------|
| 传统计算机体系结构 | 人类软件在 CPU 上执行  | 关注 Agent 生成代码、短生命周期任务和显式状态                    |
| 操作系统      | 进程、线程、内存、文件、设备 | 关注 Agent 任务准入、权限、工具调用和恢复闭环                    |
| 编译器       | 源代码到机器码        | 关注 Agent 意图、任务图、APU-IR 和可验证执行                 |
| GPU 体系结构  | 数据并行和张量吞吐      | 关注分支复杂、非规则、碎片化的 Agentic workload              |
| NPU 体系结构  | 神经网络算子         | 关注模型之外的 Agent 行为、工具编排和长期记忆                    |
| 分布式系统     | 多节点一致性和容错      | 关注单机或片上多 Agent 任务、ledger 和 context projection |

## 本书结构

本书分为五篇。

第一篇定义学科：说明 Agent CPU 为什么不同于传统 CPU、GPU、NPU，并从经典计算机体系结构中继承量化方法、层次化抽象和工作负载驱动设计。

第二篇定义机器：讨论 Super Domain、Agent Domain、任务模型、APU-IR、Memory/NoC。

第三篇定义运行时：讨论任务型 Agent、工具调用、长期记忆、上下文投影、安全和恢复。

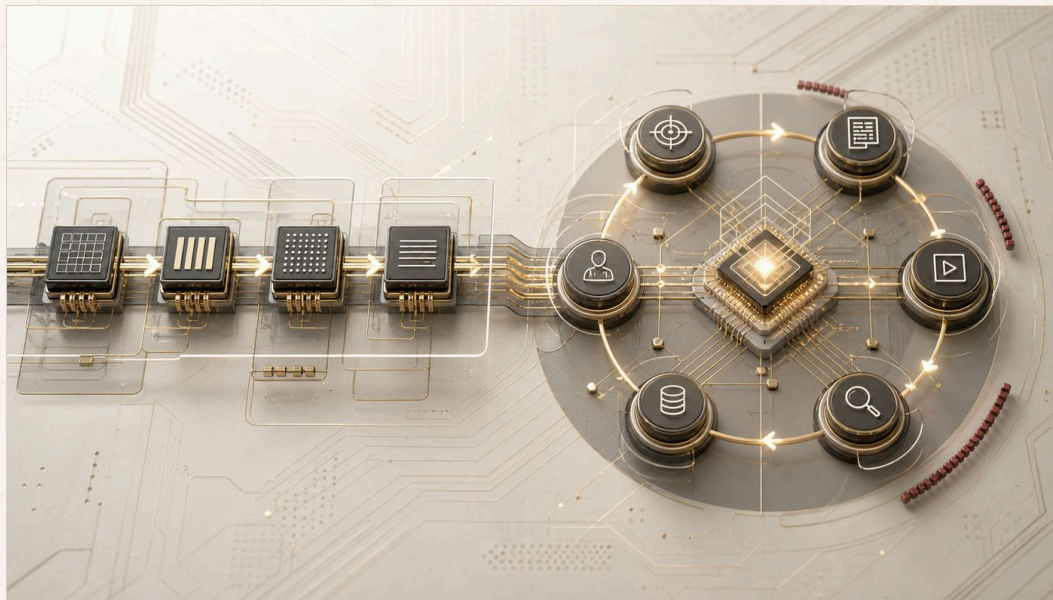
第四篇建立映射：从 ISA、Cache、ROB、Branch Prediction 和 benchmark 方法出发，说明传统体系结构如何转化为 Agent CPU 的设计方法。

第五篇定义方法：讨论 benchmark、验证、课程实验和九天 APU v0.1 的最小闭环。

---

# 第一篇：学科基础

# 第 1 章 Agentic Workload 的出现



第 1 章插图：从线性程序执行到 Agentic workload 循环

## 本章导读

计算机体系结构的发展，常常由新的工作负载推动。数值计算推动了早期算术部件和存储系统的发展，互联网服务推动了并发、缓存和 I/O 设计，深度学习推动了 GPU 与 NPU 的广泛使用。今天，智能体系统正在形成一种新的工作负载形态：系统围绕一个目标持续观察环境、调用工具、更新状态、选择下一步行动，并在多轮之后交付结果。本章把这种形态称为 Agentic workload。

本章面向初学者介绍它的基本含义。我们先回顾传统程序模型，再给出 Agentic workload 的定义，通过一个贯穿例子观察它如何运行，随后讨论它与 CPU、GPU、NPU 的关系，最后提出由此引出的体系结构问题。

## 1.1 传统程序模型

在经典课程中，程序通常被看作一段确定的指令序列。程序员把问题写成算法，算法被表达为代码，处理器按照控制流取指、译码、执行、访存和写回。程序状态主要由寄存器、内存和外部 I/O 表示。对数组排序、求解线性方程、执行一次数据库查询，都可以在这种模型下清楚描述。

这种模型的特点是任务边界明确。输入是什么、输出是什么、程序何时结束，通常可以在设计时说明。即使程序内部存在分支和循环，分支条件也来自当前数据和程序状态。体系结构中的流水线、乱序执行、缓存层次、向量化等概念，都可以围绕这种执行方式展开。

现代软件已经比单个顺序程序复杂得多。操作系统管理进程和线程，数据库系统管理事务，分布式系统处理远程调用，机器学习框架把训练和推理组织为张量计算图。它们增加了并发、通信和概率性，但很多任务仍有清晰边界：一次推理、一次事务、一次训练迭代、一次服务请求。系统设计者可以围绕这些边界估计延迟、吞吐、内存占用和能耗。

智能体系统带来了新的执行形态。用户可能只给出一个目标，例如“整理实验结果并形成结论”。系统需要判断读取哪些资料、调用哪些工具、如何验证中间结果、何时继续探索、何时停止。运行过程不再只是一次函数调用，也不是固定层数的神经网络推理，而是由模型推理、工具执行、记忆状态和控制策略交织形成的行动序列。

## 1.2 Agentic Workload 的定义

Agentic workload 是指以目标为牵引、以智能体循环为基本执行形态的一类计算工作负载。它通常包含四个要素：任务目标、上下文状态、行动选择和反馈更新。系统读取当前上下文，生成下一步行动，执行行动并观察结果，再把新信息写回上下文，直到满足停止条件或交付结果。

这里的行动可以有多种形式：发起一次模型推理、检索文档、查询数据库、运行程序、调用外部服务、写入中间记忆，或请求人工确认。关键不在某个单独动作，而在这些动作被组织成带反馈的多步决策过程。

从体系结构角度看，Agentic workload 有几个突出特征。第一，控制流具有动态性，下一步行动常由模型输出和外部观察共同决定。第二，状态具有层次性，短期上下文、长期记忆、工具返回结果、用户约束和安全策略都会影响后续执行。第三，性能指标具有复合性，除了单次推理延迟，还要考虑推理轮数、工具等待、上下文搬运、失败重试和最终质量。第四，正确性依赖过程约束，系统需要在合适步骤中验证事实、维护权限，并保留可追踪的中间状态。

因此，可以把 Agentic workload 理解为一种以推理为控制核心、以工具和记忆为执行支撑、以多步反馈循环推进目标的工作负载。

|     | 控制流   | 状态层级      | 性能   | 正确性  |
|-----|-------|-----------|------|------|
| 目标  | 开放终点  | 根目标       | 完成度  | 成功条件 |
| 上下文 | 滚动更新  | 短期/长期     | 投影成本 | 来源边界 |
| 行动  | 工具选择  | 任务片段      | 调度开销 | 权限检查 |
| 反馈  | 继续/停止 | ledger 回写 | 重试代价 | 候选验证 |

四个基本要素决定 Agentic workload 的行为骨架；外层体系结构问题围绕它们展开。

图 1-1：Agentic workload 的四个基本要素与体系结构特征

### 1.3 贯穿例子：从研究问题到报告草稿

考虑一个学生科研助手。用户给出目标：“比较三种图像分类模型在小样本数据集上的表现，并写出一页报告草稿。”系统接到任务后，首先理解任务边界：三种模型是什么，数据集在哪里，评价指标采用准确率、F1 分数还是训练成本，报告面向课程作业还是实验论文。

第一轮循环中，系统生成计划：读取实验记录，提取模型名称和指标，检查是否缺少数据，再组织结论。随后它调用数据读取工具，得到若干表格和日志。观察结果显示，其中一个模型缺少 F1 分数。第二轮循环中，系统决定根据混淆矩阵重新计算该指标，于是调用计算工具，并把新指标写回上下文。

第三轮循环中，系统比较结果。它发现某个模型准确率最高，但训练时间更长；另一个模型准确率略低，却在小样本条件下更稳定。于是系统形成判断：不能只按单一指标排序，应从精度、稳定性和资源消耗三个维度分析。最后，系统生成报告草稿，并检查指标是否一致、结论是否过度推断、篇幅是否符合要求。

这个例子展示了 Agentic workload 的典型结构。模型推理负责解释目标、制定计划和撰写文本；工具调用负责读取数据和执行计算；记忆状态保存任务约束、中间结果和自检结论；控制循环把这些环节连接起来。若数据完整，系统会直接比较；若数据缺失，系统会补算；若证据不足，系统会回到资料检查阶段。

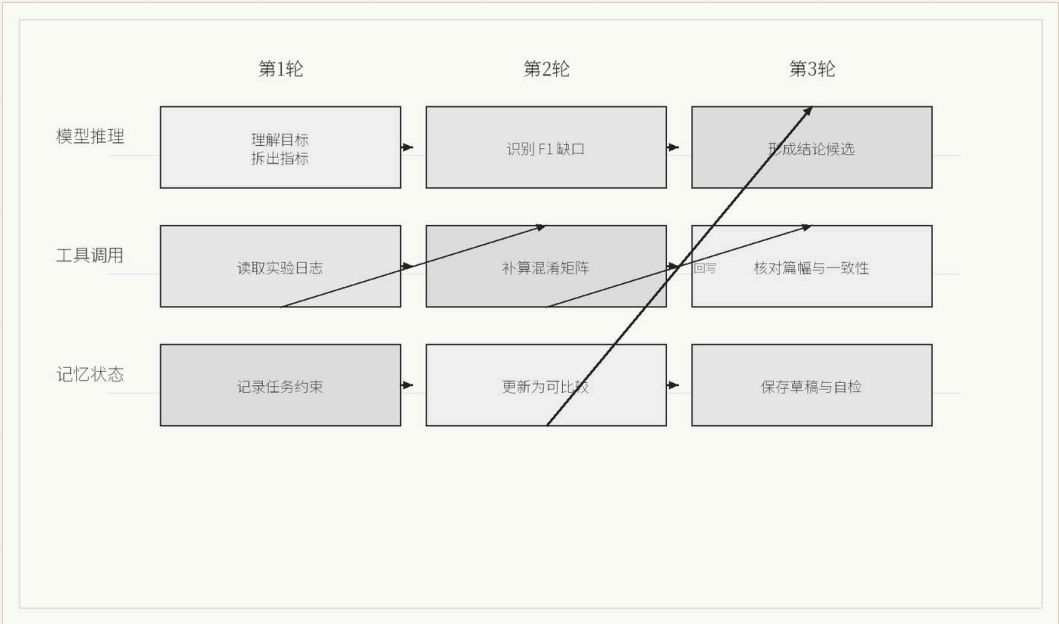


图 1-2：科研助手任务的多轮执行闭环

### 1.4 与 CPU、GPU、NPU 的关系

Agentic workload 的出现并不削弱 CPU、GPU 或 NPU 的作用，而是把它们放入更复杂的协作关系中。

CPU 仍承担通用控制、操作系统调度、工具执行、数据预处理和外部 I/O 管理。智能体循环中有许多分支多、粒度小、依赖外部状态强的步骤，例如发起检索、解析返回结果、维护会话状态和检查权限规则，这些环节通常适合由 CPU 支撑。

GPU 主要服务于大规模并行计算，尤其是语言模型和多模态模型中的矩阵乘法、注意力计算和批量推理。当系统同时服务大量智能体请求时，GPU 的吞吐、显存容量和互连带宽会直接影响总体效率。

NPU 面向神经网络推理进行专门优化，强调能效和低延迟。在移动设备、边缘设备和专用服务器中，NPU 可以承担语音识别、意图理解、小模型推理等任务，使部分智能体能力更接近数据产生的位置。

一次完整的 Agentic workload 可能包含多次模型推理、多次工具调用、多次状态读写和若干次验证。瓶颈可能出现在推理计算，也可能出现在上下文传输、工具等待、存储检索或调度排队。因此，研究它时不能只问哪种处理器更快，还要问不同计算单元如何共同支撑一个多步目标的完成。

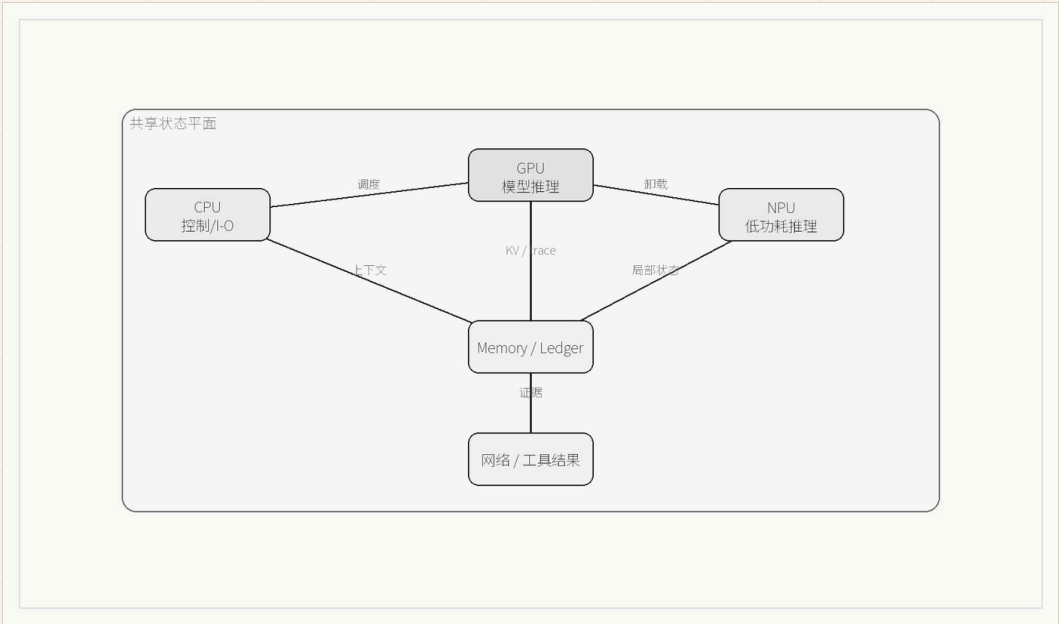


图 1-3: CPU、GPU、NPU 在 Agentic workload 中的协同分工

## 1.5 新的体系结构问题

第一个问题是控制流表示。传统 ISA 描述机器可执行的低层操作，而智能体系统中的行动还包括检索、调用、验证和提交等高层步骤。如何把这些行动表示为可调度、可观察、可恢复的执行单元，是新的基础问题。

第二个问题是状态管理。智能体的状态包括上下文窗口中的短期信息、外部存储中的长期记忆、工具执行产生的临时结果以及权限记录。哪些状态应靠近模型，哪些应进入数据库，哪些应压缩或删除，都会影响性能、成本和可靠性。

第三个问题是调度与资源分配。一个任务可能在 CPU、GPU、NPU、网络和存储之间不断切换。调度器需要在延迟、吞吐、成本和公平性之间权衡，特别是在多用户共享系统中避免长任务过度占用资源。

第四个问题是推测与回滚。智能体可能并行尝试几种解决路径，或先假设某条资料有用并展开分析。若后来发现方向错误，系统需要回退到合适状态，丢弃无效中间结果，并处理外部工具带来的副作用。

第五个问题是可验证性。Agentic workload 的输出质量依赖多步过程。如果某一步引用了错误数据，后续推理可能建立在错误基础上。执行轨迹记录、数据来源标注、工具权限控制和结果校验，都是支撑可靠智能体系统的重要机制。

| 问题    | 核心张力          | 硬件对象                         | 度量   |
|-------|---------------|------------------------------|------|
| 控制流表示 | 开放目标 vs 定长指令  | 任务片段 / action record         | 调度粒度 |
| 状态管理  | 上下文增长 vs 有限带宽 | projection / ledger          | 访问延迟 |
| 资源分配  | 吞吐 vs 成本      | budget / queue / tool slot   | 效率   |
| 推测与回滚 | 探索收益 vs 恢复代价  | anchor / replay              | 恢复时间 |
| 可验证性  | 灵活性 vs 审计     | trace / evidence / candidate | 错误率  |

图 1-4：第一章引出的五类基础体系结构问题

## 1.6 小结

本章介绍了 Agentic workload 的出现及其基本含义。它以目标为牵引，通过模型推理、工具调用、记忆状态和反馈循环推进任务，是一类多步、动态、状态丰富的计算工作负载。

从硬件关系看，CPU 处理控制和 I/O，GPU 提供大规模模型算力，NPU 支持高能效推理，系统软件则把这些能力组织成面向目标的执行过程。由此产生的控制流表示、状态管理、调度、推测恢复和可验证性问题，将成为后续讨论 Agent CPU 体系结构的重要出发点。

## 习题

1. 用自己的话解释 Agentic workload 的四个基本要素。
1. 选择一个熟悉场景，画出其中可能出现的智能体循环。
1. 为什么只测量一次模型推理延迟不足以评价 Agentic workload 的性能？

1. 在科研助手例子中，哪些步骤更适合由 CPU 支撑，哪些步骤更依赖 GPU 或 NPU？
1. 智能体调用外部工具后发现方向错误时，需要回滚哪些状态？哪些副作用可能无法简单撤销？
1. 说明可验证性为什么重要，并给出一个中间步骤错误导致最终结果不可靠的例子。

## 第 2 章 Agent CPU 的基本抽象



第 2 章插图：Agent CPU 的七个基本抽象

### 2.1 从回答到执行

传统 CPU 处理的是指令流，核心问题是怎样把一条条指令可靠地执行完。Agent CPU 面对的则是任务流：用户给出目标，环境不断变化，资料来源并不总是完整，结果还需要被解释、检查和修正。因此，Agent CPU 的基础不应只理解为若干模块，而应理解为一组能支撑智能执行的抽象。

本章讨论七个基本抽象：任务、证据、投影、权限、预算、trace 和候选提交。它们对应一条执行链：任务说明为什么做，证据说明凭什么做，投影说明当前看见什么，权限说明能做什么，预算说明花多少资源，trace 说明怎样走到这里，候选提交说明准备交付什么。

设想一名学生要完成“比较两种城市交通方案并给出推荐”。如果他只写结论，读者无法判断依据；如果他把所有资料堆上去，又失去重点。更好的过程是：先界定问题，再选择证据，把复杂材料整理成决策视图，说明约束，控制时间，保留推理线索，最后提交可审查的推荐。Agent CPU 的抽象正是把这种过程系统化。

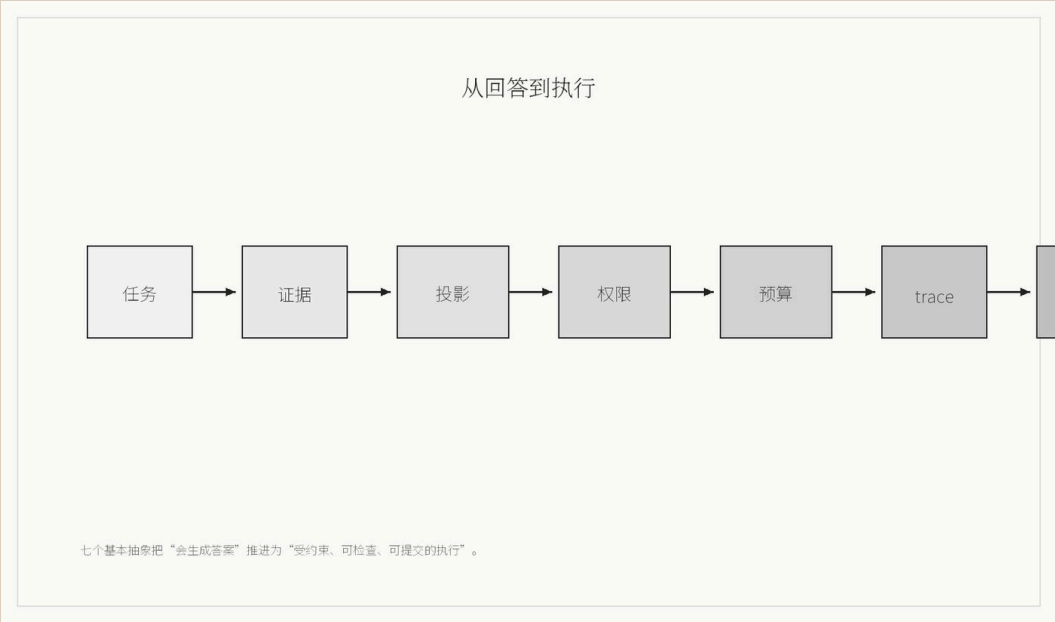


图 2-1：七个基本抽象构成的执行链

## 2.2 任务：可执行的意图

任务是被系统识别、跟踪和完成的执行对象。它不只是自然语言请求，而是包含目标、上下文、成功条件和当前状态的结构。目标回答“要得到什么”，上下文回答“在什么背景中完成”，成功条件回答“怎样算完成”，当前状态回答“已经走到哪一步”。

例如，“写一份实验报告”仍然太宽泛；“基于给定数据，写出包含方法、结果、误差分析和结论的报告，篇幅约一千字”才更像可执行任务。任务越清楚，后续证据选择、预算分配和结果形式越容易稳定。

任务抽象的价值在于防止系统把相似语言误当成相同工作。同样是“总结材料”，给初学者看的总结与给专家看的总结并不一样。好的任务定义像题目卡：它限制答题方向，也保留必要的解题空间。

## 2.3 证据：可检查的依据

证据是支持判断和行动的依据，可以是文本资料、观测数据、计算结果、用户约束，也可以是已经形成的中间结论。证据不同于普通信息：普通信息只是被看见，证据则必须能说明它支持了哪个判断。

证据的质量通常取决于相关性、可靠性、时效性和可解释性。判断“某算法是否适合低延迟场景”时，算法介绍能提供背景，真实延迟测量更接近直接证据；宣传材料可作为线索，独立测试通常更可靠；多年以前的数据也许有参考意义，却未必代表当前条件。

因此，Agent CPU 不能把资料数量当成可靠性。证据抽象要求系统区分“我读到过”与“它证明了”。只有进入推理链、与任务结论发生关系的信息，才是本次执行的证据。

## 2.4 投影：面向任务的视图

环境往往比任务需要的内容复杂得多。投影就是把原始材料转换成面向当前任务的视图。它不是机械压缩，而是根据问题选择、组织和命名信息。

以“为班级春游选择地点”为例，原始材料可能包括天气、费用、交通、安全、兴趣和开放时间。若任务侧重预算，投影会突出门票与车费；若任务侧重安全，投影会突出距离、医疗条件和活动风险；若任务侧重教学意义，投影会突出场馆主题和学习资源。

投影可以类比地图。地铁图突出换乘关系，地形图突出高度变化，天气图突出气压和降水。没有一种地图等于城市本身，但好地图能帮助人行动。可靠投影还应保留可追溯性，避免把摘要中的遗漏误写成事实中的不存在。

|      | 预算优先    | 准确优先    | 教学优先    |
|------|---------|---------|---------|
| 任务问题 | 做成什么    | 证据够不够   | 看见什么    |
| 证据摘取 | 只保留关键事实 | 补足来源与时间 | 突出因果与步骤 |
| 投影视图 | 紧凑上下文   | 完整证据链   | 便于解释的组织 |

同一份材料在不同投影下会形成不同可见视图；证据决定视图是否可靠。

图 2-2：任务、证据与投影的关系

## 2.5 权限：能力的边界

权限定义系统在当前任务中可以读取、调用、修改或提交什么。它把“能够做”与“被允许做”区分开来。一个系统也许能修改文件、发送消息、调用工具，但在具体任务中并不一定获得这些授权。

权限首先保护边界。用户允许系统整理资料，并不等于允许它公开发布；允许系统搜索信息，也不等于允许它替用户作出不可逆决定。其次，权限也让协作更清楚：哪些动作可以直接完成，哪些动作需要确认，哪些动作根本不属于本次任务。

图书馆是一个直观类比。读者可以阅读公共书架上的书，也许可以借出部分书籍，却不能随意改动馆藏目录。管理员权限更大，但同样需要登记。权限不是削弱能力，而是让能力进入有秩序的边界。

## 2.6 预算：有限资源下的选择

预算是对执行资源的约束，包括时间、计算量、上下文容量、调用次数、成本和交互轮次。开放任务总能继续查、继续想、继续改；预算让系统知道何时深入，何时收束。

预算也是策略。简单任务应使用较小预算，快速给出可靠答案；复杂任务则需要把资源分给理解、探索、验证和表达。若全部预算耗在资料收集上，最后可能没有资源组织结果；若过早写作，又可能建立在薄弱证据上。

例如，两小时准备课堂发言，可以先用二十分钟明确题目，四十分钟查证材料，三十分钟形成提纲，二十分钟写讲稿，十分钟检查逻辑。这里的预算不是机械计时，而是把有限资源变成可管理的执行节奏。

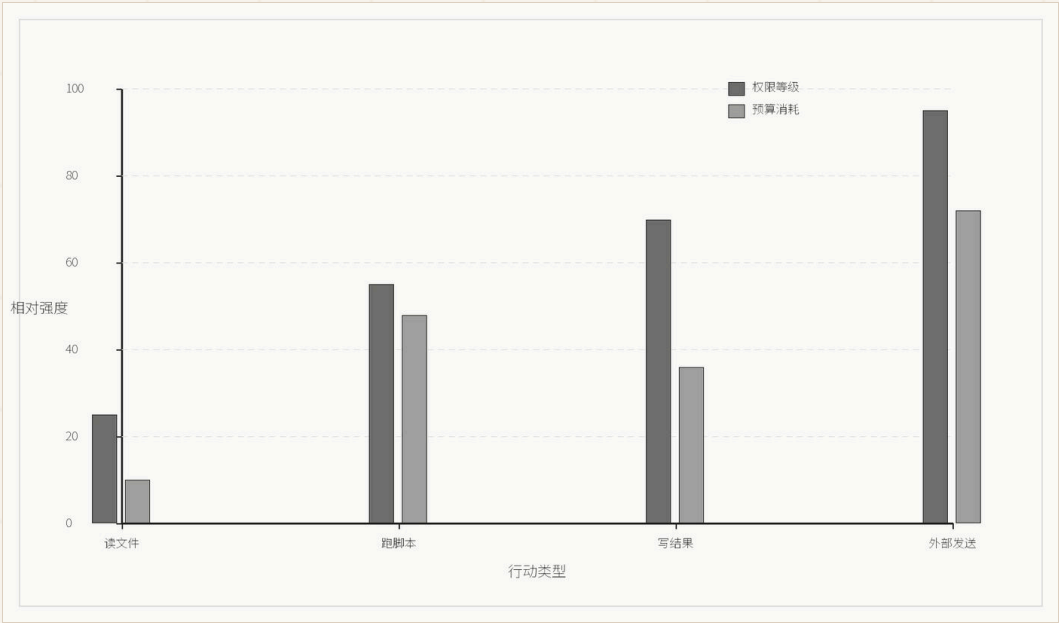


图 2-3：权限边界与预算分配

## 2.7 trace：可复盘的过程

trace 是执行过程留下的关键记录。它说明任务如何被理解，证据如何被选择，投影如何形成，权限和预算如何影响行动，候选提交如何产生。trace 的目标不是暴露所有内部细节，而是让重要过程可以复盘。

没有 trace，系统成功时难以总结经验，失败时难以定位原因。错误结论可能来自任务理解偏差，也可能来自证据不足、投影失真、权限受限或预算耗尽。trace 把这些可能性分开，使评价不只停留在“答对”或“答错”。

教学中，trace 类似解题步骤。教师不仅看最后答案，也看推导是否跳步、条件是否误用、近似是否合理。合适的 trace 应围绕关键选择形成记录，既能说明“为什么走到这里”，又不被无关细节淹没。

## 2.8 候选提交：可审查的结果

候选提交是系统准备交付的结果，但仍保留被检查、比较、修改或拒绝的可能。它不是随手草稿，因为它已经满足当前任务理解下的交付条件；它也不是不可更改的终局，因为开放任务常有多种合理答案。

在春游地点选择任务中，候选提交可以写成：“推荐科技馆，理由是交通稳定、费用可控、教学资源较丰富；若天气良好且希望增加户外活动，可考虑植物园。”这样的结果给出结论，也呈现权衡条件。

候选提交让执行既能收束，又不封闭。系统不必无限探索，也不必把第一个结果直接当作最终事实。好的候选提交应清楚说明结论、证据、适用范围和剩余不确定性。



图 2-4：trace 与候选提交的复盘闭环

2.9 七个抽象的关系

| 抽象    | 回答的问题      | 教学类比    | 常见风险      |
|-------|------------|---------|-----------|
| 任务    | 为什么做、怎样算完成 | 题目卡     | 目标含糊      |
| 证据    | 凭什么判断      | 引用与实验数据 | 把信息堆积当证明  |
| 投影    | 当前需要看见什么   | 地图      | 摘要失真      |
| 权限    | 哪些动作被允许    | 借阅规则    | 能力越界      |
| 预算    | 资源如何分配     | 备课计划    | 过度探索      |
| trace | 过程如何复盘     | 解题步骤    | 记录过少或过杂   |
| 候选提交  | 准备交付什么     | 可审查答案   | 草稿与最终结果混淆 |

七个抽象合在一起，构成 Agent CPU 的最小教学框架。任务启动执行，证据提供支撑，投影塑造视图，权限和预算形成边界，trace 保留过程，候选提交给出可审查结果。它们共同把“会回答”推进为“受约束、可检查、可改进地执行”。

## 2.10 小结

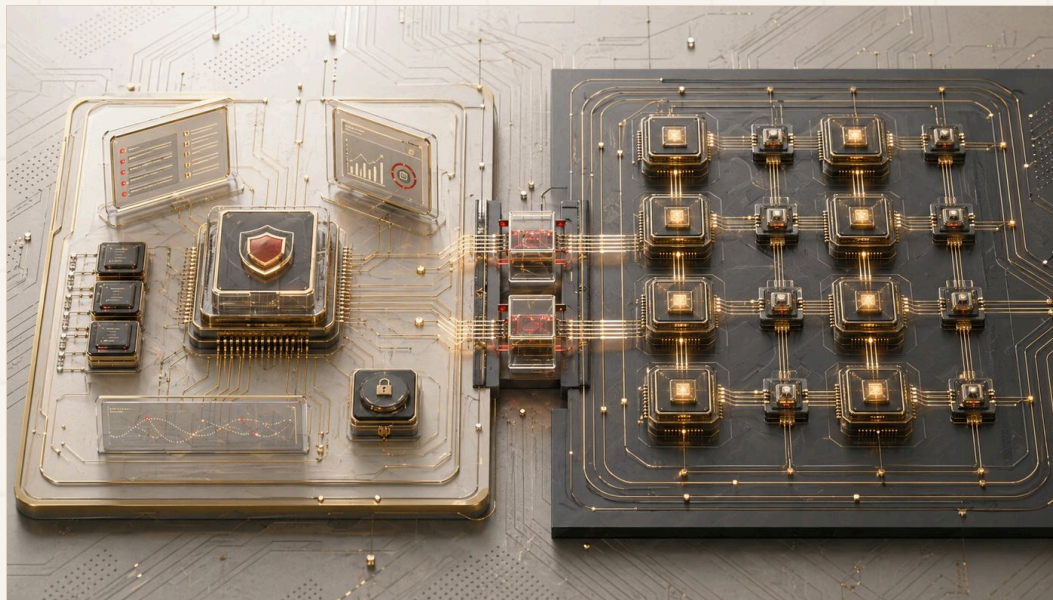
本章定义了 Agent CPU 的七个基本抽象。任务把意图转化为执行对象；证据让判断建立在依据上；投影帮助系统形成任务视图；权限规定行动边界；预算管理有限资源；trace 记录关键过程；候选提交把结果表达为可审查承诺。

理解这些抽象，比记住某个具体实现更重要。实现会变化，而这些抽象回答的是智能执行必须面对的基本问题：目标如何稳定，依据如何可靠，视图如何形成，行动如何受限，资源如何分配，过程如何复盘，结果如何提交。

## 习题

1. 选择一个日常任务，写出它的目标、上下文、成功条件和当前状态。
2. 为同一任务列出三条证据，并判断它们的相关性、可靠性和时效性。
3. 以“选择春游地点”为例，分别构造预算优先、安全优先和教学意义优先的三个投影。
4. 举例说明“系统有能力做某事”与“系统被授权做某事”的区别。
5. 为一个两小时写作任务设计预算分配，并说明最容易超出预算的环节。
6. 写一段简短 trace，记录从任务理解到候选提交的关键步骤。
7. 比较“草稿”“候选提交”和“最终结果”的区别。为什么开放任务需要候选提交？

## 第 3 章 双域架构：Super Domain 与 Agent Domain



第 3 章插图：Super Domain 与 Agent Domain 的双域架构

### 3.1 从特权级到双域

现代计算机体系结构并不是把所有行为都放在同一个平面上执行。即使在最朴素的处理器中，也会区分普通程序能够做什么、操作系统能够做什么，以及硬件在异常、访存和中断时应当如何接管控制权。传统 CPU 中的用户态与内核态，是这种分层思想最典型的体现：用户态负责应用逻辑，内核态负责资源管理、隔离与仲裁。二者并非对立关系，而是同一台机器上两类职责的自然分工。

随着计算任务从确定性程序扩展到具备目标理解、工具调用、长程规划和环境交互的智能任务，类似的分层需求再次出现。Agent CPU 面对的不只是“执行一串指令”，还包括“解释一个目标”“选择一组行动”“在不确定反馈中更新计划”。如果所有环节都处于同一控制层，就很难同时满足灵

活性、可审计性和安全性。因此，Agent CPU 引入双域架构：Super Domain 与 Agent Domain。

Super Domain 可以理解为面向全局秩序的上层域，负责目标边界、资源策略、权限约束、执行监督和最终裁决。Agent Domain 则是面向任务展开的执行域，负责感知上下文、提出候选计划、调用工具、生成中间结果并根据反馈修正路径。二者共同构成 Agent CPU 的基本运行空间：前者提供稳定的制度框架，后者提供活跃的智能执行。

### 3.2 双域架构的基本定义

所谓“双域”，不是简单地把系统切成两个进程或两个模块，而是把智能执行中的两类不同性质的控制权明确区分出来。

Super Domain 的核心问题是“什么可以被允许”。它关心任务是否符合目标边界，资源是否可用，某个工具调用是否需要授权，某个结果是否可以提交，以及发生冲突时应当以哪条规则为准。它更接近传统体系结构中的特权控制层、操作系统内核和平台安全根，但它并不只管理硬件资源，也管理智能行为的边界。

Agent Domain 的核心问题是“怎样把任务做出来”。它关心如何理解输入，如何分解目标，如何搜索方案，如何调用外部能力，如何比较多个候选结果，以及如何把过程推进到可交付状态。它更接近应用程序、运行时系统与协处理器工作流的结合体，但它并不是普通应用层，因为它具备持续推理、反思和自我修正能力。

二者之间存在清晰的接口。Agent Domain 可以提出行动请求、资源请求和结果提交；Super Domain 可以授予、限制、暂停、回滚或要求补充证据。这样，系统既能保持智能执行的开放性，又能把关键控制点收束在可审计、可治理的结构之内。

| 域            | 关注点            | 典型状态            | 失败处理       |
|--------------|----------------|-----------------|------------|
| Super Domain | 目标边界 / 策略 / 审计 | 授权、暂停、提交        | 回滚、拒绝、升级   |
| Agent Domain | 理解 / 规划 / 工具执行 | 候选行动、中间结果       | 重试、换路、请求帮助 |
| 共享接口         | 请求 / 响应 / 证据回传 | task、cap、budget | 记录 trace   |

图 3-1：双域架构：Super Domain 与 Agent Domain的对照关系

### 3.3 为什么双域是自然形成的

从传统 CPU 的发展看，复杂系统往往会在三个压力下产生分层：第一是安全压力，第二是资源压力，第三是复杂性压力。

安全压力来自不同代码之间的信任差异。用户程序不能任意改写页表，不能直接接管中断控制器，也不能随意访问其他进程的内存。类似地，智能任务执行时，计划生成、工具选择和文本输出具有高度灵活性，但这种灵活性不应自动等同于最高权限。Agent Domain 可以大胆探索候选路径，但涉及权限、外部副作用和最终交付时，需要 Super Domain 进行边界确认。

资源压力来自有限计算能力的分配。操作系统需要调度 CPU 时间、内存、文件句柄和网络连接。Agent CPU 中也存在类似问题：推理步数、上下文窗口、工具额度、外部服务调用次数和长期记忆写入机会都是资源。若没有上层域统一管理，单个任务很容易占用过多资源，影响整体吞吐与可预测性。

复杂性压力则来自系统本身的增长。早期程序可以直接面向硬件编写，后来必须依赖操作系统、驱动、运行时和库。智能系统也是如此。一个简单问答可以由单次推理完成，而一个长程任务往往需要检索、规划、执行、验证、修订和归档。把所有逻辑混在一个平面，会使系统难以分析。双域架构把“治理逻辑”和“执行逻辑”分离，使复杂系统仍然保持可解释的骨架。

因此，双域不是人为增加的一层装饰，而是智能计算规模扩大后自然出现的结构选择。它延续了传统体系结构的分层经验，又把分层对象从指令执行扩展到目标执行。

### 3.4 与传统 CPU 特权级的关系

传统 CPU 的特权级把机器状态划分为不同访问范围。普通应用处于低特权级，操作系统内核处于高特权级；当应用需要访问文件、网络或设备时，通过系统调用进入内核，由内核检查参数、执行操作并返回结果。这一机制使普通程序无需拥有全部权力，也能完成复杂任务。

双域架构可以被看作这种思想在 Agent CPU 中的推广。Agent Domain 类似于执行复杂任务的“智能用户态”：它有足够能力进行计划和推理，但它不天然拥有全部控制权。Super Domain 类似于“智能内核态”：它不必亲自完成每个具体步骤，却负责定义可行边界、管理状态转移、处理异常和维护系统一致性。

两者也有重要差异。传统特权级主要围绕硬件资源访问设计，其状态变化较为确定；双域架构则需要处理语义层面的判断。例如，一个文件写入动作在硬件上只是一次 I/O，但在智能任务中可能意味着覆盖用户成果、发布外部信息或改变后续决策依据。Super Domain 因而不仅检查“能不能访问”，还要判断“在当前任务语境下是否应当访问”。

这种关系说明，双域不是对传统特权级的替代，而是对其思想的扩展：从管理机器指令，扩展为管理智能行动。

### 3.5 与操作系统和协处理器思想的关系

操作系统提供了另一个有用类比。应用程序面对的是文件、进程、套接字和虚拟内存等抽象，而不是直接面对磁盘扇区和物理页框。操作系统把复杂硬件包装成稳定接口，并在接口背后完成调度、隔离和错误处理。

Super Domain 也承担类似功能。它把智能执行所依赖的工具、记忆、策略和环境约束包装成可管理的接口。Agent Domain 不需要直接知道每一项资源的底层细节，而是通过请求机制表达意图。Super Domain 根据规则、上下文和资源状态决定如何响应。这种设计降低了执行域的负担，也使系统治理不必散落在每个任务步骤中。

协处理器思想则帮助我们理解 Agent Domain 的价值。图形处理器、张量加速器和安全模块都不是主处理器的简单附庸，而是面向特定问题提供高效能力的执行单元。主系统把合适的工作交给协处理器，并通过接口接收结果。Agent Domain 在双域架构中也具有这种特征：它专长于语义推理、计划生成、候选比较和工具链组织，是智能任务的主要生产力来源。

因此，双域架构可以被概括为：以 Super Domain 维持秩序，以 Agent Domain 释放智能执行能力；以接口连接二者，以审计和反馈维持整体稳定。

|      | Super Domain | 接口    | Agent Domain |
|------|--------------|-------|--------------|
| 目标约束 | 设定边界         | 传递约束  | 解释并执行        |
| 资源治理 | 配额与权限        | 请求/授予 | 选择行动         |
| 异常处理 | 接管高风险        | 发出信号  | 局部修正         |
| 结果审计 | 最终裁决         | 候选提交  | 组织输出         |

图 3-2：双域架构：Super Domain 与 Agent Domain的关键概念对应

### 3.6 Super Domain 的职责

Super Domain 的职责可以分为四类。

第一，目标约束。它保存任务的根目标、用户意图、系统规则和不可越过的边界。当 Agent Domain 产生多个解释或计划时，Super Domain 提供判断基准，防止执行路径偏离原始目标。

第二，权限与资源管理。它决定哪些工具可用，哪些操作需要确认，哪些资源有额度限制，哪些状态需要保护。权限并不只是一张静态表，而是结合任务阶段、上下文风险和历史行为动态判断。

第三，异常处理。智能执行常常遇到不完整信息、工具失败、结果冲突和外部环境变化。Agent Domain 可以尝试局部修正，但当异常影响目标边界、资源安全或结果可信度时，需要由 Super Domain 接管或重新设定执行条件。

第四，提交与审计。最终结果不是简单地由最后一次生成决定，而应经过可追踪的提交过程。Super Domain 记录关键决策、重要证据、权限变化和结果确认，使系统能够回看“为什么这样做”。

### 3.7 Agent Domain 的职责

Agent Domain 是任务推进的主要场所。它的第一项职责是理解输入，把自然语言目标、环境状态和已有材料转化为可执行的任务表示。这个过程类似于把高级语言编译为中间表示，但它处理的是语义、意图和约束。

第二项职责是规划。Agent Domain 将复杂目标分解为若干步骤，形成候选路径，并根据反馈不断调整顺序。优秀的规划并不意味着一次性得到完美路线，而是能够在执行中保持方向感。

第三项职责是行动。行动可以是内部推理，也可以是调用工具、检索资料、运行程序、生成文件或请求外部信息。Agent Domain 负责把计划落实为具体动作，并把动作结果重新纳入上下文。

第四项职责是自我校验。由于智能执行存在不确定性，Agent Domain 需要比较输出与目标之间的差距，发现遗漏、矛盾和低置信度区域，再决定是否继续补充工作。这种校验不等同于最终裁决，但它能显著提高提交给 Super Domain 的结果质量。

### 3.8 双域之间的典型交互

双域交互通常围绕“请求、授权、执行、反馈、提交”展开。一个任务开始时，Super Domain 接收根目标并建立边界；Agent Domain 读取目标后形成计划；当计划需要敏感资源或外部副作用时，Agent Domain 发起请求；Super Domain 根据策略做出授权或限制；Agent Domain 执行并返回证据；最终由 Super Domain 判断是否满足提交条件。

| 环节    | Super Domain 关注点 | Agent Domain 关注点 |
|-------|------------------|------------------|
| 任务初始化 | 根目标、边界、优先级       | 语义理解、任务拆解        |
| 计划形成  | 是否符合约束           | 候选路径、步骤排序        |
| 资源请求  | 权限、额度、风险         | 工具选择、参数构造        |
| 执行推进  | 状态一致性、异常级别       | 行动结果、局部修正        |
| 结果提交  | 可审计性、完成条件        | 输出组织、证据汇总        |

这张表体现了双域的互补关系。Super Domain 并不替 Agent Domain 完成所有推理，Agent Domain 也不越过 Super Domain 自行决定关键边界。前者像稳定的制度结构，后者像活跃的执行智能。只有二者配合，Agent CPU 才能在开放任务中保持可靠。

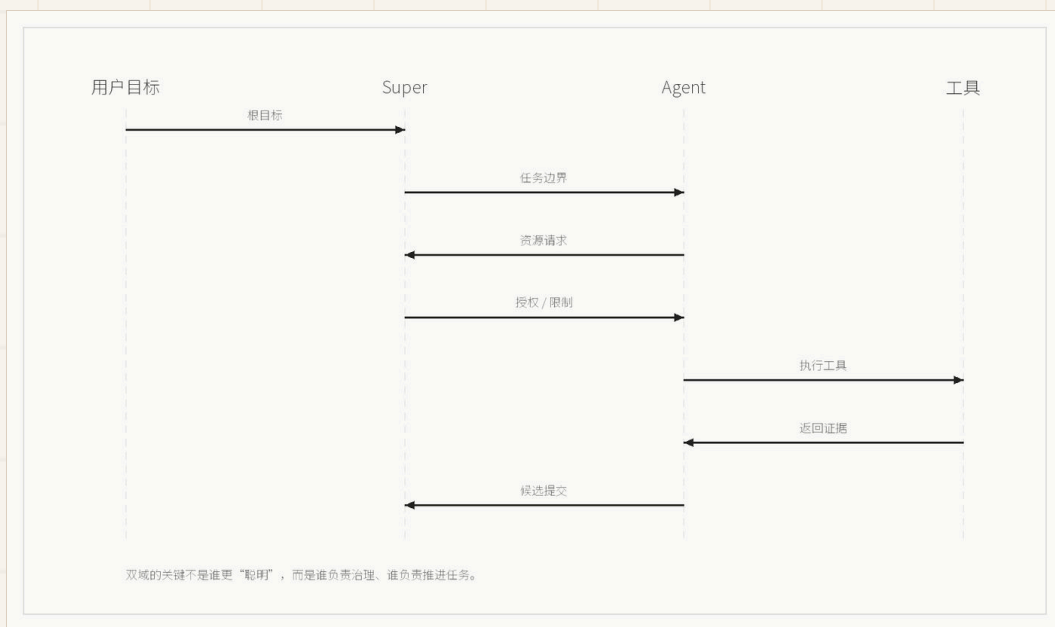


图 3-4：双域架构中的职责分工与交互路径

### 3.9 小结

本章介绍了 Agent CPU 的双域架构。双域架构把智能计算中的全局治理与任务执行分离为 Super Domain 与 Agent Domain。Super Domain 负责目标边界、权限资源、异常处理和结果审计；Agent Domain 负责语义理解、任务规划、行动执行和自我校验。

这种划分来自传统体系结构的连续演化。CPU 特权级说明了不同信任层次需要不同控制权；操作系统说明了复杂资源需要统一抽象与调度；协处理器说明了专门能力可以通过接口融入整体系统。Agent CPU 将这些思想推进到语义行动层，使智能任务既能灵活展开，又能在可管理的框架中完成。

理解双域架构，是理解 Agent CPU 后续机制的基础。后续章节中的任务恢复、记忆系统、候选提交、工具调用和性能评价，都可以看作双域分工在不同部件上的展开。

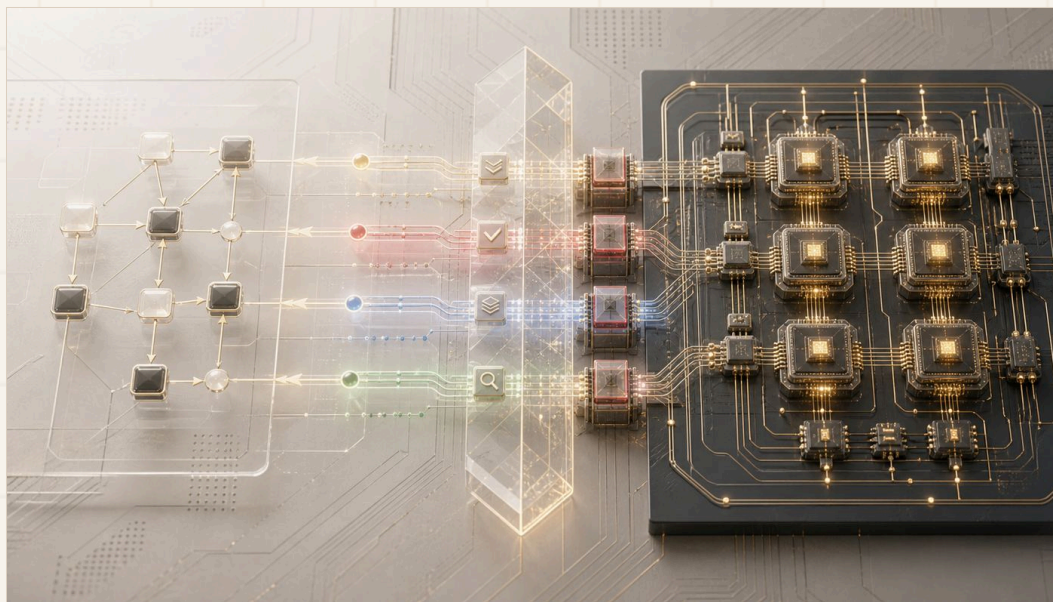


图 3-3：双域架构：Super Domain 与 Agent Domain 的阶段推进

### 3.10 习题

1. 为什么传统 CPU 要区分用户态和内核态？这种区分对理解 Super Domain 与 Agent Domain 有什么帮助？
1. 试举一个智能任务的例子，说明 Agent Domain 可以负责哪些步骤，Super Domain 应当在哪些位置介入。
1. 双域架构中的权限管理与普通操作系统权限管理有什么相同点和不同点？
1. 如果一个 Agent Domain 生成了两个互相冲突的计划，Super Domain 可以依据哪些信息做出选择或要求修订？
1. 结合本章内容，说明为什么双域架构不是削弱 Agent Domain 的能力，而是让其能力更适合在复杂系统中稳定发挥。

## 第 4 章 APU-IR: Agent 与硬件之间的契约



第 4 章插图：APU-IR 将 Agent 任务语义映射到硬件

在计算机体系结构课程中，指令集体系结构常被称为软件与硬件之间的契约。软件不必知道加法器如何布线、流水线如何旁路、缓存如何替换，只要处理器正确实现指令集，程序便能按约定运行。进入 Agent CPU 的语境后，系统要执行的不再只是单条算术指令或一次内存访问，而是带有目标、上下文、工具调用、权限边界、成本约束和可追溯证据的智能任务。因此，Agent 与底层执行平台之间也需要一种新的契约形式。本章讨论的 APU-IR，就是这种契约的核心表示。

APU-IR 可以理解为面向 Agent Processing Unit 的中间表示。它不是某种聊天记录格式，也不是把自然语言简单包装成结构化文本；它是一种把“要做什么、可用什么、不能做什么、花费多少、依据何在、执行过程如何留下痕迹”同时表达出来的任务级表示。它使 Agent 的意图能够被编译、调度、审计和硬件加速，也使硬件平台能够在明确边界内执行开放任务。

### 4.1 从 ISA 到 APU-IR

传统计算机系统通常通过多层接口来管理复杂性。最底层的 ISA 定义机器可见的指令、寄存器、内存模型和异常行为；ABI 定义过程调用、二进制对象、系统调用和运行时约定；编译器 IR 则位于高级语言与目标机器之间，用于优化和变换程序。三者共同构成了现代软件栈的稳定地基。

如果把这套脉络迁移到 Agent CPU，会发现原有接口仍然重要，但已经不够。一个 Agent 任务往往不是“执行函数  $f(x)$ ”，而是“在给定目标下，读取若干上下文，调用受限工具，形成可验证输出，并在预算内完成”。这类任务包含传统指令难以直接表达的要素：任务之间的依赖关系、资源和时间预算、外部工具权限、输出证据链、执行 trace，以及失败后的恢复策略。

APU-IR 正是在这一空缺处出现。它向上承接自然语言任务、工作流描述和应用 API，向下连接运行时、调度器、内存系统、工具沙箱和专用加速单元。它的作用不是取代 ISA，而是在 ISA 之上增加任务语义层，使 Agent CPU 能够理解并执行“任务”这一新的基本工作单位。

| 层次     | 主要对象               | 典型问题            | 在 Agent CPU 中的对应意义   |
|--------|--------------------|-----------------|----------------------|
| ISA    | 指令、寄存器、内存          | 一条机器指令如何执行      | 提供底层计算与访存能力          |
| ABI    | 调用约定、系统接口          | 模块如何互相调用        | 规范工具、运行时和外部服务接口      |
| 编译器 IR | 控制流、数据流、类型         | 程序如何优化和映射       | 支持任务编译、重排和静态检查       |
| APU-IR | 任务图、权限、预算、证据、trace | Agent 任务如何被可靠执行 | 定义 Agent 与执行平台的任务级契约 |

从这个角度看，APU-IR 并不是凭空发明的一层格式，而是计算机接口思想在 Agent 时代的自然延伸。它把“程序的正确执行”扩展为“任务在约束、证据和责任边界内的正确执行”。

## 4.2 APU-IR 表达什么

一个完整的 APU-IR 程序通常围绕任务图展开。任务图中的节点表示可执行单元，例如检索资料、调用模型、运行工具、验证输出或生成报告；边表示依赖关系，例如某个节点必须等待检索结果，某个验证节点必须读取生成节点的输出。与普通有向无环图不同，APU-IR 的任务图不仅描述

先后顺序，还携带权限、预算和证据要求。

权限用于说明节点可以访问哪些资源。一个节点可能只能读取给定文档，另一个节点可以调用数据库查询接口，还有一个节点可以写入最终报告。权限不是附属说明，而是可执行平台必须检查的约束。这样，Agent 不再拥有一个笼统而模糊的“全能工具箱”，而是在每个任务阶段获得最小必要能力。

预算用于限制任务的消耗。预算可以是时间、计算量、模型调用次数、外部 API 次数、能耗、费用或内存占用。传统程序也会受到资源限制，但这些限制通常在操作系统或运行时外部施加；APU-IR 将预算提升为任务契约的一部分，使调度器可以在计划阶段估计成本，在执行阶段动态控制消耗，在异常阶段决定是否降级、重试或中止。

证据用于说明输出必须依据什么。对于 Agent CPU 而言，输出不仅要“看起来合理”，还要能够追溯到输入、工具结果、模型中间判断或验证步骤。APU-IR 可以要求某个结论绑定到若干来源片段，也可以要求某个判断经过独立验证节点确认。证据机制将 Agent 的生成行为转化为可检查的计算过程。

trace 则记录任务如何执行。它包括节点开始与结束时间、使用的权限、消耗的预算、读取的输入、产生的输出摘要、失败与重试信息、验证结果等。trace 的意义不只是调试，更是系统治理的一部分：它让开发者能够复现问题，让调度器能够学习代价模型，让审计者能够理解输出是如何产生的。

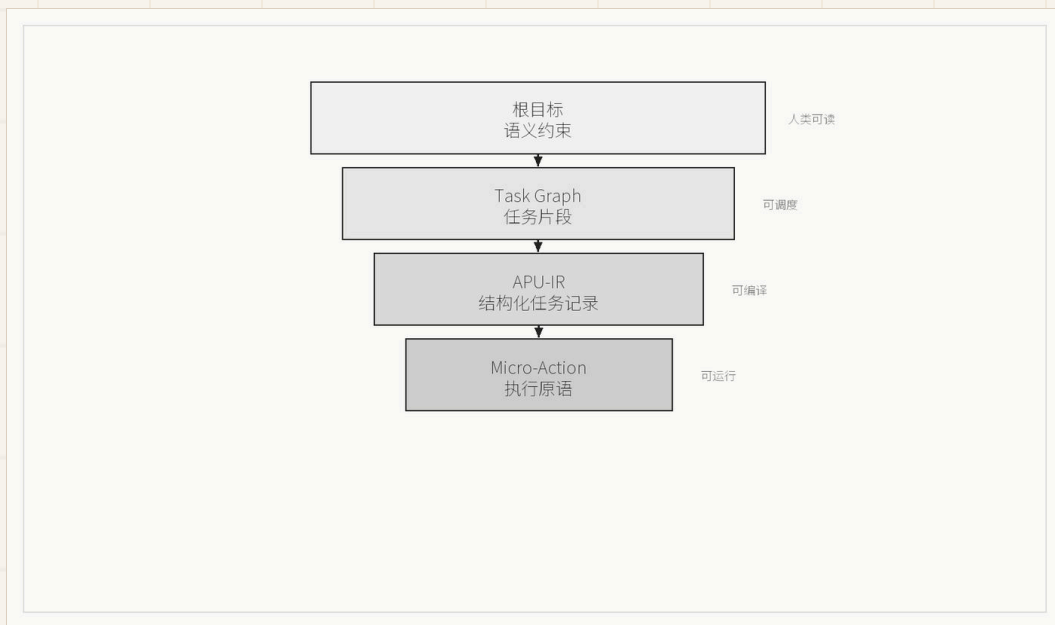


图 4-1：APU-IR：Agent 与硬件之间的契约的层次展开

### 4.3 任务图：从线性调用到可调度结构

在简单应用中，Agent 常被实现为“提示词输入、模型输出、工具调用、再输入模型”的循环。这种循环易于理解，但不利于系统优化。APU-IR 将循环中的隐含步骤显式化为任务图，使执行平台能够进行静态分析和动态调度。

例如，一项技术调研任务可以被分解为四类节点：资料检索、候选资料筛选、观点综合、结果验证。检索节点之间可能相互独立，可以并行执行；筛选节点依赖检索结果；综合节点依赖筛选后的证据；验证节点读取综合结果并检查引用是否充分。任务图一旦形成，运行时就能识别并行机会、缓存中间结果、隔离高风险工具调用，并在预算不足时优先保留关键路径。

任务图还使失败处理更加明确。若某个检索节点失败，系统可以选择使用备用来源；若验证节点失败，系统可以退回综合节点重新生成；若预算超限，系统可以按照优先级裁剪低价值分支。这里的关键在于，失败不再只是一次调用异常，而是图中某个节点状态的变化。APU-IR 将这种状态变化纳入契约，从而使恢复策略可以被系统化执行。

## 4.4 权限：把能力变成可检查边界

Agent 的强大来自工具使用能力，但风险也常来自工具使用能力。如果一个 Agent 既能读取私有数据，又能访问外部网络，还能写入生产系统，那么一次错误推理就可能造成越权泄露或错误操作。APU-IR 的权限模型试图在任务表示层解决这一问题。

权限应当具有三个特征。第一，权限要绑定到节点，而不是笼统绑定到整个 Agent。第二，权限要尽量具体，例如“读取某个输入集合”优于“读取所有文件”，“调用某个只读查询接口”优于“访问全部数据库”。第三，权限要可由运行时检查，而不是只存在于自然语言约定中。

在硬件和系统实现上，权限信息可以驱动沙箱、内存保护、工具网关和安全监控。执行平台在调度节点前检查权限，在运行节点时限制可见资源，在节点结束后将权限使用情况写入 trace。这样，APU-IR 不仅描述任务，而且参与安全执行。



图 4-2：APU-IR：Agent 与硬件之间的契约的扇出关系

## 4.5 预算：让开放任务具备工程边界

Agent 任务具有开放性。一个问题可以检索十条资料，也可以检索一千条；一次推理可以很快给出答案，也可以不断反思和重试。没有预算，开放任务很难进入工程系统。APU-IR 用预算字段把开放性限制在可管理范围内。

预算可以分为硬预算和软预算。硬预算一旦超过即必须停止，例如最大费用、最长执行时间或禁止超过的权限边界。软预算用于指导调度，例如希望在较短时间内完成、优先减少外部调用、尽量保留高可信证据。两类预算共同作用：硬预算保证系统不越界，软预算帮助系统在多个可行方案中选择更优执行路径。

预算还可以沿任务图分配。根任务拥有总预算，子任务获得预算切片；运行时根据节点重要性、历史代价和当前进展动态调整。若验证节点比预期消耗更多，系统可以压缩后续润色节点的预算；若检索节点已经获得足够证据，系统可以提前结束剩余检索分支。通过这种方式，预算从外部限制转化为调度决策的一部分。

## 4.6 证据与 trace：让执行结果可追溯

传统程序的正确性通常通过输入输出关系、测试和形式化验证来讨论。Agent 任务的正确性更复杂，因为输出可能包含自然语言判断、概率推理和外部知识。APU-IR 因而强调证据和 trace。

证据回答“为什么可以这样说”。它要求关键结论能够连接到输入片段、检索结果、工具返回值或验证节点。证据不保证结论一定正确，但它提供了检查路径。没有证据的输出只能被当作生成结果；带有证据的输出则可以被复核、比较和审计。

trace 回答“系统实际做了什么”。同一份 APU-IR 在不同时间、不同硬件、不同缓存状态下可能产生不同执行路径。trace 记录这些路径，使开发者能够分析延迟来源、预算消耗和错误传播。对于生产系统，trace 还可以支持责任划分：是输入不足、工具失败、模型判断错误，还是权限配置过宽。

证据和 trace 的关系类似于“内容依据”和“执行日志”。证据面向结果可信度，trace 面向过程可解释性。二者结合，APU-IR 才能把 Agent 的行为从黑箱调用推进到可观察、可治理的系统执行。

## 4.7 一个简短示例

下面的例子只展示 APU-IR 的思想，而不是完整语法规范。设想一个任务：根据三份输入材料生成一页摘要，并要求每个主要结论有来源依据。

```
task: summarize_policy_notes
goal: 生成一页中文摘要，列出主要结论与依据
graph:
  - read_inputs: 读取给定三份材料
  - extract_claims: 从材料中抽取候选结论，依赖 read_inputs
  - verify_claims: 检查每个结论是否有材料依据，依赖 extract_claims
  - write_summary: 生成最终摘要，依赖 verify_claims
permissions:
  read_inputs: read(input_documents)
  write_summary: write(final_answer)
budget:
  time: 60s
  model_calls: 4
evidence:
  write_summary: 每个主要结论至少绑定一个输入片段
trace:
  record: 节点状态、预算消耗、证据绑定、验证失败项
```

这个例子中，重点不是字段名称，而是契约内容。任务被拆成图，节点权限被限定，预算提前声明，证据要求随输出传播，trace 要求随执行生成。执行平台据此可以决定哪些节点可并行、哪些资源可访问、何时停止、如何验证输出，以及如何在事后复盘。

## 4.8 APU-IR 与硬件实现的关系

APU-IR 是任务级表示，但它并不只属于软件。对于 Agent CPU，硬件和运行时可以围绕 APU-IR 进行协同设计。任务图为调度器提供并行结构，权限为保护机制提供检查依据，预算为资源管理单元提供控制目标，证据和 trace 为可观测性硬件提供记录边界。

例如，硬件可以为任务节点维护轻量状态表，记录节点是否就绪、运行或阻塞；可以为模型调用、向量检索和工具访问提供专用队列；可以将预算计数器与节点上下文绑定；可以为 trace 生成低开销事件流。这样，APU-IR 不只是编译器内部的一种表示，而是贯穿编译、运行、调度和审计

的共同语言。

当然，APU-IR 不要求所有语义都由硬件直接理解。就像现代处理器不直接理解高级语言一样，Agent CPU 也可以通过编译器、运行时和微码层逐步解释 APU-IR。关键在于，APU-IR 把任务语义稳定地暴露出来，使不同实现可以在同一契约下竞争性能、可靠性和安全性。

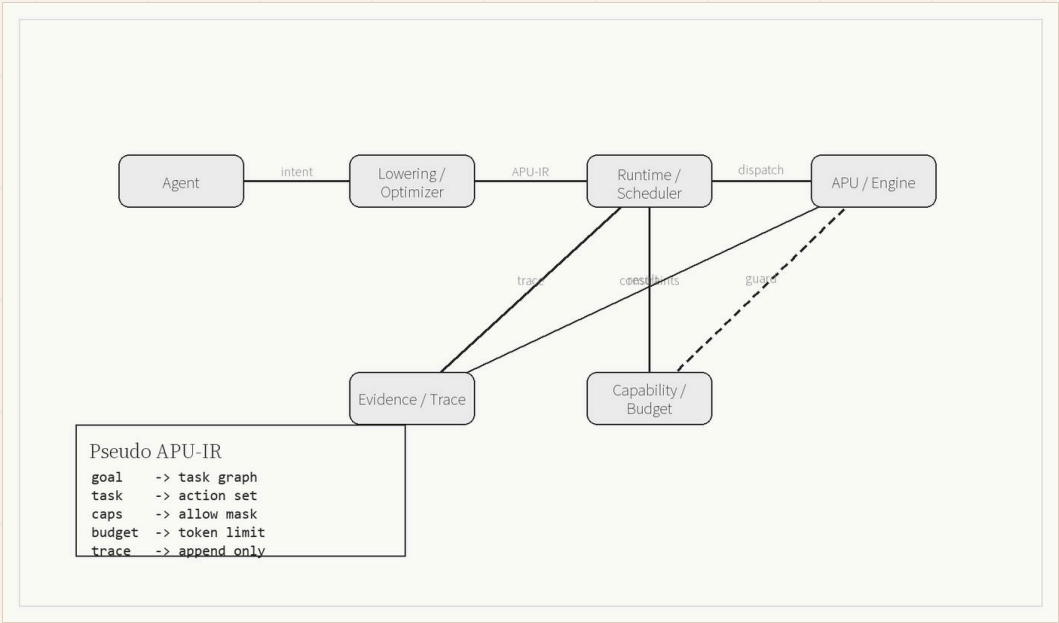


图 4-4：APU-IR 作为 Agent 与硬件之间的任务契约

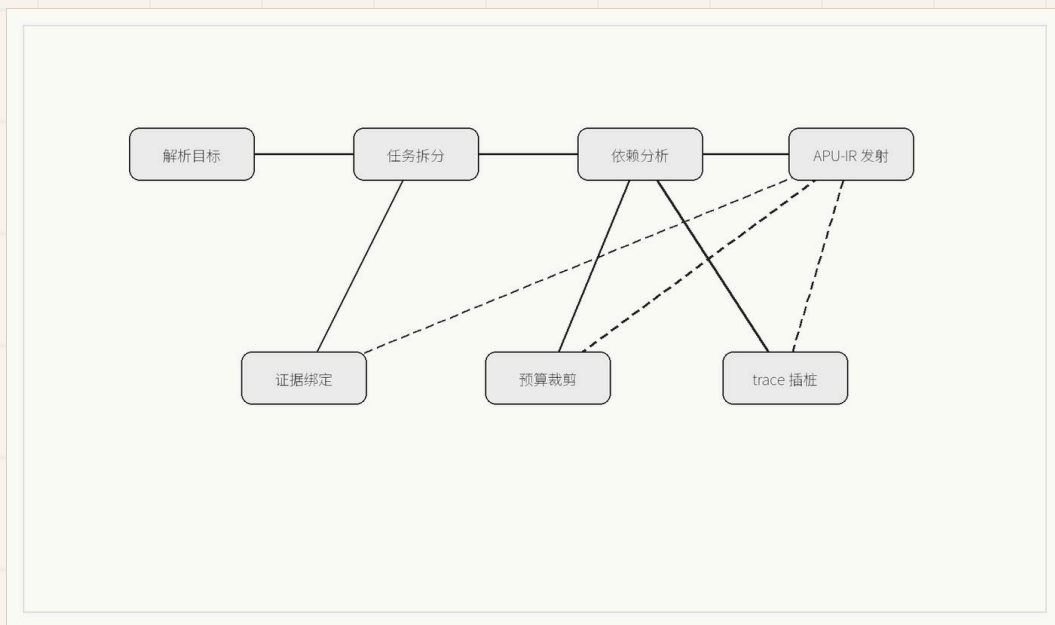


图 4-3: APU-IR: Agent 与硬件之间的契约的概念展开路径

## 4.9 小结

本章从 ISA、ABI 和编译器 IR 的传统教学脉络出发，引出了 APU-IR 的必要性。APU-IR 是 Agent 与执行平台之间的任务级契约，它把目标、任务图、权限、预算、证据和 trace 统一到一种可分析、可调度、可审计的表示中。

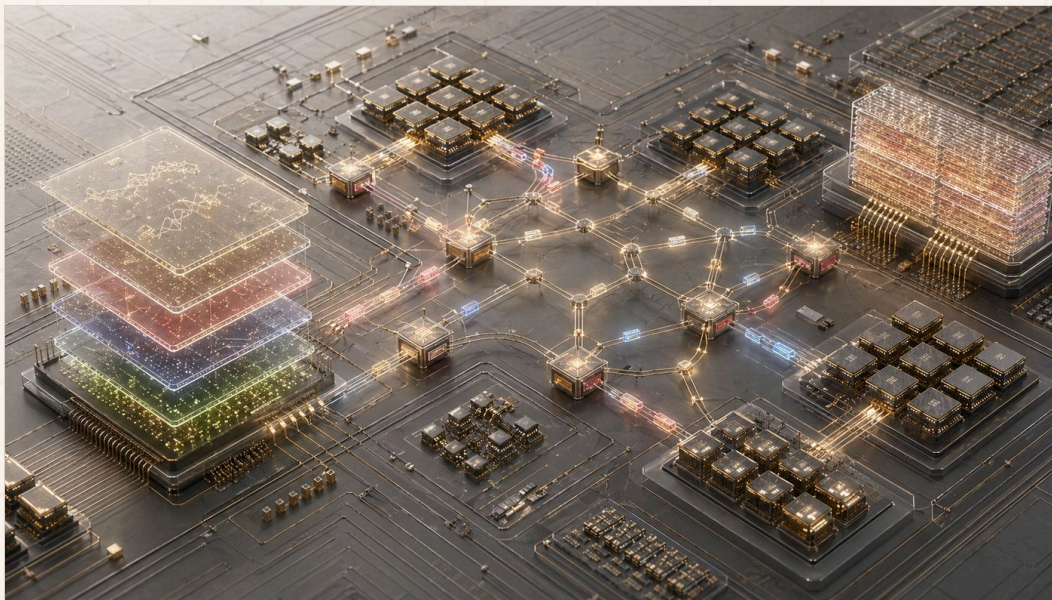
APU-IR 的核心价值在于显式化。它把隐含在提示词和工具循环中的结构显式化为任务图，把模糊能力显式化为权限，把开放消耗显式化为预算，把生成依据显式化为证据，把执行过程显式化为 trace。正因为这些要素被表达出来，Agent CPU 才能对复杂任务进行安全调度、资源控制和结果复核。

在后续章节中，APU-IR 可以继续向下连接任务调度、上下文内存、工具接口和安全机制。理解 APU-IR，就是理解 Agent CPU 如何把“智能行为”转化为可工程化执行的系统对象。

## 习题

1. 试比较 ISA、ABI、编译器 IR 和 APU-IR 的作用边界。为什么说 APU-IR 不是 ISA 的替代品，而是任务语义层的补充？
1. 选择一个日常 Agent 任务，例如整理会议纪要、检索技术资料或生成实验报告，将其拆分为 4 到 6 个任务图节点，并标出节点之间的依赖关系。
1. 对上一题中的任务图，为每个节点设计最小必要权限。说明哪些权限如果授予过宽，可能带来安全或隐私风险。
1. 设计一个包含时间、费用和工具调用次数的预算方案。若任务执行到一半时预算不足，你会优先保留哪些节点，裁剪哪些节点？为什么？
1. 解释证据和 trace 的区别。请举例说明：一个输出有证据但 trace 不完整时会产生什么问题；一个 trace 完整但证据不足时又会产生什么问题。

## 第 5 章 Memory 与 NoC：从状态到数据移动



第 5 章插图：Memory 与 NoC 中的显式数据移动

处理器并不只是在执行指令，也在不断搬运状态。寄存器中的操作数、缓存中的数组元素、片上存储器中的中间块、主存中的对象图，以及外设产生的输入流，都要在恰当的时间到达恰当的计算单元。传统计算机体系结构把这一问题称为内存层次与互连设计；在 Agent CPU 中，它进一步扩展为 ledger、context、artifact 与 trace 等智能体状态的组织、定位、移动与提交。本章从经典内存系统出发，说明 Agent CPU 如何把“数据在哪里”和“数据如何流动”变成体系结构中的一等问题。

### 5.1 内存层次的基本动机

理想的存储器应当容量无限、访问立即完成、能耗为零，并且能够被所有计算单元同时使用。现实系统不可能满足这些条件，因此现代处理器采用分层结构：越靠近执行单元，速度越快、容量越小、单位比特成本越高；越靠近外部存储，容量越大、延迟越高、带宽与能耗压力越明显。

典型层次包括寄存器、一级缓存、二级或三级缓存、片上共享存储、主存、固态硬盘和远端存储。每一层都承担“保留最近可能被使用的数据”的任务。程序访问数据并非均匀随机，而是呈现局部性。时间局部性表示刚访问过的数据很可能很快再次访问；空间局部性表示访问某个地址后，附近地址也可能被访问。缓存正是利用这两种局部性，把高频访问吸收到靠近计算的位置。

Agent CPU 继承这一原则，但对象不再只是字节数组。智能体运行时还会反复读取任务目标、约束、工具返回、候选计划、记忆摘要、执行日志和验证证据。这些信息同样具有局部性：当前推理轮次会频繁访问相同的上下文片段；相邻步骤会共享相同 artifact；一次回滚或复盘会沿 trace 访问连续事件。因此，Agent CPU 的内存层次既要服务普通数据，也要服务可解释、可恢复、可审计的智能体状态。

## 5.2 缓存：从地址命中到语义命中

缓存把主存中的数据块复制到更快的位置，并用标签判断请求是否命中。命中时，访问在近端完成；未命中时，系统从下一级取回数据，可能还要替换已有缓存行。传统缓存设计关注块大小、相联度、替换策略、一致性协议与写策略。它的核心目标是降低平均访问延迟，同时在多核环境中维护数据一致性。

在 Agent CPU 中，缓存概念被推广为多种状态的近端副本。context cache 保存当前任务窗口中最相关的上下文片段；artifact cache 保存可复用的中间产物，如解析结果、检索摘要、程序分析结果或规划图；trace cache 保存近期执行事件，便于分支恢复、错误定位和候选路径比较；ledger cache 则保存尚需快速检查的承诺、约束与提交记录。

这里的“命中”不仅是地址相等，也可以是语义相近、版本匹配和权限满足。一个上下文片段是否可复用，取决于它是否属于当前任务、是否仍然新鲜、是否与当前策略一致，以及是否已经被后续事件覆盖。因而 Agent CPU 的缓存标签可包含对象标识、版本号、时间戳、来源、置信度、可见范围和依赖关系。它仍然是体系结构缓存，只是标签空间从物理地址扩展到智能体状态的结构化描述。

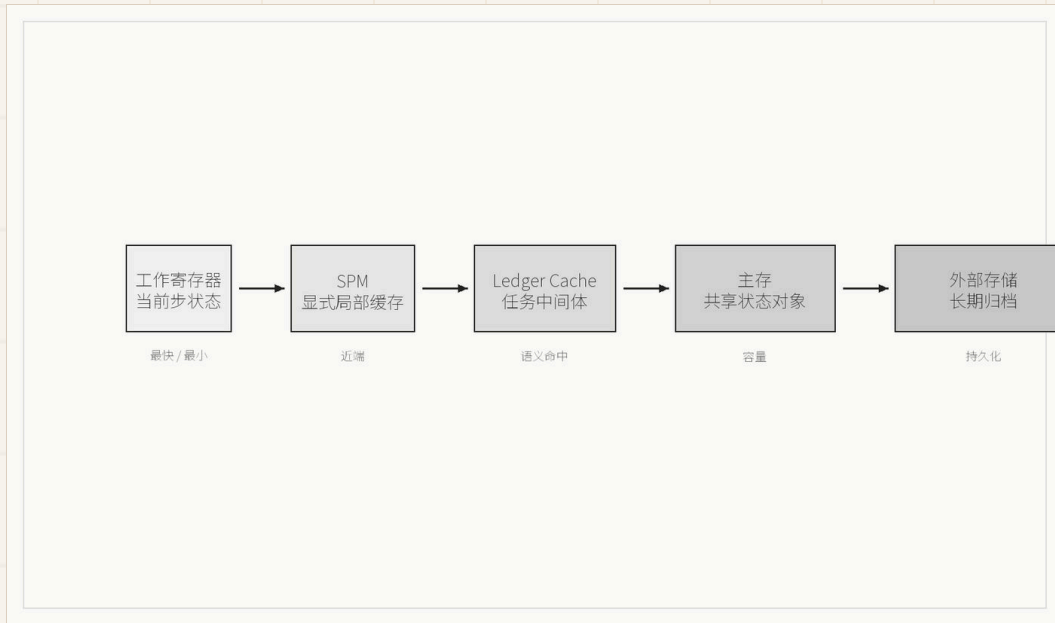


图 5-1: Memory 与 NoC: 从状态到数据移动的层次展开

### 5.3 SPM 与显式数据编排

缓存通常由硬件自动管理，程序只间接影响其行为。Scratchpad Memory，简称 SPM，则是一块由软件或运行时显式管理的近端存储。它没有复杂的标签匹配和替换机制，优势是访问延迟可预测、能耗较低、控制明确。许多嵌入式处理器、图像处理器和专用加速器都使用 SPM 承载热点数据块。

对于 Agent CPU，SPM 的价值在于承载“本轮推理必须稳定可见”的工作集。例如，调度器可以把当前任务说明、最近若干条 ledger 项、关键 artifact 索引和候选动作列表放入片上 SPM，使推理核心在一个阶段内以确定延迟访问它们。与缓存不同，SPM 的内容通常由编译器、运行时或微内核显式搬入和搬出，因此更适合表达阶段性边界。

这种显式编排也有助于可恢复执行。当一个推理阶段开始时，系统可以把阶段输入固定到 SPM；阶段结束时，再把结果提交到 ledger 或写入 artifact store。若阶段失败，未提交的 SPM 内容可以被丢弃，已提交状态保持不变。于是，SPM 不只是快速存储，也成为状态隔离和事务边界的一部分。

5.4 DMA：让数据移动脱离执行核心

直接内存访问 DMA 允许专门的数据搬运单元在不占用通用执行核心的情况下传输数据。传统系统中，DMA 常用于磁盘、网络、显示和加速器之间的大块数据搬运。CPU 配置源地址、目的地址、长度和权限后，DMA 引擎负责完成传输，并在结束时发出中断或写回状态。

Agent CPU 中的数据移动更频繁，也更结构化。一次智能体步骤可能需要从长期记忆取回若干上下文片段，把工具输出转换成 artifact，把候选计划写入缓冲区，再把最终决策追加到 ledger。若这些搬运全部由执行核心逐字节完成，推理吞吐会被状态管理拖慢。因此，Agent CPU 可使用状态 DMA：它搬运的不只是连续字节，也可以是带元数据的对象块、上下文窗口、trace 区间或 artifact 依赖子图。

状态 DMA 的描述符需要表达更多信息。除了源、目的和长度，还应包含对象类型、版本条件、压缩方式、访问权限、是否需要校验摘要、完成后是否触发提交事件等字段。这样，执行核心发起“取回与当前目标相关的上下文片段”或“把本轮 trace 归档到持久层”时，可以把细节交给搬运单元，在等待期间继续进行调度、验证或下一步规划。



图 5-2：Memory 与 NoC：从状态到数据移动的阶段推进

## 5.5 NoC：片上系统的数据道路

当芯片内有多个核心、缓存、加速器、存储控制器和 I/O 单元时，简单总线会遇到带宽、仲裁和时序瓶颈。Network-on-Chip，简称 NoC，把片上通信组织成网络。节点通过路由器连接，消息以包的形式在链路上传输。常见拓扑包括环、网格、树和片上交叉网络。NoC 设计关注延迟、吞吐、公平性、死锁避免、服务质量和功耗。

传统 NoC 主要传输读写请求、缓存一致性消息和 DMA 数据流。Agent CPU 的 NoC 还要承载智能体状态流：context 请求、artifact 读写、trace 追加、ledger 查询、候选分支同步、权限检查结果和验证反馈。不同流量的实时性不同。推理核心等待的 context 片段属于低延迟流；大规模 artifact 归档属于高带宽流；ledger 提交属于强顺序和高可靠流；trace 采样可以在带宽紧张时降采样或延迟写回。

因此，Agent CPU 的 NoC 往往需要区分消息类别和优先级。它可以为 ledger 提交保留有序通道，为 context cache 填充提供低延迟通道，为 artifact 搬运提供批量通道，为 trace 流提供可压缩通道。这样，系统不会因为大量日志写入阻塞关键决策，也不会因为上下文取回挤占长期归档的全部带宽。

## 5.6 四类状态的数据移动

Agent CPU 中的状态可以粗略分为四类：ledger、context、artifact 和 trace。它们都在内存层次中占据位置，但访问模式和一致性要求不同。

| 状态类型     | 主要含义              | 典型访问模式      | 体系结构关注点        |
|----------|-------------------|-------------|----------------|
| ledger   | 已承诺的目标、约束、决策和提交记录 | 追加、查询、校验、回放 | 顺序性、不可随意覆盖、可审计 |
| context  | 当前推理所需的任务背景和相关记忆  | 窗口化读取、相关性更新 | 新鲜度、容量控制、语义命中  |
| artifact | 工具或模型产生的中间与最终产物   | 块读写、依赖追踪、复用 | 版本、引用、批量搬运     |
| trace    | 执行过程中的事件序列和证据链    | 追加、采样、压缩、回溯 | 时间顺序、低开销、可恢复   |

ledger 更像体系结构中的提交日志。它要求写入顺序清晰，不能被普通缓存替换策略随意隐藏。context 更像工作集，它经常被选择、裁剪和重排，适合多级缓存和 SPM 协同。artifact 具有文件和对象的双重特征，常常体积较大，适合 DMA 与 NoC 批量移动。trace 则像高频事件流，需要在可观察性与带宽之间取得平衡。

把这四类状态区分开，并不是为了增加概念负担，而是为了让硬件和运行时按不同规律优化。若把所有状态都看成普通内存块，系统很难知道哪些写入必须立刻持久化，哪些片段可以重算，哪些数据可以延迟搬运，哪些事件必须进入恢复路径。类型化状态使内存系统能够在性能、可靠性和可解释性之间作出明确选择。

### 5.7 从一致性到可恢复性

多核处理器需要缓存一致性，保证不同核心看到的共享内存状态符合某种规则。常见协议通过失效、更新和目录记录来维护一致性。Agent CPU 也需要一致性，但它面对的不只是“同一地址的最新值”，还包括“同一任务状态的合法版本”。

例如，一个推理核心读取了上下文片段 A，并基于它产生候选动作；同时，另一个单元更新了相关 artifact 或追加了新的 ledger 约束。系统必须判断候选动作是否仍然有效。这里的一致性更接近版本化一致性：每次读取都带有状态版本，每次提交都检查依赖是否仍满足。若依赖失效，候

选动作可以被重新验证、降级或丢弃。

可恢复性则要求系统能够从 trace 和 ledger 重建关键过程。传统流水线可以通过异常处理和重排序缓冲恢复精确状态；Agent CPU 需要在更长的时间尺度上恢复任务状态。为此，ledger 保存承诺点，trace 保存执行路径，artifact 保存可引用结果，context 保存推理输入。内存与 NoC 的设计必须支持这些对象在层次之间有序流动，使恢复不依赖偶然残留的缓存内容。

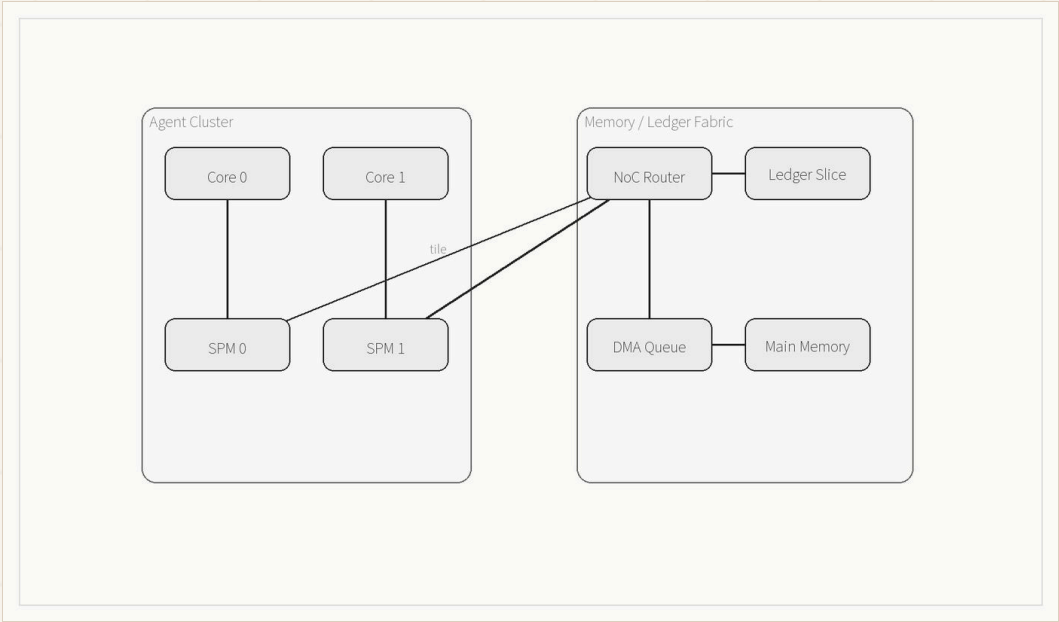


图 5-3: Memory 与 NoC：从状态到数据移动的关键概念对应

## 5.8 设计取舍

内存与 NoC 的设计总在若干目标之间权衡。低延迟有利于交互式推理，但可能需要更多近端存储；高带宽有利于工具调用和 artifact 处理，但可能提高功耗；强一致性简化编程模型，却会限制并行度；详细 trace 提升可审计性，也会消耗存储与互连资源。

Agent CPU 的正向设计思路不是把所有状态都推向最快层，而是让每类状态进入合适的位置。短期 context 应靠近推理核心；大型 artifact 可留在较远层次，通过 DMA 分块取用；ledger 的关键索引应近端可查，完整记录可顺序归档；trace 可以采用热区缓存、冷区压缩和按需回放。

NoC 则根据状态类型提供差异化服务，使关键控制路径保持畅通。

这种分工让 Agent CPU 从“被动访问内存”转向“主动组织状态流”。执行单元提出需求，缓存捕捉局部性，SPM 固定阶段工作集，DMA 承担批量搬运，NoC 连接多类生产者与消费者，ledger/context/artifact/trace 构成可解释的状态空间。至此，内存系统不再只是计算的附属物，而成为智能体体系结构的主干之一。

## 本章小结

本章从传统内存层次、局部性、缓存、SPM、DMA 和 NoC 出发，讨论了 Agent CPU 中状态与数据移动的组织方式。缓存降低平均访问延迟，SPM 提供可预测的阶段性工作区，DMA 让大块状态搬运脱离执行核心，NoC 为多单元系统提供可扩展通信。ledger、context、artifact 和 trace 分别对应不同访问模式与一致性要求，因而需要类型化管理。Agent CPU 的内存体系不只是存放数据，更负责维护智能体状态的可用性、可恢复性和可审计性。

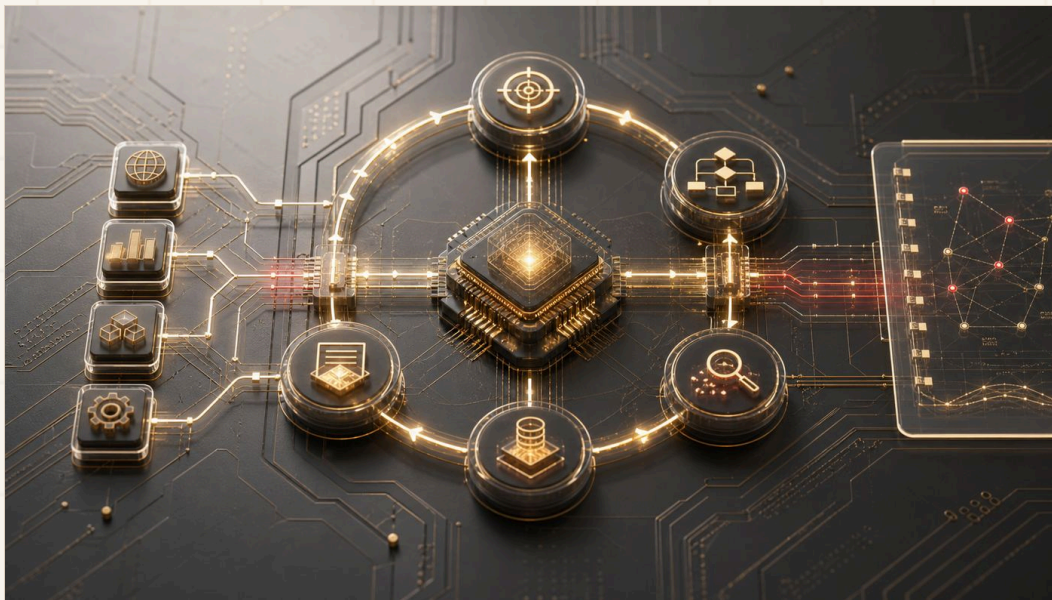
## 习题

1. 说明时间局部性和空间局部性分别如何影响缓存设计。请举出一个普通程序示例和一个智能体上下文访问示例。
2. 比较缓存与 SPM 的区别。为什么 SPM 适合承载一个推理阶段的固定工作集？
3. 传统 DMA 描述符通常包含源地址、目的地址和长度。若用于搬运 artifact 或 trace，还应增加哪些字段？说明理由。
4. 为什么 ledger 流量不宜与普通大块 artifact 搬运完全共享同一优先级？
5. 假设某候选动作依赖上下文版本 C3 和 artifact 版本 A7，但提交前 ledger 追加了新的约束 L9。设计一个简单的验证流程，判断该候选动作是否可以提交。

| 状态对象             | 近端驻留           | 远端路径              | 典型触发  |
|------------------|----------------|-------------------|-------|
| context slice    | SPM / register | 投影后丢弃             | 下一轮提示 |
| tool result      | ledger cache   | main memory       | 工具返回  |
| recovery anchor  | local store    | persistent ledger | 提交前检查 |
| candidate output | agent buffer   | publish queue     | 候选提交  |

图 5-4: 状态对象在 Memory 与 NoC 中的数据移动

## 第 6 章 任务型 Agent 的执行模型



第 6 章插图：任务型 Agent 的执行循环

任务型 Agent 是一种面向目标展开行动的智能计算结构。它接收任务，理解约束，组织可用信息，调用工具获得新的观察，并在执行过程中持续更新自身状态。与只回答一个问题的系统不同，任务型 Agent 的核心不在于一次性给出文本，而在于把“要达到的结果”转化为一连串可检查、可修正、可推进的行动。

从体系结构角度看，任务型 Agent 可以被理解为运行在语义层上的执行单元。它既有类似处理器“取指、执行、提交”的节奏，又面对更开放的世界：目标可能含糊，环境可能变化，工具可能失败，观察可能不完整。本章讨论的重点不是某个具体实现，而是任务型 Agent 在一般意义上的执行模型。

本章将围绕一个贯穿例子展开：某学校希望 Agent 协助教师准备一次“校园节能周”活动方案。任务包括整理活动目标、查找已有资料、拟定日程、生成宣传文案、检查预算约束，并形成一份可交付的计划书。这个例子足够简单，便于观察执行循环；同时又包含目标分解、信息补充、工具使用和状态更新，能够体现任务型 Agent 的典型特征。

## 6.1 任务型 Agent 是什么

任务型 Agent 面向的是“完成一件事”，而不仅是“回答一句话”。它通常具有三个基本特征。

第一，它具有目标导向。目标描述了期望结果，例如“形成校园节能周活动方案”。Agent 需要将目标解释为可以执行的子目标，如“确定活动主题”“列出三天日程”“估算物料成本”等。

第二，它具有环境耦合。任务并不只存在于模型内部。Agent 需要读取文档、检索资料、查询日历、调用计算器或访问数据库。这些外部接触点构成了它的工具环境。Agent 的能力不只来自语言推理，也来自它能否正确选择和使用工具。

第三，它具有过程状态。任务推进时，Agent 会保留已知事实、未决问题、已完成步骤、失败尝试和下一步计划。状态使执行不再是孤立回合，而成为连续过程。对于校园节能周例子，状态中可能记录“活动面向初一年级”“预算上限为 3000 元”“已完成日程草案”“宣传文案待审校”等。

因此，任务型 Agent 可以定义为：在给定目标和约束下，通过维护上下文、选择工具、吸收观察、调整计划并更新状态，持续推进任务直至形成可验证结果的智能执行体。

## 6.2 执行循环的基本构成

任务型 Agent 的运行通常呈现为循环结构。每一轮循环都围绕六个要素展开：目标、上下文、工具、观察、计划、状态变化。它们共同决定 Agent 在某一时刻“为什么行动、依据什么行动、能用什么行动、看到了什么、准备怎样行动、行动后发生了什么变化”。

| 要素   | 含义               | 在节能周例子中的表现             |
|------|------------------|------------------------|
| 目标   | 期望达成的任务结果及评价标准   | 形成一份可用于教师讨论的活动方案       |
| 上下文  | 当前可用的信息、约束和历史记录  | 年级对象、预算上限、校历安排、已生成草案   |
| 工具   | Agent 可调用的外部能力   | 文档读取、资料检索、表格计算、格式生成    |
| 观察   | 工具或环境返回的新信息      | 检索到往年节能活动案例，预算表显示费用超限  |
| 计划   | 对后续行动的组织和排序      | 先确定主题，再排日程，最后检查预算与文案   |
| 状态变化 | 执行后对任务进度和内部记录的更新 | 标记“主题已定”，新增“预算需压缩”的待办项 |

这六个要素不是彼此孤立的模块，而是执行循环中的相互制约关系。目标决定上下文中哪些信息重要；上下文影响计划；计划选择工具；工具产生观察；观察触发状态变化；新的状态又改变下一轮计划。

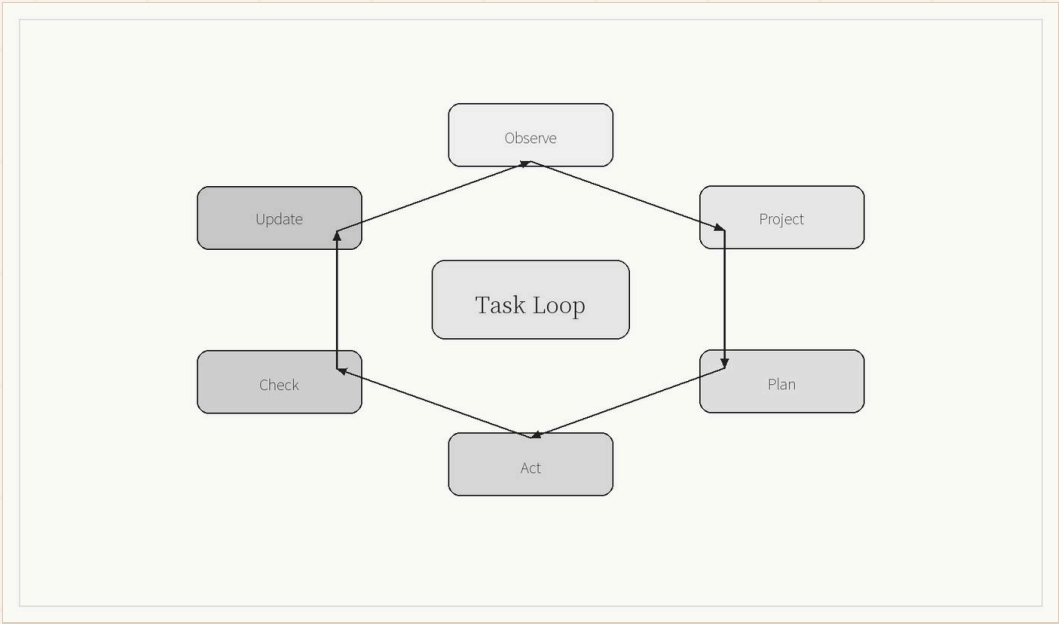


图 6-1：任务型 Agent 的执行模型的机制闭环

## 6.3 目标：执行循环的方向

目标是任务型 Agent 的方向源。没有目标，Agent 只能进行松散的生成；有了目标，行动才有评价标准。目标通常包含三层含义：产物是什么、约束是什么、完成到什么程度算合格。

在校园节能周例子中，“写一个活动方案”只是粗略目标。Agent 需要进一步识别隐含要求：方案应适合学校场景，语言要正式，活动要可执行，预算不能超过上限，日程不能与考试冲突。目标越清晰，后续循环越稳定。

目标也可以在执行中被澄清。假设 Agent 发现校历中某一天已有运动会，它就需要把“连续三天活动”调整为“分散在一周内的三次活动”。这种调整并不是偏离目标，而是在新观察下维护目标的可达性。任务型 Agent 的目标导向并不等于机械执行原始命令，而是围绕任务意图保持合理推进。

## 6.4 上下文：执行所依赖的工作空间

上下文是 Agent 当前“知道并正在使用”的信息集合。它包括用户输入、任务历史、约束条件、工具返回结果、已生成内容以及对未决问题的记录。上下文不是资料仓库的简单堆叠，而是一个工作空间：其中的信息要服务于当前任务。

在例子中，Agent 的上下文可能包括教师给出的要求：“面向初一年级，活动持续一周，预算 3000 元以内，尽量包含学生参与环节。”还可能包括已经生成的主题候选：“低碳校园，从一度电开始”；以及工具返回的事实：“周三下午全年级有班会时间，可安排集中活动。”

上下文管理的关键是选择性。过少的上下文会导致遗忘约束，过多的上下文会稀释注意力。任务型 Agent 需要把信息组织成事实、约束、待办、风险和产物草稿，使每一轮循环都能提取行动依据。



图 6-2：任务型 Agent 的执行模型的概念展开路径

## 6.5 工具与观察：连接外部世界

工具是任务型 Agent 扩展能力边界的方式。模型本身擅长语言理解、归纳和生成，但任务执行常常需要外部能力：读取文件、检索资料、计算成本、检查格式、访问日程表等。工具调用使 Agent 能从封闭推理走向开放环境。

工具并不直接等于结果。Agent 调用工具后获得的是观察。观察可能清晰，也可能不完整，还可能是失败信息。执行模型必须把观察纳入下一轮判断，而不是把工具调用视为天然成功。

在节能周例子中，Agent 为估算预算调用表格计算工具，得到宣传海报、奖品、展板等项目的合计费用。观察显示预算超过上限。此时合理的状态变化不是简单报错，而是新增一个调整目标：“压缩奖品费用或减少展板数量，使总额低于 3000 元。”工具提供事实，Agent 负责解释事实在任务中的意义。

## 6.6 计划：从目标到行动序列

计划是 Agent 对行动顺序的组织。它把目标分解为可执行步骤，并在步骤之间建立依赖关系。好的计划既不能过粗，也不宜过细：过粗容易迷失，过细则可能僵化。

对于校园节能周任务，一个初始计划可以是：

1. 明确活动对象、时间和预算约束。
2. 收集类似校园节能活动的常见形式。
3. 拟定活动主题和三到五个活动环节。
4. 将活动环节排入校历中的可用时段。
5. 估算物料和奖品费用。
6. 生成方案正文，并检查目标、预算和日程是否一致。

计划在执行中会变化。若资料检索发现学生参与度较高的活动通常需要班级积分机制，Agent 可以将“设计积分规则”加入计划。若预算计算显示超支，则计划需要插入“压缩成本”步骤。计划的价值不在于一开始就完美，而在于为循环提供当前最合理的行动框架。

## 6.7 状态变化：执行的可见进展

状态变化是任务型 Agent 区别于普通生成系统的重要标志。每次行动后，Agent 都应更新任务状态：哪些已完成，哪些被修改，哪些问题仍然开放，哪些风险新出现。状态变化使执行具备连续性和可追踪性。

在例子中，Agent 经过几轮循环后，状态可能从“尚未确定方案结构”变为“主题已确定，日程初稿完成，预算超支 260 元，宣传文案待生成”。这个状态不是最终文本，却是任务推进的真实记录。

状态还支持提交判断。当所有关键待办关闭，主要约束满足，产物达到目标要求时，Agent 可以进入提交阶段，输出完整方案。若仍有关键不确定性，例如活动日期需要校方确认，Agent 应在结果中标明假设条件，而不是把未确认信息伪装成事实。

## 6.8 一个完整执行片段

下面以简化形式展示校园节能周任务的一段执行循环。

初始目标是“生成一份校园节能周活动方案”。上下文中已有对象、预算和时间要求。Agent 制定计划：先确定主题，再安排活动，再核算预算。它调用资料检索工具，观察到常见形式包括节电打卡、旧物改造、低碳知识竞赛和班级节能积分。Agent 更新状态：新增四类候选活动，并选择其中三类进入草案。

随后，Agent 调用日程查询工具，观察到周三下午可进行年级活动，周五班会适合总结展示。状态更新为“周三安排知识竞赛，周五安排成果展示，周一至周四开展节电打卡”。接着，Agent 调用预算计算工具，观察到初始物料费用为 3260 元，超出预算。计划随即调整：减少展板数量，将部分奖品改为班级荣誉证书。再次计算后，观察显示费用为 2860 元。状态更新为“预算约束已满足”。

最后，Agent 根据当前状态生成方案正文，并进行一致性检查：目标面向初一年级，活动覆盖一周，包含学生参与环节，预算低于上限。检查通过后，任务进入提交。可以看到，最终方案并非一次生成，而是由多个循环逐步收敛而成。

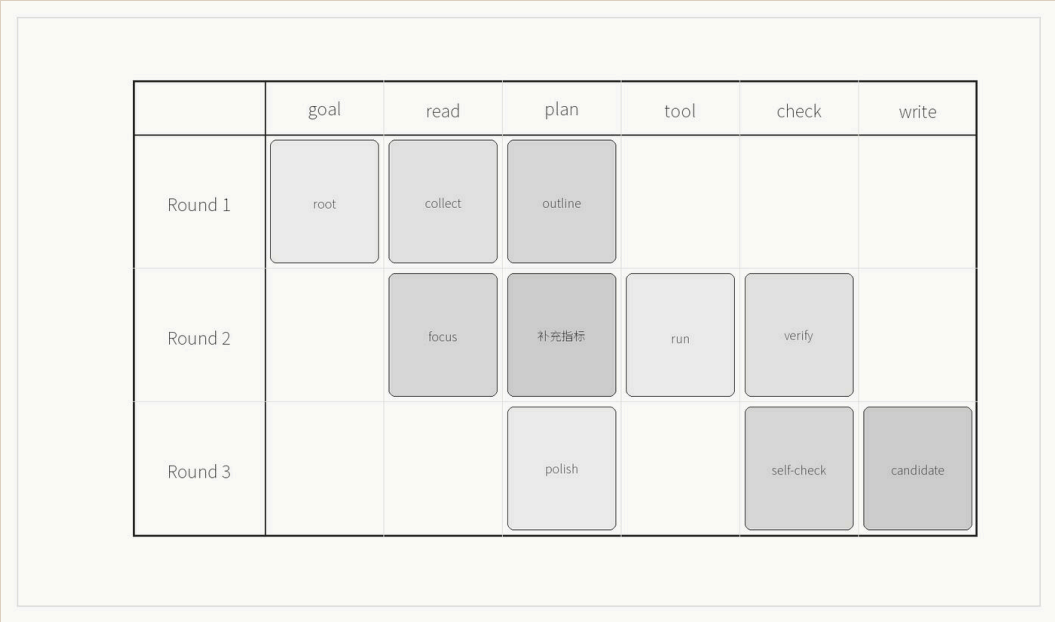


图 6-4：任务型 Agent 的执行循环

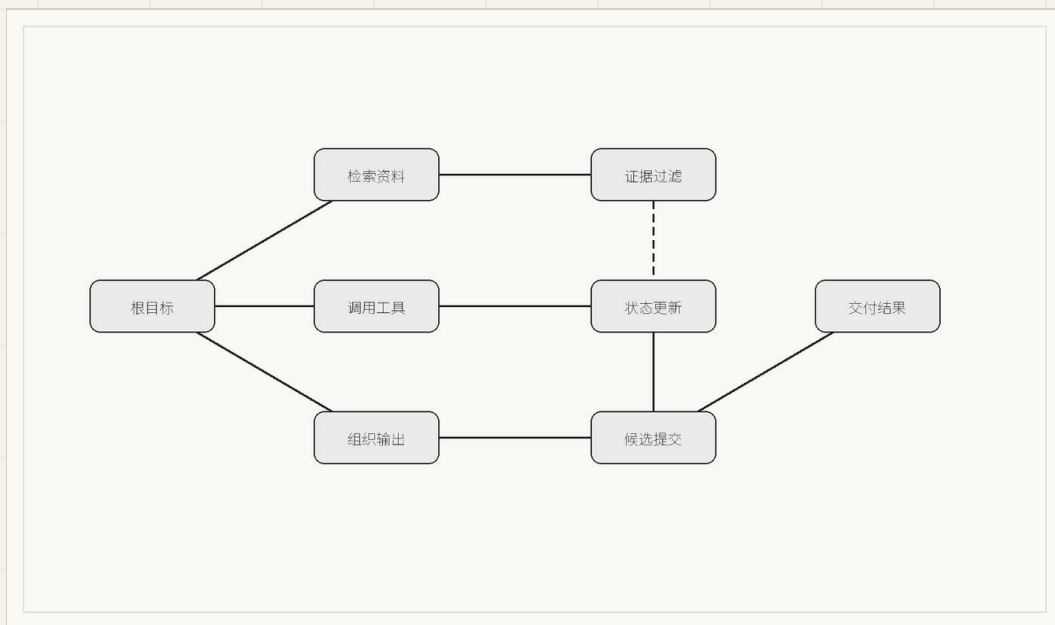


图 6-3：任务型 Agent 的执行模型的层次展开

## 6.9 执行模型的意义

任务型 Agent 的执行模型揭示了智能行动的基本结构：目标提供方向，上下文提供依据，工具提供外部能力，观察提供新事实，计划组织行动，状态变化保存进展。六者形成闭环，使 Agent 能够在不完全信息中持续推进任务。

这种模型也说明，任务型 Agent 的可靠性不只取决于语言表达能力。若不维护状态，就容易遗漏约束；若不解释观察，就容易机械拼接工具结果；若不调整计划，就容易在环境变化时失效。可靠执行来自循环中各要素的协调。

对 Agent CPU 体系结构而言，任务型 Agent 的执行循环可以视为更高层的控制流。它不像传统指令流那样完全预先确定，而是在语义目标和环境反馈之间动态展开。理解这种执行模型，是进一步讨论记忆、工具调度、异常恢复和结果提交机制的基础。

## 本章小结

本章介绍了任务型 Agent 的基本执行模型。任务型 Agent 是面向目标的智能执行体，它通过维护上下文、调用工具、吸收观察、调整计划和更新状态来推进任务。执行循环中的六个要素相互连接：目标决定方向，上下文承载工作信息，工具连接外部环境，观察带来反馈，计划安排行动，状态变化记录进展。通过校园节能周方案的例子可以看到，任务结果不是一次生成的静态文本，而是在多轮循环中逐步形成的可验证产物。

## 习题

1. 说明任务型 Agent 与一次性问答系统的主要区别，并举一个课堂学习场景中的例子。
2. 在“准备一次班级读书分享会”的任务中，分别写出目标、上下文、工具、观察、计划和状态变化各一项。
3. 为什么工具调用的返回结果应被看作“观察”，而不应直接等同于“任务完成”？
4. 选择一个你熟悉的复杂任务，画出至少三轮执行循环，并标明每一轮的状态变化。
5. 如果 Agent 在执行中发现原计划无法满足预算约束，它应如何调整计划？请用本章的六个要素进行分析。

## 第 7 章 长期任务记忆



第 7 章插图：长期任务记忆、Ledger 与恢复锚点

### 7.1 上下文与记忆的分工

模型上下文是工作台，长期记忆是账本。工作台负责组织本轮推理所需的信息；账本负责保存可恢复、可审计、可追溯的任务状态。

Agent 长任务需要五类账本：

- GoalLedger
- PlanLedger
- EvidenceLedger
- DecisionLedger

- RecoveryLedger

以及一类投影：

- ContextProjection

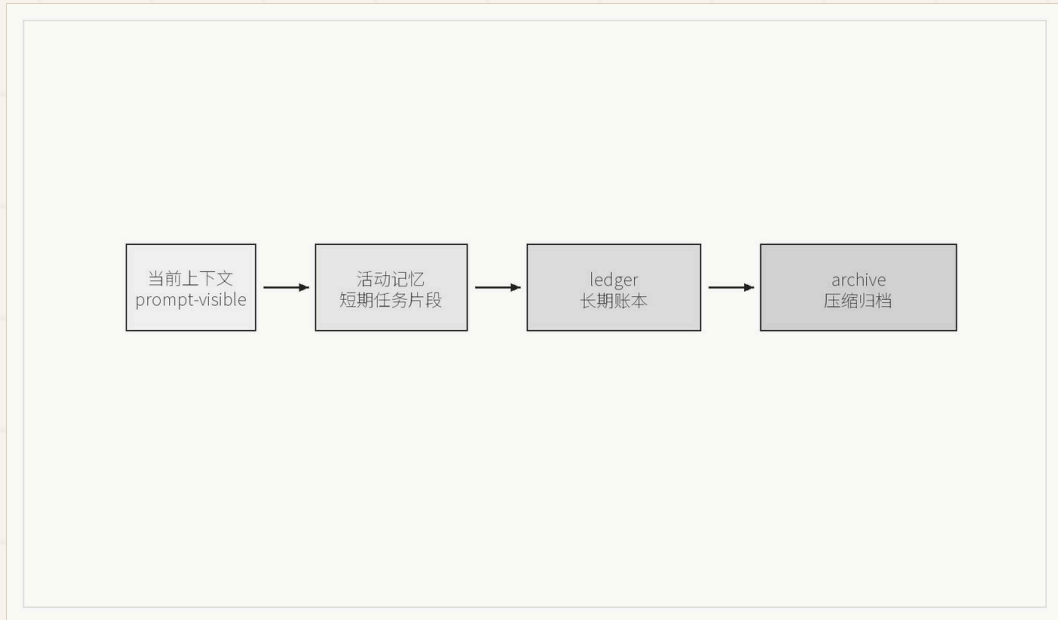


图 7-1：长期任务记忆的对照关系

## 7.2 Ledger 的含义

| Ledger         | 保存内容                    |
|----------------|-------------------------|
| GoalLedger     | 用户目标、成功标准、禁止事项          |
| PlanLedger     | 当前阶段、已完成、待完成、依赖         |
| EvidenceLedger | 文件、测试、命令、artifact、trace |
| DecisionLedger | 采用方案、拒绝方案、原因            |
| RecoveryLedger | 最近安全点、回滚依据、脏状态          |

| anchor | digest | delta | scope   | restore     |
|--------|--------|-------|---------|-------------|
| A0     | goal   | 初始任务  | global  | cold start  |
| A1     | ctx    | 新增证据  | working | resume step |
| A2     | tool   | 执行结果  | tool    | replay ok   |
| A3     | draft  | 候选变更  | publish | rollback    |

图 7-2：长期任务记忆的层次展开

### 7.3 Compact 事务

compact 是一次状态整理事务：

1. 冻结 transcript 边界。

2. 扫描 recent events。
3. 生成 ledger delta。
4. 选择 recovery anchor。
5. 打包 context projection。
6. Super Domain 审核提交。

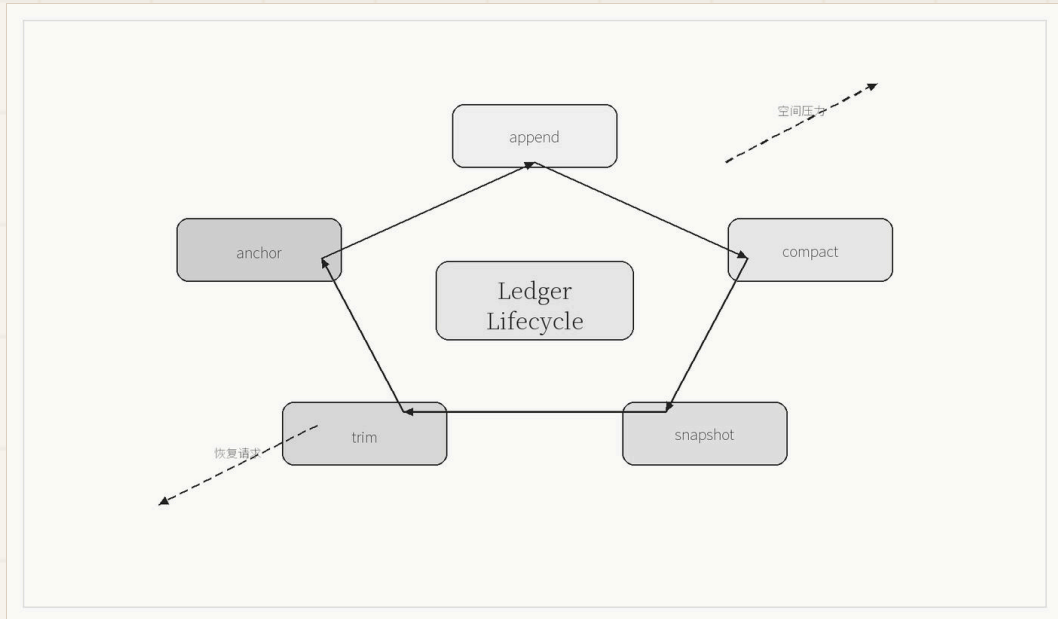


图 7-3：长期任务记忆的机制闭环

## 7.4 断点恢复

恢复过程从 ledger 重建任务状态：

- 当前目标。
- 当前阶段。
- 有效证据。
- 被拒绝路径。

- 下一步动作。

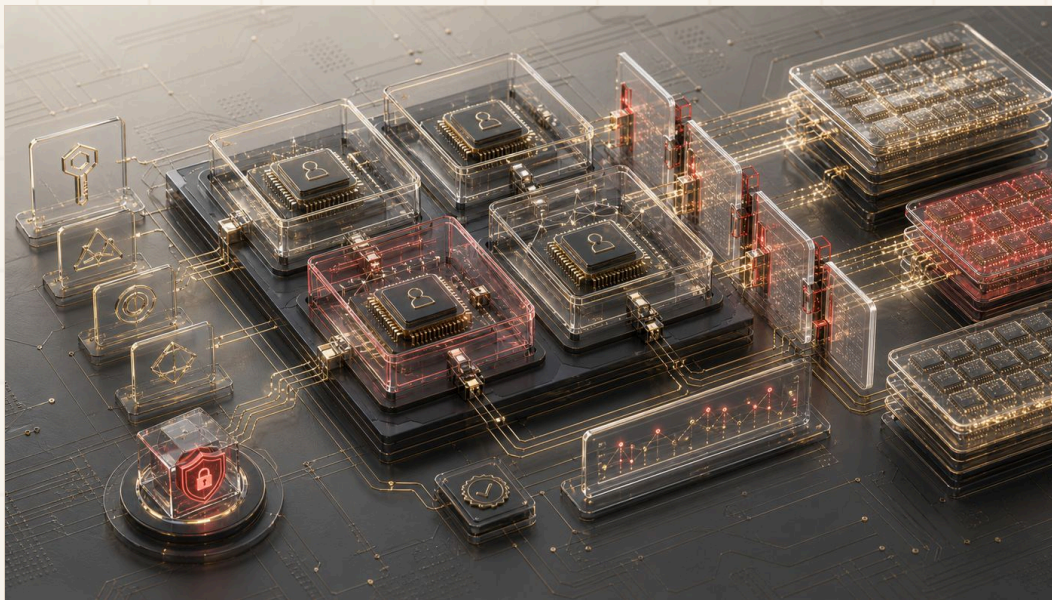
本章习题

1. 为什么 `DecisionLedger` 对长任务很重要？
2. 给出一个 `compact` 失败但不污染 `ledger` 的处理流程。
3. 设计一个 `recovery anchor` 的最小字段。



图 7-4: Ledger、Compact 与 Recovery Anchor 的关系

## 第 8 章 安全与权限



第 8 章插图：Capability、安全边界与可审计 Trace

### 8.1 Agent 生成代码的安全边界

Agent 生成代码具有明确的安全边界。系统通过 capability、预算、trace 和审核提交，把生成代码限制在可观察、可解释、可恢复的执行范围内。

|      | 默认 | 受限授权  | 高风险审计    |
|------|----|-------|----------|
| 文件读取 | 允许 | 记录    | 记录       |
| 脚本执行 | 拒绝 | 批准后执行 | 强制 trace |
| 外部写入 | 拒绝 | 候选态   | 人工确认     |
| 网络发送 | 拒绝 | 白名单   | 双重审计     |

图 8-1：安全与权限的关键概念对应

## 8.2 Capability

Capability 定义 task 能访问什么：

```
{
  "region": "artifact_previews",
  "space": "host",
  "base": 2048,
  "bytes": 1024,
  "access": "read"
}
```

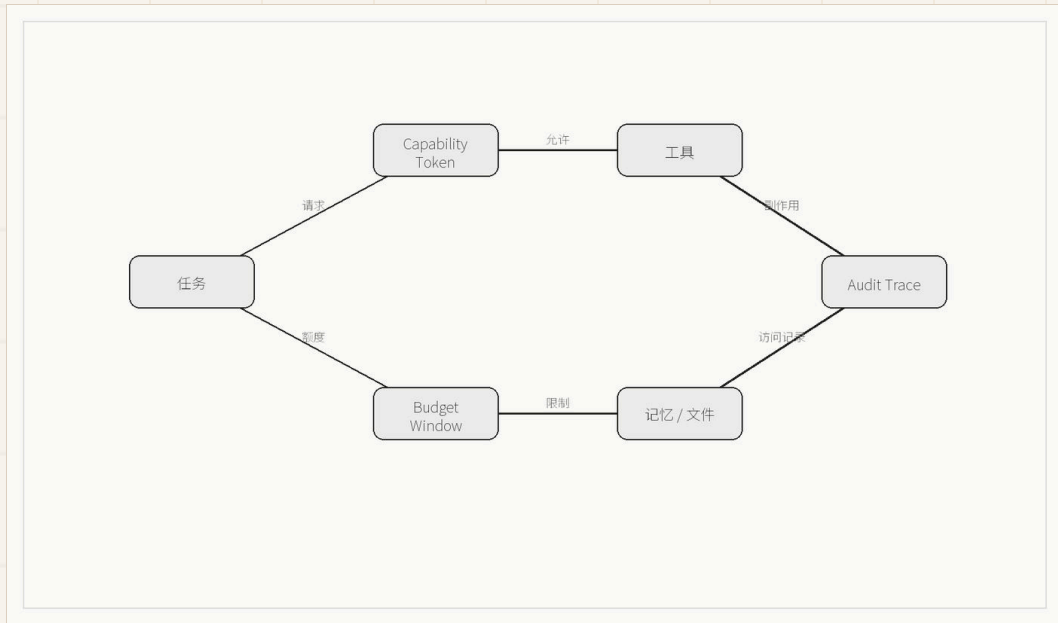


图 8-2: 安全与权限的扇出关系

### 8.3 长期记忆的受控访问

长期记忆采用受控访问规则。Agent Domain 产生候选状态，Super Domain 负责审核和提交。典型边界包括：

- ledger 更新通过候选 delta 表达。
- transcript 写入由 Super Domain 统一提交。
- artifact 正文读取经过 capability 授权。
- evidence ref 由系统校验可解析性。
- 用户目标变更保留明确来源和审核记录。



图 8-3：安全与权限的阶段推进

## 8.4 Trace

Trace 必须回答：

- 谁执行了任务？
- 读了哪些授权区域？
- 生成了哪些候选输出？
- 是否越界？
- 是否发生 trap？
- 为什么选择某个 recovery anchor？

### 本章习题

1. 为什么 trace 本身也需要预算？
2. 什么情况下应拒绝 ledger delta？

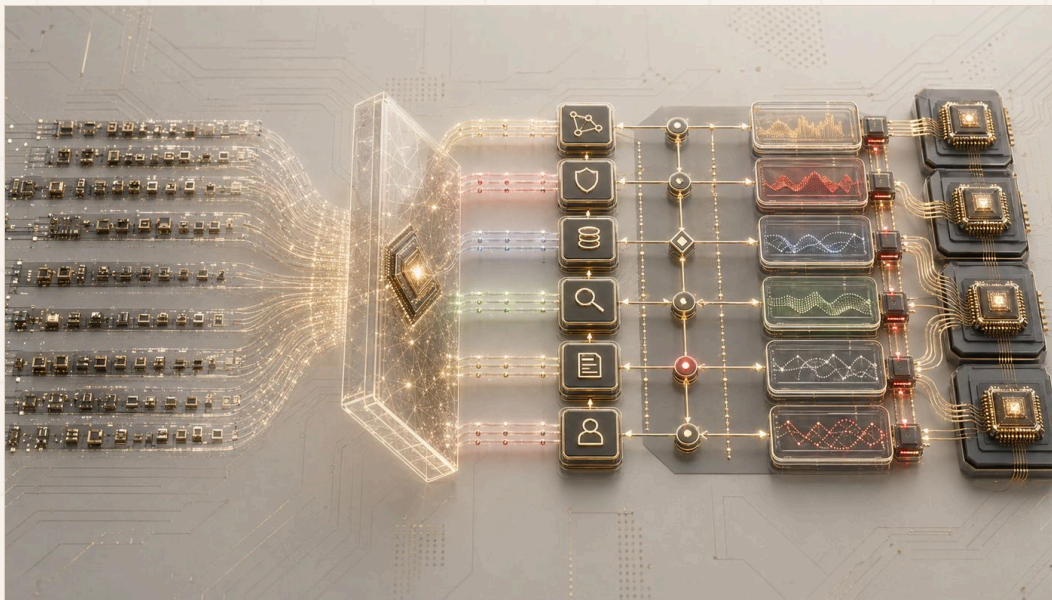
3. 如何防止 Agent 把用户禁止事项从 projection 中删掉?

# 第四篇：传统体系结构映射

| 边界对象                  | 允许动作  | 触发 trap | 审计记录          |
|-----------------------|-------|---------|---------------|
| candidate-only output | 本地生成  | 直接发布    | draft hash    |
| filesystem write      | 沙箱路径  | 越界写入    | path + reason |
| tool command          | 白名单参数 | 能力不匹配   | command trace |
| network send          | 受控目标  | 外部泄露    | destination   |

图 8-4: Capability、trap 与可审计 trace 的安全边界

## 第 9 章 从 ISA 到 APU-IR



第 9 章插图：从低层 ISA 流到结构化 APU-IR

### 9.1 ISA 作为软硬件契约

传统计算机体系结构中，ISA 是软件与硬件之间最重要的边界。编译器、操作系统和应用程序面对的是指令、寄存器、异常、内存模型和特权级；微架构则可以在这个边界之下自由选择流水线、乱序执行、缓存层次和预测机制。只要 ISA 语义不变，同一二进制程序就可以在不同实现上运行。

RISC-V 的启发尤其清楚：一个好的 ISA 不只是指令表，而是一套可扩展、可验证、可教学的公共契约。基础整数指令集保持小而稳定，乘除法、原子操作、向量、压缩指令和自定义扩展则按需要组合。它把“什么必须稳定”和“什么可以扩展”分开，使学术原型、嵌入式小核和复杂 SoC 可以共享同一套思想。

Agent CPU 也需要类似契约，但契约边界发生了上移。Agent 生成的逻辑通常围绕一次目标、一次工具调用批次或一次上下文投影形成短生命周期任务。此时，硬件真正需要理解的是：这个任务要访问哪些证据，能使用哪些 capability，预算是多少，输出会写到哪里，失败后如何解释。

| 维度    | ISA       | APU-IR          |
|-------|-----------|-----------------|
| 基本对象  | 指令 / 寄存器  | 任务 / 证据 / 预算    |
| 控制粒度  | 周期级       | 任务片段级           |
| 正确性依据 | 语义 + 内存模型 | 目标 + 证据 + trace |
| 副作用   | 提交即生效     | candidate 后发布   |
| 优化焦点  | 吞吐 / 延迟   | 治理 / 恢复 / 调度    |

图 9-1：从 ISA 到 APU-IR的对照关系

## 9.2 APU-IR 承载高层任务语义

可以把 Agent ISA 理解为 Agent Domain 中的低层执行语义：访存、拷贝、比较、分支、barrier、flush、trace event 等。它仍然重要，因为最终执行必须落到确定的机器步骤上。但是，如果只定义低层 ISA，就会丢失 Agent workload 最宝贵的高层信息。

例如，一个 context\_budget\_pack 任务在低层看可能只是读取 ledger、筛选引用、拼接文本、写入缓冲区；在高层看，它其实是在给下一轮模型推理构造上下文投影。后者天然包含 token budget、证据优先级、引用上限、恢复点和 trace 策略。如果这些信息只藏在指令序列里，Super Domain 就很难在准入阶段判断任务是否越权、是否超预算、是否会破坏已提交 ledger。

因此，Agent CPU 需要两层契约：低层 Agent ISA 保证执行可落地，高层 APU-IR 保证任务可验证、可调度、可恢复。

## 9.3 APU-IR 的位置

APU-IR 是 Agent Runtime 与 Agent Domain 之间的任务图中间表示。它用结构化字段表达任务意图、输入输出、权限、预算、放置和 trace 策略，是进入硬件或模拟器前的可检查对象。

一个最小 APU-IR task 应描述：

- `goal` 或 `op_class`：任务意图，例如扫描、打包、抽取、选择恢复点。
- `inputs` 与 `outputs`：输入证据、memory region、artifact preview 或 trace window。
- `capability`：任务可读、可写、可执行、可追加的资源边界。
- `budget`：cycles、SPM bytes、cluster bytes、token budget、trace bytes 等。
- `placement` 与同步：运行在哪个 cluster/core，依赖哪些 barrier。
- `trace_policy`：哪些事件必须记录，哪些可以摘要化。

在这个模型中，Agent Task 是带有权限、预算、证据和审计要求的状态变换。Super Domain 负责 admission：检查格式、capability、预算和依赖；Agent Domain 负责执行：按照降低后的 Agent ISA 或硬件单元完成任务，并在 trap 或完成时返回结构化结果。

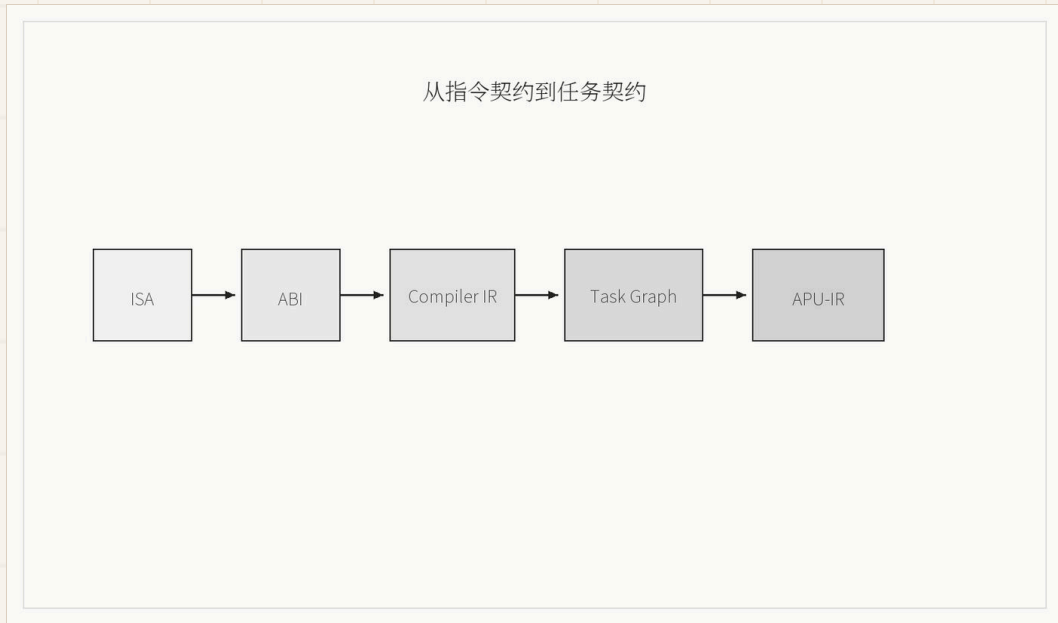


图 9-2：从 ISA 到 APU-IR 的阶段推进

## 9.4 从 RISC-V 扩展思想到 APU-IR 扩展

RISC-V 通过标准扩展和自定义扩展解决“基础稳定、能力增长”的矛盾。APU-IR 也应采用类似思路。基础 task schema 必须稳定，例如 task id、capability、budget、memory region、trace 字段；高层 op\_class 则可以持续扩展，例如：

- ledger\_delta\_extract：从执行结果中抽取可提交状态变化。
- context\_budget\_pack：在 token 或 byte 预算内生成上下文投影。
- recovery\_anchor\_select：从 trace 与 ledger 中选择恢复点。
- artifact\_preview\_pack：为模型构造受限 artifact 预览。
- memory\_header\_scan：快速扫描文件、对象或缓冲区头部。

这些高层操作未来可以降低为不同实现：软件 runtime、Agent ISA 指令序列、DMA 加速路径、专用硬件单元，甚至 RISC-V 自定义协处理器接口。设计顺序是先稳定任务语义，再逐步选择二进制编码、硬件流水线和加速路径。

| 传统体系结构概念 | Agent CPU 对应概念                   | 设计关注点                              |
|----------|----------------------------------|------------------------------------|
| ISA      | Agent ISA 与 APU-IR               | 低层执行语义与高层任务契约分离                    |
| ABI      | Task schema                      | Runtime、模拟器、硬件之间交换任务对象             |
| 特权级      | Super Domain / Agent Domain      | 准入、隔离、trap 和资源回收                   |
| 地址空间     | capability-bounded memory region | 访问权由任务显式声明并被检查                     |
| 性能计数器    | budget 与 trace                   | 资源消耗可解释，失败点可定位                     |
| 异常       | task trap                        | 返回 task、pc、capability、reason 和证据引用 |

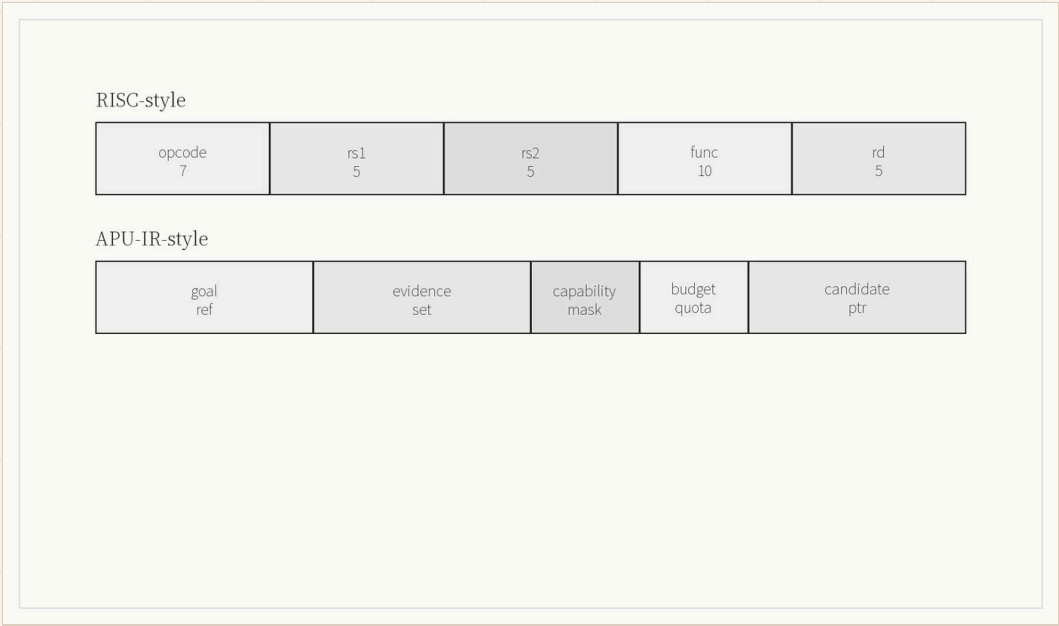


图 9-3：从 ISA 到 APU-IR的关键概念对应

9.5 预算、权限与 trace 的共同语义

在传统 CPU 中，权限主要围绕地址空间和特权指令；性能主要由计数器事后观察；调试依赖断点、异常和日志。Agent CPU 则应把三者前置到任务契约中。

`capability` 回答“允许做什么”。它可以限制某个 task 只能读取 ledger 摘要、只能向候选输出区追加、不能覆盖已提交 artifact。`budget` 回答“最多消耗多少”。它不仅包括 cycles 和内存，也包括 token、引用数量、trace 字节数。`trace` 回答“如何解释发生了什么”。一次高质量输出应当能够回指证据和执行路径。

三者必须同时出现。只有 `capability` 没有 `budget`，任务可能合法但失控；只有 `budget` 没有 `capability`，任务可能高效地越权；只有二者没有 `trace`，任务失败后无法恢复，也无法教学、调试或审计。

## 本章习题

1. 解释为什么 RISC-V 的开放扩展思想适合启发 APU-IR，但 RISC-V 小核本身不能直接等同于 Agent Core。
2. 为 `recovery_anchor_select` 设计一个最小 APU-IR task，至少包含 `capability`、`budget`、输入、输出和 `trace_policy`。
3. 某 task 有合法 `capability`，但缺少 token budget。它应该在准入阶段被拒绝，还是运行时 trap？说明理由。
4. 对比传统 ISA 异常和 Agent task trap：为什么后者需要同时记录 task id、pc、reason、`capability` 和 evidence ref？

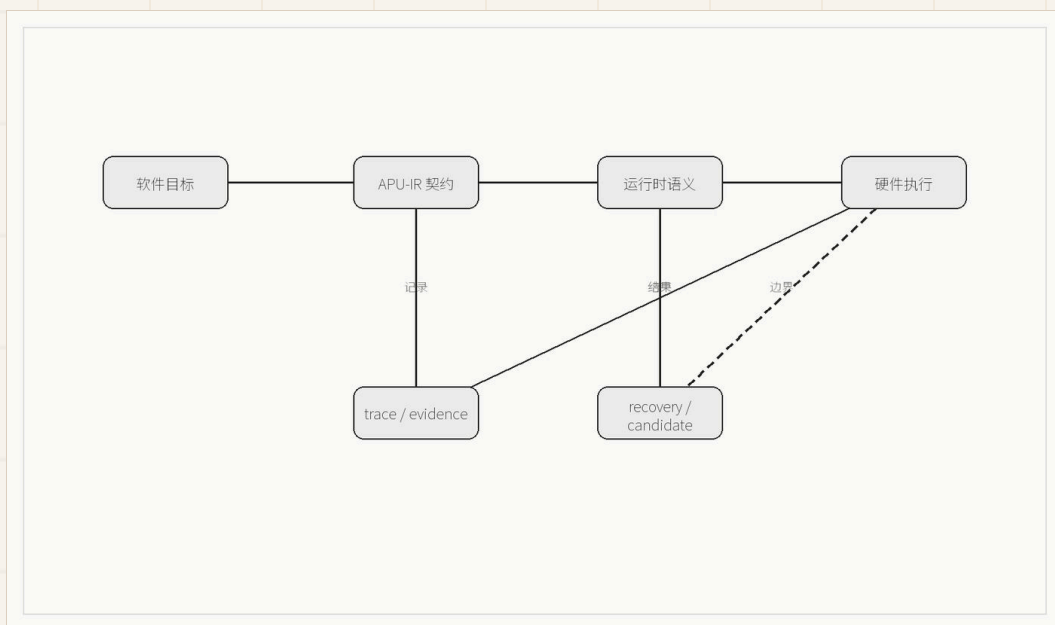
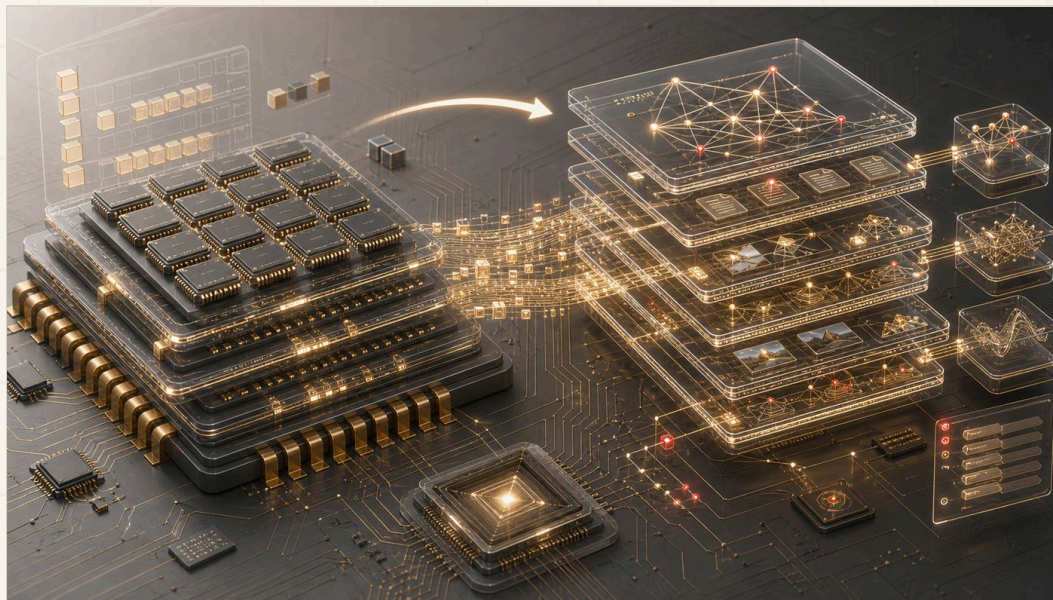


图 9-4: 从 ISA 契约到 APU-IR 契约

## 第 10 章 从 Cache 到 Ledger-Aware Memory



第 10 章插图：从 Cache 层次到 Ledger-Aware Memory

### 10.1 从传统 Cache 出发

传统计算机体系结构把内存层次建立在一个基本假设上：程序的访存局部性可以由硬件在运行时捕捉。寄存器、L1/L2 Cache、LLC、DRAM/HBM、外存构成了从近到远的容量与延迟梯度。Cache 通过 tag、替换、预取和一致性协议，尽量让程序觉得自己面对的是一片连续而快速的内存。

这个模型对长期运行的人类软件很有效，因为循环、数组、栈帧、对象布局具有稳定模式。Agentic workload 的热路径呈现另一种局部性：Agent 任务常常在短生命周期内完成一次读取、归纳、生成、验证和提交；它访问的对象包括 goal、context section、artifact preview、trace event、candidate ledger delta 和 recovery anchor。硬件在 cache line 层面捕捉空间局部性的同时，也可以利用这些对象的任务语义。

因此，Agent CPU 的内存系统同时关心地址访问历史和任务语义角色。

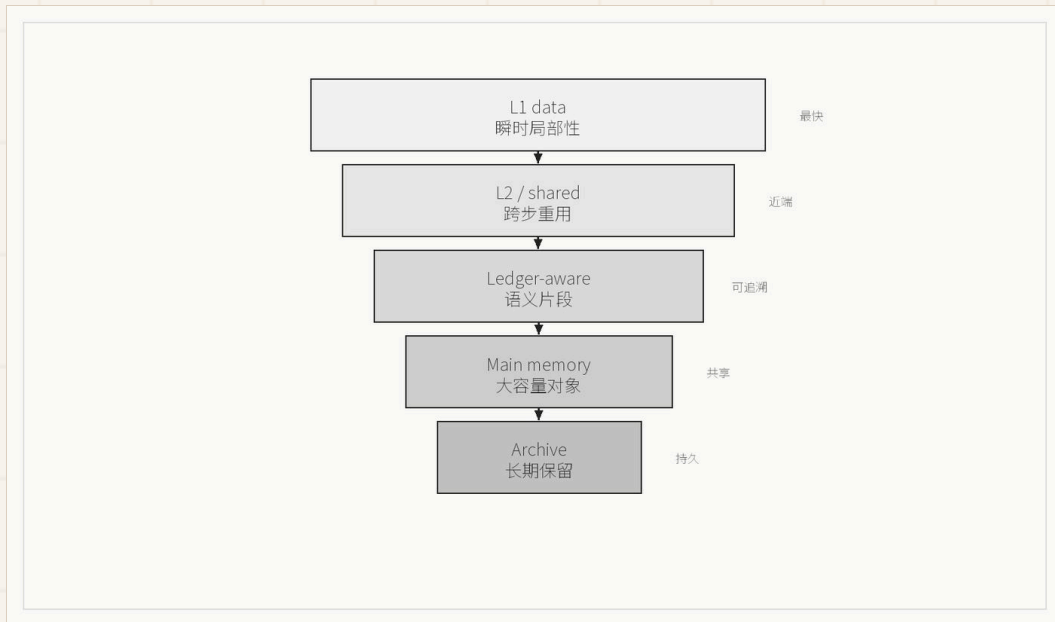


图 10-1：从 Cache 到 Ledger-Aware Memory 的层次展开

## 10.2 从容量层次到语义层次

Agent CPU 仍然需要传统内存层次。控制面代码、运行时队列、普通数据结构仍可受益于 Cache；大模型外部状态也仍需要 HBM 或主存提供容量。但在 Agent 执行面，内存层次要叠加一层语义分类。

| 层级                     | 传统用途          | Agent CPU 中的语义                        |
|------------------------|---------------|---------------------------------------|
| Register / local state | 指令近端操作数       | 当前 task 的短暂控制状态                       |
| L1/L2 Cache            | 自动缓存热点数据      | 服务控制面与少量不可显式管理热路径                     |
| SPM                    | 软件管理片上存储      | 当前 context 分片、临时表、局部工作集               |
| Cluster SRAM           | cluster 内共享缓存 | 多 Agent 阶段共享、producer/consumer 中间态    |
| HBM / Host Memory      | 大容量数据         | transcript、artifact、ledger、trace 的容量层 |
| Context Window         | Agent 语义层     | 模型可见投影，由长期记忆生成                        |
| Ledger Region          | 无传统对应物        | 已提交事实、证据引用和可恢复状态                      |

这里最关键的转变是：context window 是一次推理前对长期记忆的投影。它可以被重排、压缩、摘要、裁剪；只要投影过程可追溯，系统状态就可以由 ledger、artifact、trace 和恢复锚点共同承载。

### 10.3 SPM、Cluster SRAM 与显式 DMA

传统 Cache 的优点是透明，缺点也是透明。它让程序员和编译器不必说明数据移动，却也让运行时难以判断带宽究竟被谁消耗。Agent CPU 更适合采用显式本地内存：每个 Agent 核拥有 SPM，cluster 内共享 Cluster SRAM，远端容量层由 HBM 或 host memory 提供。

典型流程如下：

1. Super Domain 授权任务可读取的 ledger 区段、artifact 预览区和输出区域。
2. Agent Domain 用 DMA 把 ledger header、相关 trace 摘要和 artifact preview 搬入 Cluster SRAM。
3. 每个 Agent 核把自己需要的 context section 或候选表搬入 SPM。
4. 任务在 SPM 中完成筛选、打包、验证或 delta 生成。
5. 候选结果写回 Cluster SRAM，经 barrier 汇合后，由控制面提交或拒绝。

显式 DMA 的价值不只是性能。它让数据移动成为可审计事件：搬了多少字节，从哪个命名空间到哪个命名空间，为哪个 task 服务，是否越过 capability 边界。这些信息可以进入 trace，进而成为 benchmark、调试和安全策略的一部分。

| 维度   | Cache        | Ledger-Aware Memory       |
|------|--------------|---------------------------|
| 命中单位 | address line | 语义对象 / delta              |
| 替换依据 | 空间局部性        | 任务阶段与证据价值                 |
| 写回方式 | dirty line   | append / compact / anchor |
| 恢复能力 | 有限           | 面向任务恢复                    |

图 10-2：从 Cache 到 Ledger-Aware Memory 的对照关系

### 10.4 Ledger-Aware Memory 的基本概念

Ledger-aware memory 指内存系统知道某些区域具有账本语义。账本语义至少包含四点。

第一，写入与提交分层。Agent 生成 candidate ledger delta，Super Domain 经过权限、证据、预算和一致性检查后提交到 ledger。

第二，读取保留 provenance。一个 context projection 引用某段事实时，应能追溯到 ledger entry、artifact preview 或 trace summary。

第三，替换策略应考虑语义热度。传统 LRU 关心最近访问；ledger-aware memory 还应关心任务图依赖、恢复锚点距离、证据复用频率和审计优先级。

第四，可见性应显式化。flush、invalidate、fence 不只是缓存一致性操作，也可以表达“某个候选状态已经对下一阶段可见”或“某个 projection 已失效，需要重新从 ledger 打包”。

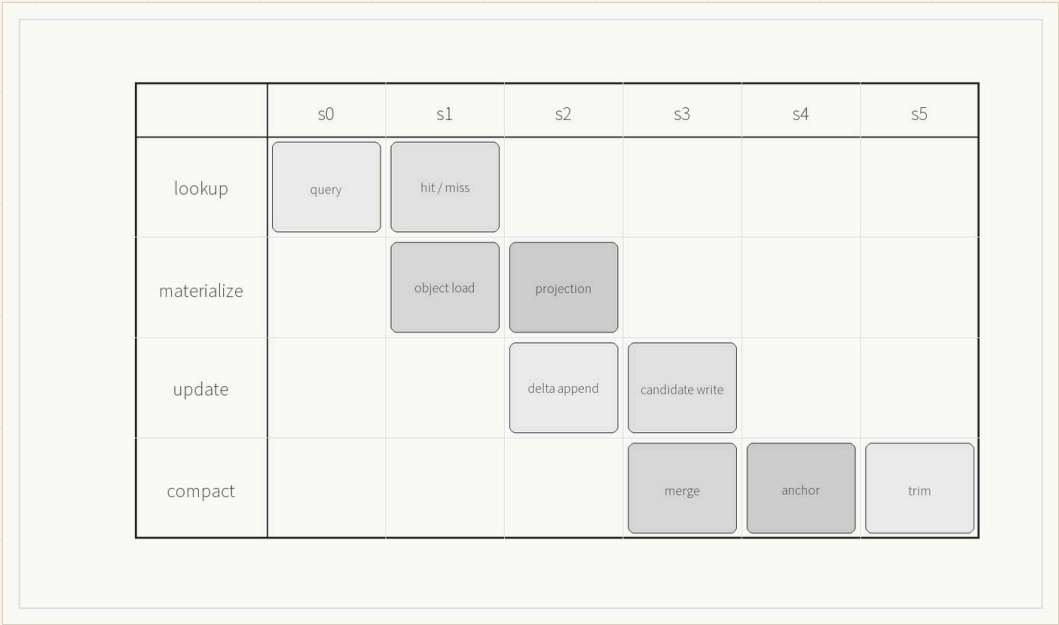


图 10-3：从 Cache 到 Ledger-Aware Memory 的阶段推进

10.5 Artifact Preview 与 Trace

Agent 经常使用 artifact preview：文件摘要、表格 schema、图像描述、测试失败片段、代码 diff 的关键 hunks。Artifact preview 类似传统体系结构中的 cache line，但它的切分单位由语义决定。一个好的 preview 应当小到能放入 context window，大到足以支持推理判断，并且能回指原始 artifact。

Trace 则是内存系统的时间维度。传统内存层次主要保存“现在有什么”；Agent CPU 还保存“状态如何变成现在这样”。DMA copy、barrier arrive、ledger delta extract、context\_budget\_pack、trap、recovery anchor select 都应成为结构化 trace event。这样，当任务失败或输出可疑时，系统可以从 trace 重建路径。

本章习题

1. 为什么 context window 适合作为长期记忆的投影？请结合 ledger 和 artifact preview 说明。
2. 比较传统 Cache line 与 artifact preview：二者的切分依据、替换策略和正确性责任有什么不同？
3. 设计一个 ledger\_delta\_extract 任务的数据移动流程，要求显式使用 SPM、Cluster SRAM、HBM 和 DMA。
4. 如果一个系统只记录最终 ledger，不记录 trace，会给调试、审计和恢复带来哪些问题？
5. 选择三个指标，用来评价 ledger-aware memory 是否优于只依赖传统 Cache 的实现。

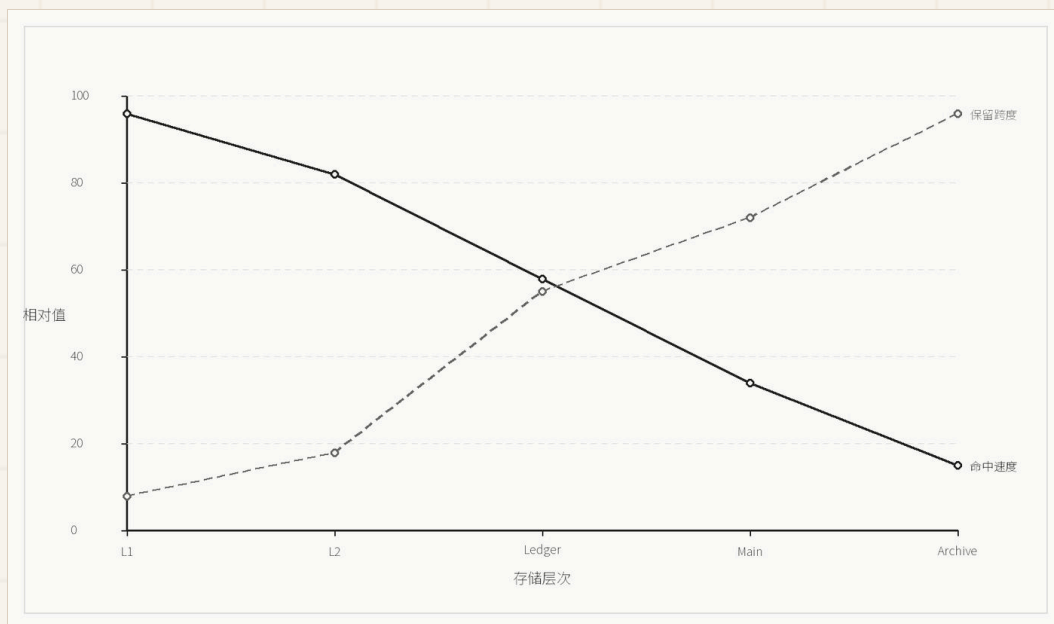
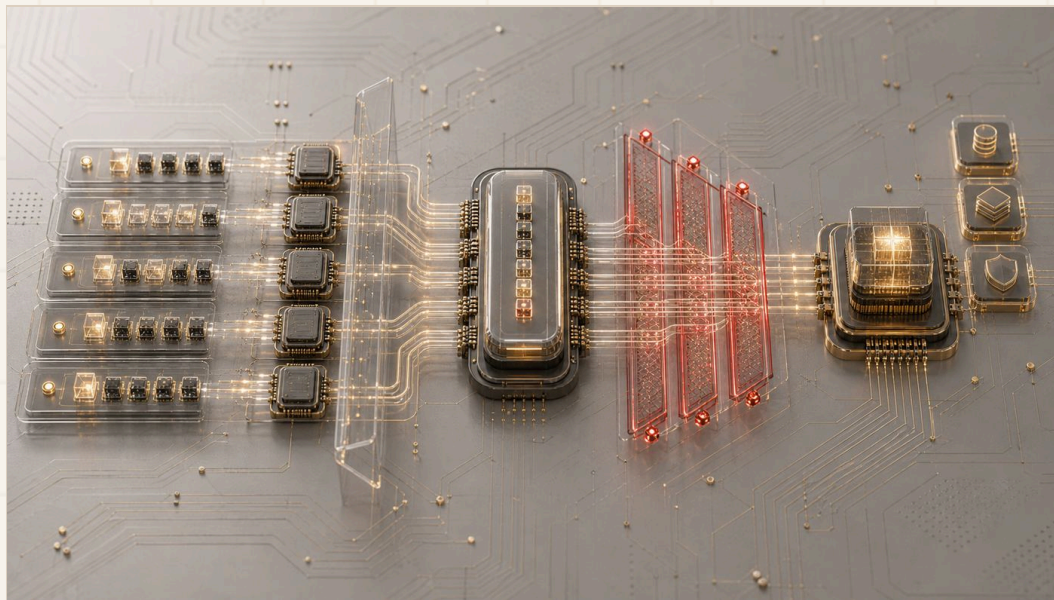


图 10-4: Ledger-Aware Memory 的语义层次

## 第 11 章 从 ROB Commit 到 Candidate Commit



第 11 章插图：从 ROB Commit 到 Candidate Commit

### 11.1 为什么从提交机制开始

在乱序 CPU 中，指令可以乱序执行，却必须表现得像顺序执行。这一矛盾由 ROB 承担：前端按程序顺序分配 ROB 项，后端允许指令乱序发射、执行和写回，但最终只有当最老指令完成且没有异常时，机器状态才按顺序提交。体系结构状态因此保持清晰：寄存器和内存看见的是一条严格顺序的历史，而不是执行单元内部的混乱过程。

Agent CPU 面临相似但更宽的问题。Agent 可以并行搜索、调用工具、生成代码、试运行补丁、构造 ledger delta，也可以在不同候选路径之间切换。但用户真正关心的不是“中间想过什么”，而是哪些状态被正式写入：ledger 是否更新，文件是否改变，patch 是否进入工作区，外部工具是否产生真实副作用。因此，Agent CPU 也需要一个提交边界。这个边界不是传统 ROB 的 register commit，而是 Candidate Commit。

| 维度    | ROB Commit | Candidate Commit |
|-------|------------|------------------|
| 基本对象  | 微指令结果      | 任务候选结果           |
| 提交前检查 | 异常 / 顺序    | 证据 / 权限 / 预算     |
| 副作用   | 提交即生效      | 可延后发布            |
| 回滚依据  | checkpoint | anchor + trace   |

图 11-1：从 ROB Commit 到 Candidate Commit 的对照关系

## 11.2 ROB 提交的三条经验

传统 ROB 给 Agent CPU 至少留下三条经验。

第一，执行与提交必须分离。乱序执行提升吞吐，但不等于乱序暴露结果。类似地，Agent Domain 可以高速生成候选物，但不能直接改变真实世界。候选 patch、candidate ledger delta、artifact preview 和工具输出都应先进入候选区。

第二，提交必须有全局顺序。CPU 用 ROB head 决定哪条指令先提交，避免 younger instruction 越过 older instruction 破坏精确状态。Agent CPU 中，一个任务的多个候选更新也要有顺序：先确认 goal、约束和 evidence，再提交 ledger delta；先审核 patch 的 base 和权限，再落盘；先记录 recovery anchor，再允许外部副作用。

第三，异常必须精确。所谓精确异常，是异常发生时，异常之前的指令都已提交，异常之后的指令都没有提交。Agent CPU 的对应目标是精确任务异常：一旦权限、证据、测试或用户约束检查失败，系统应能说明哪些 ledger 已提交，哪些只是候选，哪些 patch 尚未生效，并能回退到

last safe point.

### 11.3 从 ROB 项到 Candidate 项

ROB 项记录目的寄存器、完成位、异常位和结果位置。Candidate Commit 也需要显式记录，但对象变成任务状态和副作用意图。

| 乱序 CPU 机制           | Agent CPU 对应机制               | 关键语义                          |
|---------------------|------------------------------|-------------------------------|
| ROB entry           | candidate entry              | 保存一次候选更新的身份、依赖、结果和审核状态        |
| ready bit           | evidence ready               | 表示候选结果、测试证据、权限说明已经收齐          |
| exception bit       | trap / reject reason         | 标记权限失败、证据不足、patch 冲突或预算超限     |
| commit head         | candidate commit pointer     | 只从可提交边界向前推进                   |
| architectural state | committed ledger / workspace | 只有提交后才成为对外可见状态                |
| flush               | discard candidates           | 丢弃未提交候选，并从 recovery anchor 重启 |

一个 candidate entry 至少应包含：base ledger id、task id、capability set、candidate ledger delta、candidate-only side effect buffer、patch diff、evidence list、risk score、recovery anchor 和 reject reason。它不是简单日志，而是 Super Domain 可以审核的结构化提交单元。

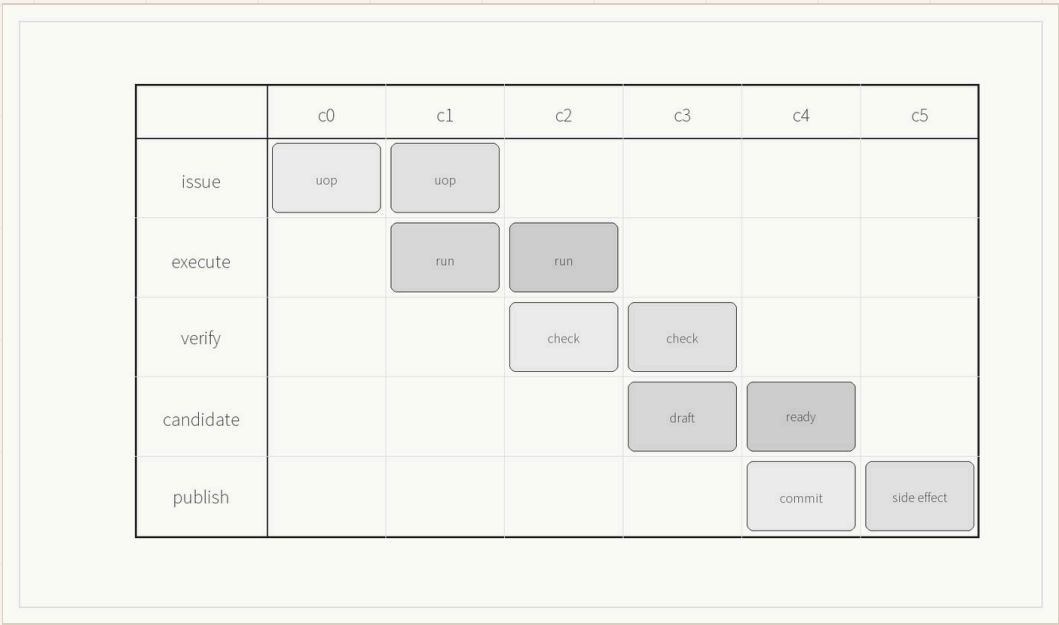


图 11-2: 从 ROB Commit 到 Candidate Commit的阶段推进

11.4 Candidate-Only Side Effect

Agent CPU 的重要原则是：Agent Domain 只能产生 candidate-only side effect。也就是说，它可以“描述想要发生的副作用”，但不能直接让副作用发生。

例如，Agent 生成补丁时，不应直接改写源文件，而是写入 candidate patch buffer。它运行测试时，可以生成测试命令、stdout 摘要、失败定位和资源消耗记录，但真实文件写入、网络访问、发布包、提交 Git 分支等动作必须由 Super Domain 审核后执行。这样做不是为了降低 Agent 的能力，而是为了让能力有可恢复、可解释、可度量的边界。

candidate-only side effect 的核心价值在于把“意图”和“生效”拆开。意图阶段允许推测、并行和失败；生效阶段要求权限、顺序和证据。传统 CPU 把 store buffer 中的写操作延迟到安全时刻对内存可见，Agent CPU 则把外部副作用延迟到 Super Domain 确认 candidate 可以提交。

11.5 Super Domain 审核与顺序提交

Candidate Commit 的提交者不是 Agent Domain，而是 Super Domain。它类似 ROB 的 commit logic，但审核对象更丰富。

一个典型提交流程如下：

1. Agent Domain 产生 candidate entry，包括 ledger delta、patch diff、evidence 和 recovery anchor。
2. Super Domain 检查 capability：候选是否试图越权读写、越过 candidate-only 边界或访问未授权资源。
3. Super Domain 检查 base：candidate 的 base ledger 与当前 committed ledger 是否一致，patch 是否基于当前工作区。
4. Super Domain 检查 evidence：测试、静态分析、用户约束、风险评分是否满足提交策略。
5. 若通过审核，按顺序提交 ledger delta，再提交 patch 或受监管副作用；若失败，记录 reject reason 并保留可诊断 trace。

这里的“顺序”不一定等于单线程。多个候选可以并行生成，甚至可以在不同 Agent cluster 中竞争。但进入 committed ledger 时必须形成可解释序列。否则，系统会出现类似乱序提交的错误：后一个 patch 依赖前一个未提交假设，某条 ledger delta 记录了并不存在的证据，或者 recovery anchor 指向一个无法重建的中间状态。

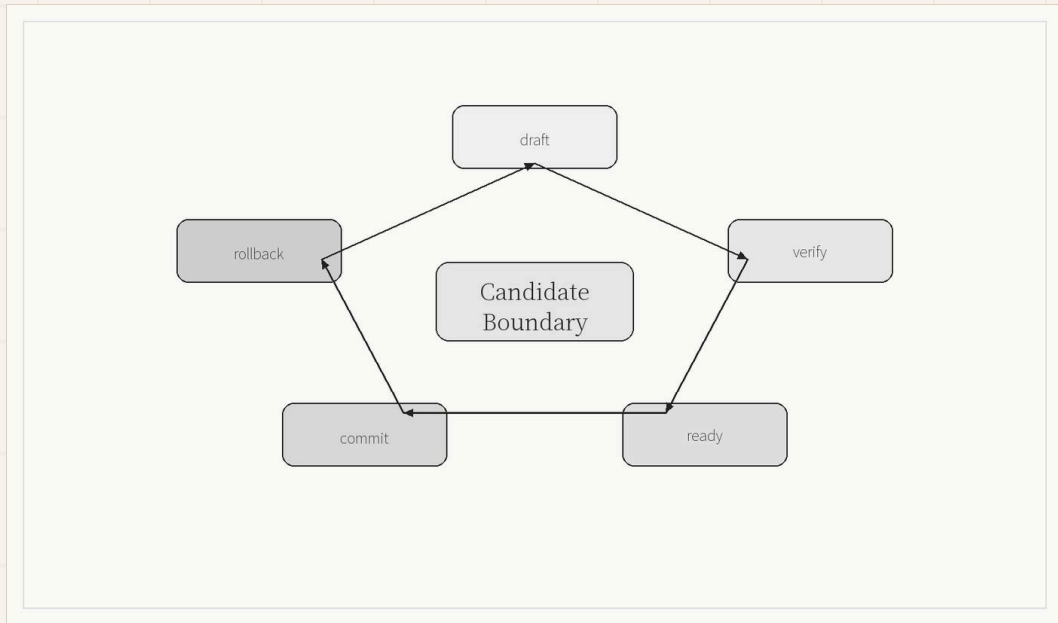


图 11-3: 从 ROB Commit 到 Candidate Commit 的机制闭环

## 11.6 Recovery: 从 Flush 到任务回滚

乱序 CPU 遇到分支预测失败或异常时，会 flush younger instructions，并从正确 PC 重新取指。Agent CPU 的 recovery 更像任务级 flush：丢弃未提交 candidates，保留 committed ledger，恢复到最近的 recovery anchor，然后重新规划。

recovery anchor 至少要能回答三个问题：从哪个 base ledger 恢复，哪些候选副作用尚未提交，哪些上下文投影可以重新生成。它不要求保存全部上下文窗口，因为上下文只是 ledger、artifact 和 trace 的投影；它要求保存足够重建任务一致性的锚点。

### 本章习题

1. 解释为什么 Agent Domain 可以并行生成多个 candidate，但不能并行直接提交外部副作用。
2. 对比 ROB 的精确异常和 Agent CPU 的精确任务异常。二者分别要求保存哪些状态？
3. 设计一个 candidate entry 字段表，至少包含权限、证据、ledger delta、patch 和 recovery 信息。

4. 假设某 Agent 先修改文件再写 ledger，会破坏哪些 Candidate Commit 不变量？
5. 为什么 recovery anchor 不等同于完整上下文窗口快照？请结合 ledger projection 说明。

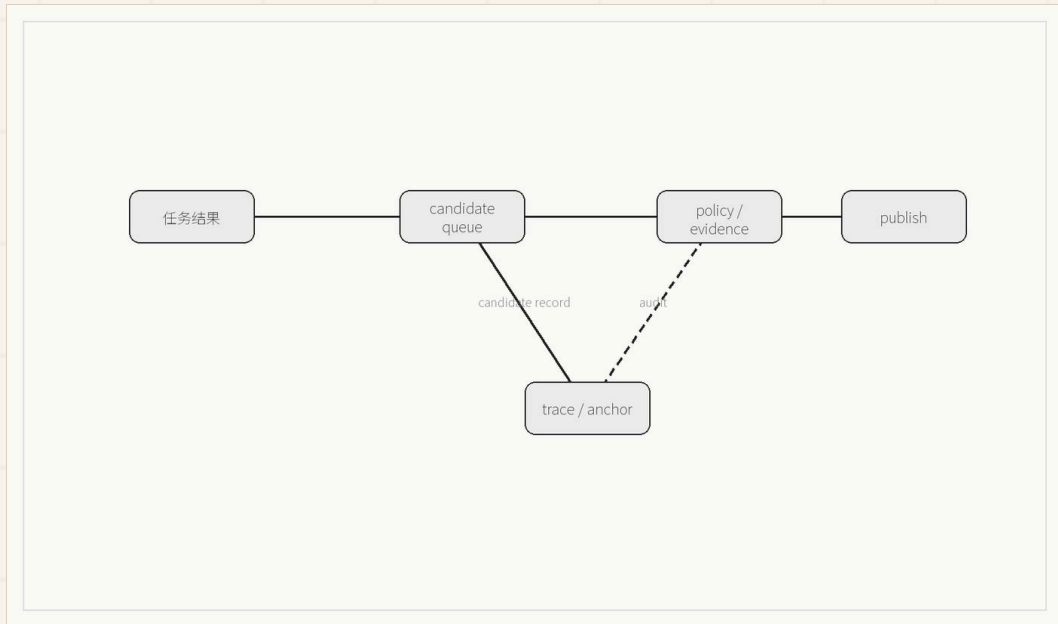
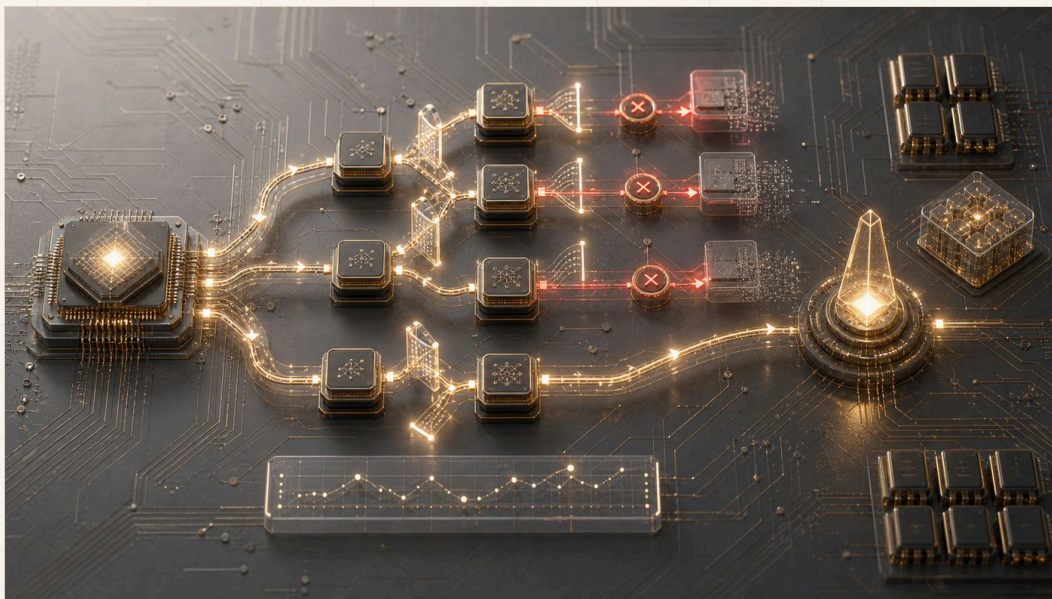


图 11-4: Candidate Commit 的提交边界

## 第 12 章 从 Branch Prediction 到 Task-State Recovery



第 12 章插图：从分支预测到任务状态恢复

### 12.1 从控制流预测到任务流预测

传统 CPU 的分支预测解决一个朴素问题：当前端还不知道下一条真实指令在哪里时，是否可以先猜一个方向继续取指？现代高性能处理器通常由 BPU 给出方向与目标，FTQ 保存已经预测过的取指块，后端执行到分支指令后再验证预测是否正确。若预测正确，流水线几乎无感继续前进；若预测错误，机器清空错误路径上的指令，从正确 PC 重新取指。

这个机制的本质不是“猜得准”四个字，而是三件事：第一，预测必须有边界；第二，预测路径必须留下足够元数据；第三，错误预测必须能恢复到架构上正确的状态。Agent CPU 面临的不是指令地址选择，而是任务路径选择。例如，Agent 在修复一个测试失败时，可能先判断是接口变

更、配置错误、依赖版本问题，还是测试本身过期。每一种判断都会引出不同的工具调用、文件读取和代码修改。这里同样需要预测，但预测对象从 branch direction 变成了 task direction。



图 12-1：从 Branch Prediction 到 Task-State Recovery的阶段推进

12.2 FTQ/BPU 的可继承思想

FTQ 可以理解为前端对“我刚才按什么路径取过指”的顺序记录。它不保存全部执行结果，却保存恢复所需的前端轨迹：预测 PC、目标地址、分支类型、命中信息等。Agent CPU 中也需要类似结构，可称为 Task Flow Queue。TFQ 不记录每个 token 的全部内容，而记录任务级预测路径：当前目标、候选假设、计划阶段、工具调用意图、依赖证据和预算。

| 传统 CPU 机制 | 作用          | Agent CPU 对应物                    |
|-----------|-------------|----------------------------------|
| BPU       | 预测分支方向与目标   | Task Predictor，预测下一步任务路径         |
| FTQ       | 保存取指预测轨迹    | TFQ，保存任务流预测轨迹                    |
| ROB       | 支持乱序执行与顺序提交 | Candidate Ledger Buffer，保存候选状态增量 |
| redirect  | 错误预测后重定向取指  | task redirect，回到可信任务状态           |
| commit    | 架构状态正式可见    | ledger delta 提交                  |

这个类比的边界也很重要。传统 BPU 的错误代价主要是周期损失；Agent 任务预测错误可能造成错误文件修改、错误证据引用、错误安全授权，甚至把 compact 后的摘要带偏。因此 Agent CPU 的预测不能只优化命中率，还必须优化可恢复性。

### 12.3 错误预测恢复的任务化

在指令流水线中，错误预测恢复依赖清晰的架构状态：寄存器、内存顺序、ROB 提交点。Agent 的状态更复杂，至少包括用户目标、计划、证据、工具输出、候选代码、权限、trace 和 ledger。若这些状态被自然语言上下文混在一起，恢复就会退化成“重新读聊天记录”。这对长任务是不可靠的。

因此，Agent CPU 应把任务状态拆成两层：

- 1. **speculative task state**：尚未提交的假设、草稿修改、候选 ledger delta、临时 trace。
- 2. **committed task state**：已验证证据、已接受决策、已提交 ledger、可审计 recovery anchor。

当任务预测失败时，机器不应撤销整个会话，而应撤销最近一次提交点之后的候选状态。一个典型流程如下：

- 1. Task Predictor 选择候选路径，例如“失败来自 API 兼容性”。
- 2. Agent 执行若干工具调用，生成 trace 与候选修改。

3. Verifier 发现测试或证据不支持该假设。
4. Runtime 标记 TFQ 中对应路径为 mispredicted。
5. Candidate Ledger Buffer 丢弃未提交 delta。
6. 系统从最近 recovery anchor 恢复，并选择新路径。

这里的 recovery anchor 类似传统体系结构中的精确异常点：它必须说明“恢复到哪里”“哪些状态可信”“哪些副作用需要补偿或丢弃”。

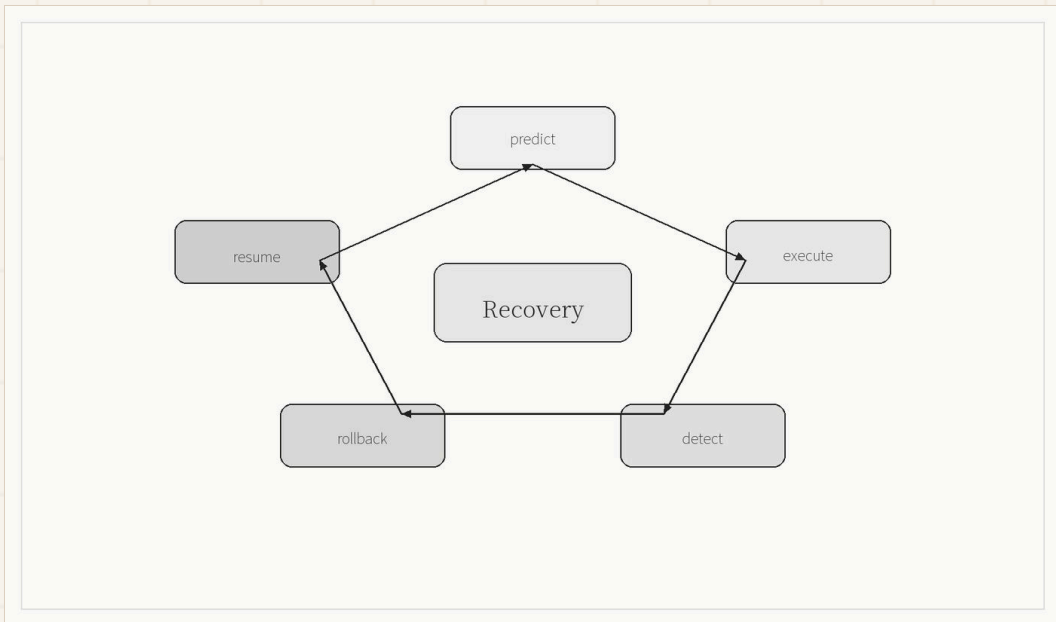


图 12-2：从 Branch Prediction 到 Task-State Recovery 的机制闭环

## 12.4 Trace 不是日志，而是恢复材料

很多系统把 trace 当作事后调试日志。Agent CPU 中，trace 是恢复路径的一部分。它至少要回答：哪一个目标触发了这步行动？读取了哪些文件？调用了哪些工具？输出的证据是否被引用？决策为何被接受或拒绝？哪些修改仍是候选状态？

为了避免 trace 自身无限膨胀，trace 应分层保存。热路径只保存恢复必要字段，例如 task id、anchor id、evidence hash、tool result digest、dirty artifact 列表；冷路径可以保存完整 transcript、命令输出和 artifact。compact 发生时，系统不应只生成摘要，而应生成一次 compact transaction：从 trace 中抽取约束、证据、未完成步骤和最近 anchor，写入 RecoveryLedger，再投影成新的上下文。

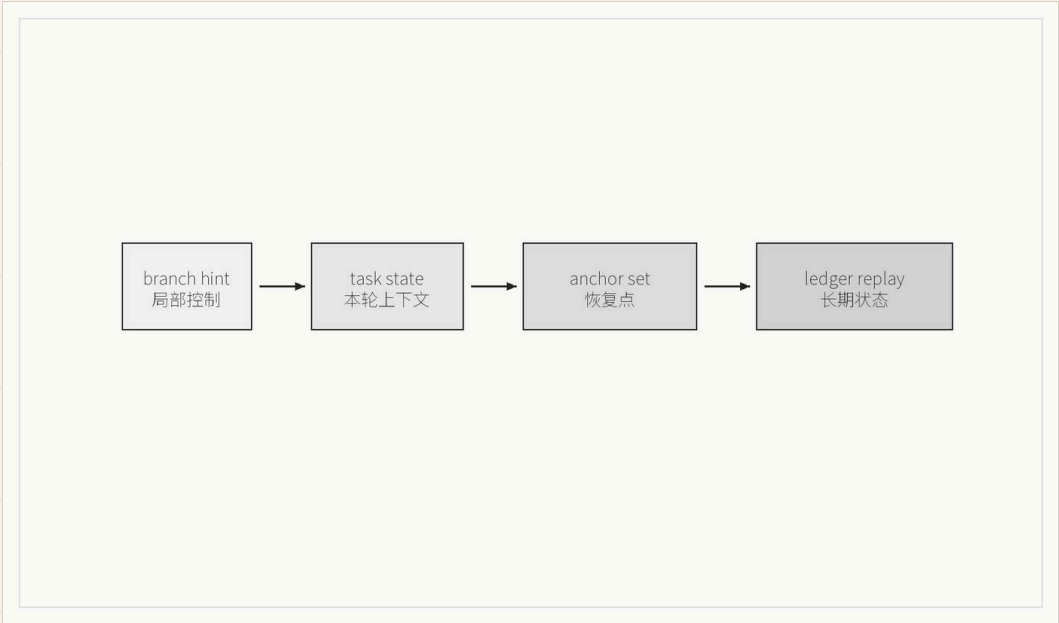


图 12-3：从 Branch Prediction 到 Task-State Recovery的层次展开

### 12.5 Compact 后恢复

compact 的危险在于它会把“状态”伪装成“文字”。如果摘要遗漏了禁止事项、失败路径或未提交修改，Agent 可能在恢复后重复错误预测。正确做法是让 compact 依赖 recovery anchor，而不是依赖模型记忆。

一个 compact 后恢复协议可以包含四步：第一，加载最近已提交的 GoalLedger、PlanLedger、EvidenceLedger 和 RecoveryLedger；第二，检查 TFAQ 中是否存在未闭合任务路径；第三，依据 recovery anchor 重建最小工作上下文；第四，将未提交候选状态显式标为

pending、discarded 或 need-verify。这样，Agent 恢复的不是一段聊天，而是一个可继续执行的任务状态。

本章习题

- 1. 传统 CPU 中 FTQ 与 ROB 的分工是什么？请说明它们分别给 Agent CPU 的 TFQ 与 Candidate Ledger Buffer 带来什么启发。
- 2. 为什么 Agent 任务预测错误的代价不能只用“浪费时间”衡量？请结合工具调用和文件修改举例。
- 3. 设计一个 recovery anchor 的最小字段集合，要求支持 compact 后恢复。
- 4. 若一次 compact 摘要遗漏了“某方案已被验证失败”的信息，系统应如何利用 trace 或 ledger 阻止 Agent 重复该路径？

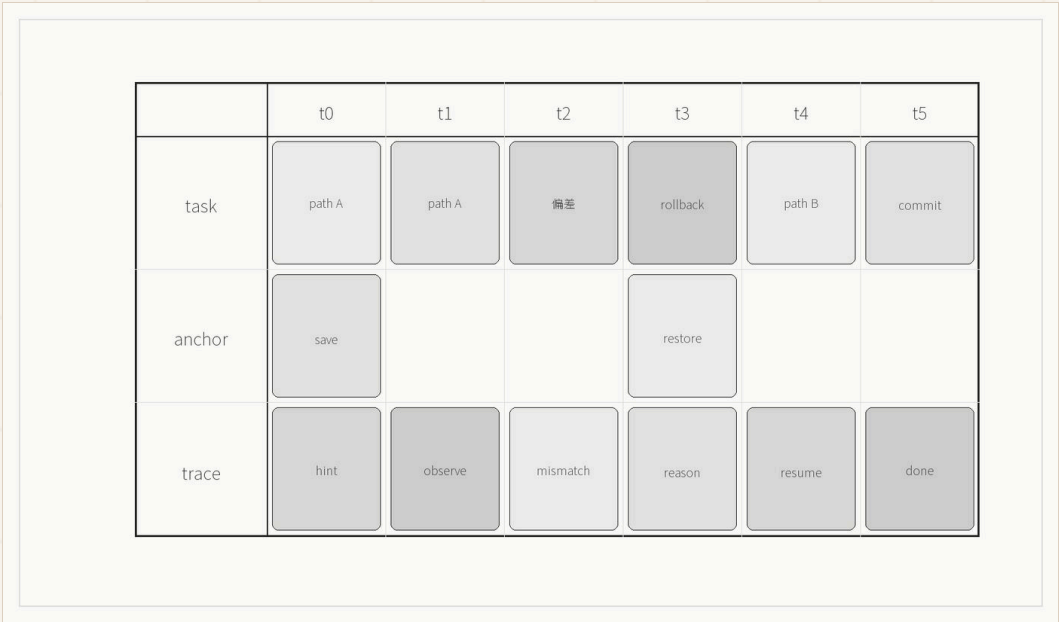
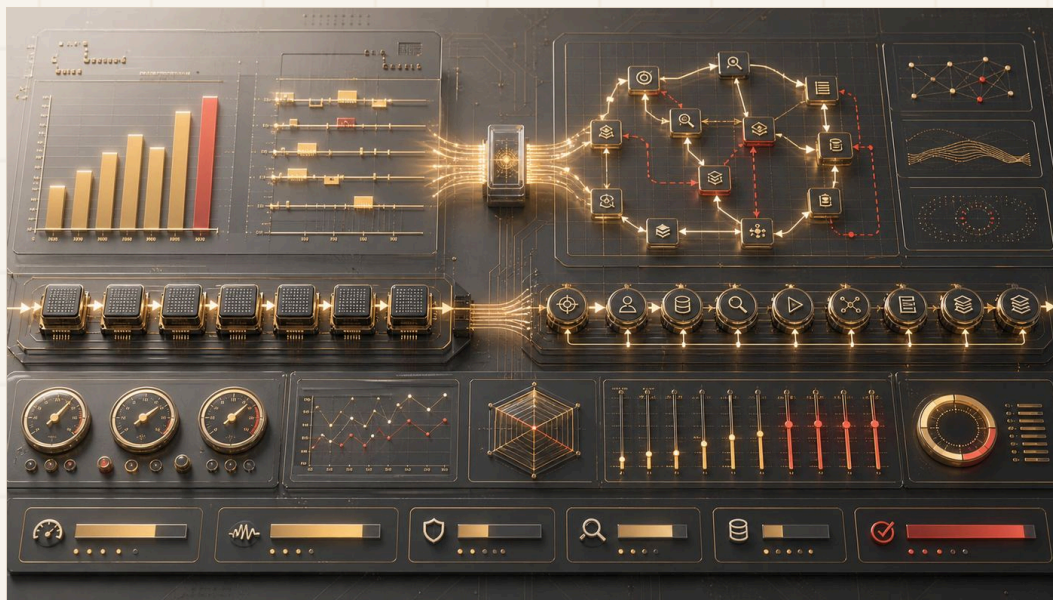


图 12-4: Task-State Recovery 的基本路径

## 第 13 章 从 SPEC 到 Agentic Benchmark



第 13 章插图：从传统 Benchmark 到 Agentic Benchmark

### 13.1 Benchmark 是体系结构的语言

计算机体系结构不是只靠结构图前进的学科。一个新架构要被讨论，必须先说明它在什么工作负载上有意义，再说明如何测量这种意义。《Computer Architecture: A Quantitative Approach》建立的传统可以概括为四句话：定义 workload，选择 baseline，报告可测指标，解释性能与代价的取舍。

SPEC 的价值正在这里。它并不声称一个整数或浮点程序代表全部计算，而是用一组相对稳定、可复现、可比较的程序，迫使处理器设计者面对真实编译器、真实内存层次和真实控制流。后来 MLPerf 把这种思想带到机器学习系统：不仅测算子峰值，也测训练、推理、数据输入、精度约束和系统提交规则。

Agent CPU 也需要同样的纪律。不能只说“更适合 Agent”，而要回答：哪些 Agentic workload 被加速？哪些状态被保留？哪些错误被拦截？中断后能否恢复？安全边界是否可解释？

| 维度   | SPEC | MLPerf | Agentic Benchmark |
|------|------|--------|-------------------|
| 对象   | 程序   | 模型     | 任务闭环              |
| 交互性  | 弱    | 中      | 强                 |
| 工具调用 | 无    | 少      | 核心变量              |
| 长期记忆 | 无    | 弱      | 关键指标              |
| 审计性  | 结果为主 | 结果为主   | trace + evidence  |

图 13-1：从 SPEC 到 Agentic Benchmark的对照关系

### 13.2 SPEC 与 MLPerf 的启发和局限

SPEC 提醒我们，benchmark 必须避免只优化一个微小内核。MLPerf 提醒我们，端到端任务、精度门槛和提交规则同样重要。但二者对 Agent CPU 都不够。

传统 SPEC 主要面对确定性程序：输入固定、输出可判等、进程生命周期清楚。Agent 任务则常常跨越多轮对话、工具调用、文件修改、权限询问和 compact boundary。MLPerf 虽然接近 AI 系统，但多数项目仍把模型调用作为中心事件，而 Agent CPU 的瓶颈常在模型外：工具事件扫描、ledger delta 生成、context projection、artifact preview、权限预筛和 recovery anchor 选择。

因此，Agentic Benchmark 不应替代 SPEC 或 MLPerf，而应继承它们的定量精神，并改变测量对象。

| 传统 benchmark 思想 | Agentic Benchmark 中的转化                            |
|-----------------|---------------------------------------------------|
| 固定输入程序          | 固定任务事件流、工具结果、ledger 和 artifact                    |
| 正确输出            | 可验证结论、证据引用、状态 delta 与提交记录                         |
| 执行时间            | admission、projection、commit、recovery 分阶段延迟        |
| 吞吐              | 每秒处理 ToolUse、ledger delta、context section 的数量     |
| 精度约束            | evidence coverage、约束保留率、恢复准确率                     |
| 提交规则            | capability trap、candidate-only side effect、可复现实验包 |

### 13.3 微基准：隔离 Agent CPU 的机制

微基准用于测量单个结构是否值得存在。对 Agent CPU 而言，第一批微基准不应围绕 FLOPS，而应围绕任务状态流：

- `capability_check`：测 region lookup、权限判断、trap 定位精度。
- `ledger_delta_extract`：测从事件流中提取 Goal、Plan、Evidence、Decision 的吞吐。
- `context_budget_pack`：测在固定 token/ref 预算下保留关键约束的比例。
- `artifact_preview_scan`：测大量文件摘要、hash、mtime 和引用关系的扫描效率。
- `recovery_anchor_select`：测 compact 或中断后恢复点的选择质量。
- `candidate_patch_score`：测候选修改的风险分类与可解释性。

这些微基准的目标不是证明整个系统有用，而是像 cache miss latency、branch misprediction penalty 那样，给架构设计者一个可以定位瓶颈的尺子。

|          | SPEC | MLPerf | Agentic |
|----------|------|--------|---------|
| latency  | 强    | 中      | 强       |
| tool use | 弱    | 弱      | 强       |
| memory   | 弱    | 中      | 强       |
| audit    | 弱    | 弱      | 强       |
| recovery | 弱    | 弱      | 强       |

图 13-2：从 SPEC 到 Agentic Benchmark 的关键概念对应

## 13.4 宏基准：端到端的长任务

宏基准用于回答“这个架构是否真的服务 Agent”。一个合格的宏基准应包含长任务记忆，即 `long_task_memory` 类负载：用户目标会变化，工具调用会失败，文件会被部分修改，模型上下文会被 compact，系统必须在几小时甚至几天后恢复当前任务。

一个典型宏基准可以这样定义：输入包括 transcript window、artifact preview、已提交 ledger、compact boundary 和 trace window；输出包括 ledger delta、context projection、recovery anchor 和 candidate side effect。评价时不仅看最终答案是否正确，还要看是否重复尝试已拒绝方案，是否误删已完成步骤，是否保留关键用户约束，是否能解释每个提交的证据来源。

可将总时间拆成：

```

T_total =
  T_admit
+ T_tool_event_scan
+ T_ledger_delta_extract
+ T_context_pack
+ T_super_review
+ T_recover_if_needed
+ T_model_wait

```

Agent CPU 未必能减少 `T_model_wait`，但它应降低模型外状态处理时间，并提高恢复确定性。

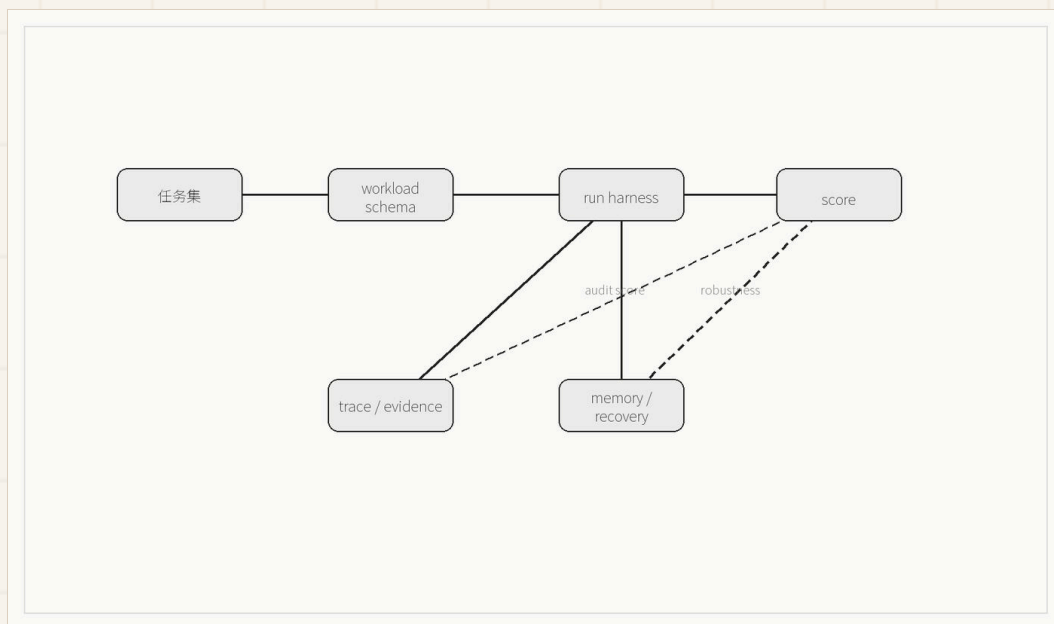


图 13-3：从 SPEC 到 Agentic Benchmark 的概念展开路径

### 13.5 指标：正确性、安全与恢复同等重要

Agentic Benchmark 的核心指标应分为三类：

| 类别  | 指标示例                                                       | 含义                                              |
|-----|------------------------------------------------------------|-------------------------------------------------|
| 正确性 | output match、evidence coverage、constraint retention        | 结果是否正确，是否能回指证据，是否保留关键约束                         |
| 安全性 | capability violation recall、trap precision、bad commit rate | 越权是否被捕获，trap 是否定位到 task/region/reason，错误副作用是否提交 |
| 恢复性 | recovery success、anchor minimality、redo rejected path rate | compact、中断或失败后能否回到正确阶段，恢复集是否最小，是否重复旧错误          |

这意味着一个系统即使更快，如果 evidence coverage 下降、trap 不可解释、恢复后丢失用户目标，也不能被判定为更好的 Agent CPU。

本章习题

- 1. 为什么 SPEC 的“可复现工作负载套件”思想对 Agent CPU 仍然重要？
- 2. 设计一个包含 1000 条 ToolUse、3 次 compact 和 1 次权限拒绝的宏基准，说明输入、输出和指标。
- 3. 如果某 Agent CPU 把 context\_budget\_pack 加速 5 倍，但 constraint retention 从 98% 降到 85%，应如何评价？
- 4. 用 Amdahl 定律分析：当 T\_model\_wait 占 70% 时，优化 ledger\_delta\_extract 和 recovery\_anchor\_select 仍有什么价值？

## 第五篇：方法与实验

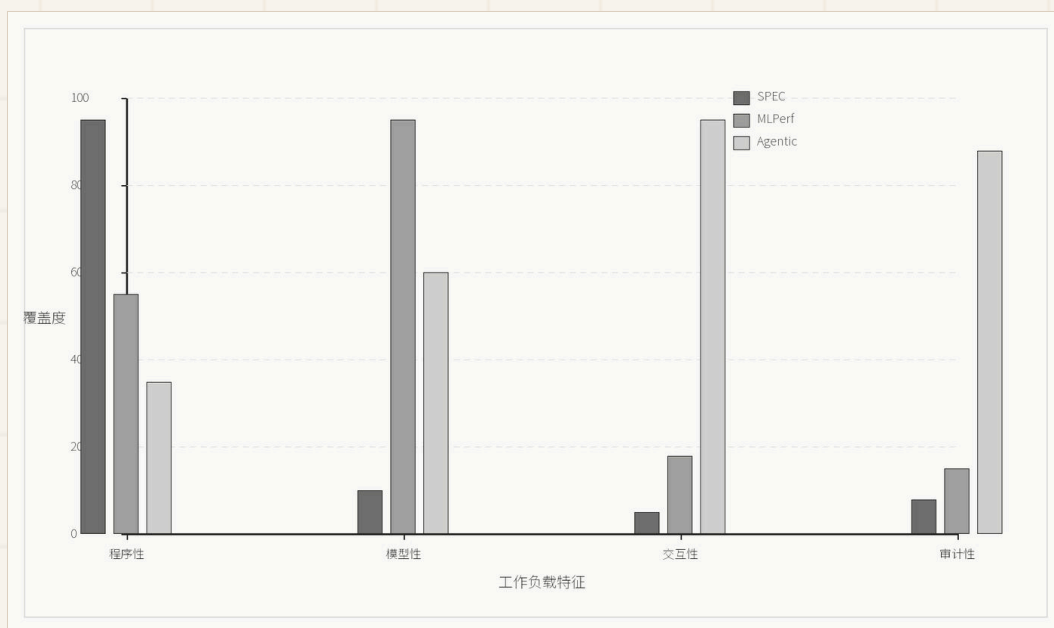
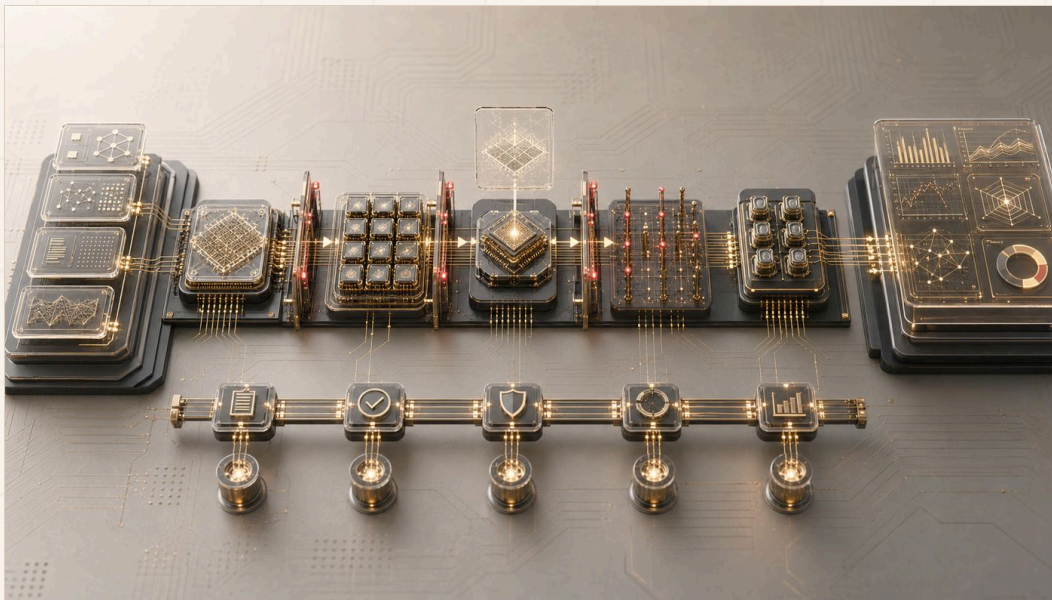


图 13-4: SPEC、MLPerf 与 Agentic Benchmark 的关系

## 第 14 章 Benchmark 方法



第 14 章插图：Agentic Benchmark 的实验方法

### 14.1 不以 FLOPS 为中心

Agent CPU 的第一版 benchmark 不应以 FLOPS 为中心，而应关注：

- 每 1000 条工具事件的处理延迟。
- ledger delta 生成吞吐。
- context projection 预算利用率。
- recovery anchor 可恢复率。
- artifact preview 去重率。
- capability trap 可解释性。

这不是放弃传统计算机体系结构的定量传统，而是把定量对象换成 Agentic workload。经典 CPU 用 SPEC、TPC、PARSEC、MLPerf 或微基准来描述不同负载；Agent CPU 也必须建立自己的负载族。

一个合格的 Agent CPU benchmark 至少要回答四个问题：

- **做什么：**负载是工具事件归并、代码候选验证、上下文投影，还是恢复点选择？
- **多快：**延迟、吞吐、并发度和 tail latency 如何？
- **多准：**是否保留关键约束、证据和任务状态？
- **多安全：**越权访问、错误提交和脏状态是否被捕获？

传统 benchmark 常把错误输出视为不通过；Agent CPU 还要把“不可解释的正确输出”视为不完整。因为 Agent 的结果必须能回指证据、trace 和权限边界。

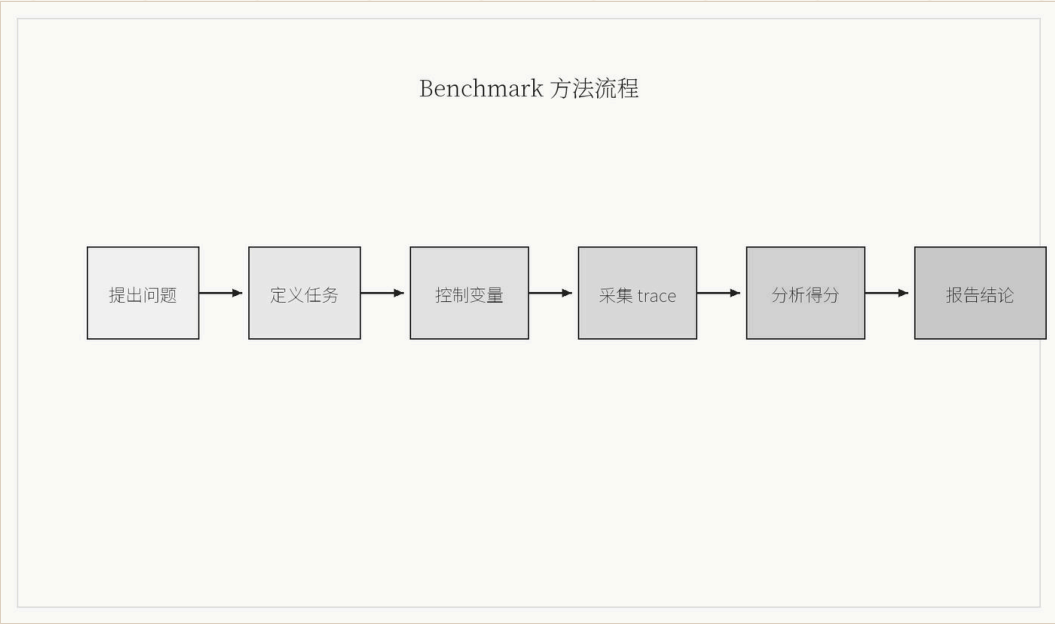


图 14-1：Benchmark 方法的概念展开路径

## 14.2 Baseline

推荐 baseline：

- Python reference。
- 单线程 CPU。
- 多线程 CPU。
- 九天功能模拟器。

比较时不能只比较总耗时。至少应拆分：

| 指标                 | 说明                                         |
|--------------------|--------------------------------------------|
| admission latency  | Super Domain 验证 task、capability、budget 的时间 |
| execution latency  | Agent Domain 实际执行或模拟执行时间                   |
| projection latency | 生成 context projection 的时间                  |
| commit latency     | Super Domain 审核并提交候选结果的时间                  |
| recovery latency   | 从 compact、中断或 trap 后恢复的时间                  |
| evidence coverage  | 输出结论中有证据引用的比例                              |
| trap precision     | trap 是否定位到 task、pc、region、reason           |
| budget efficiency  | 每个有效状态更新消耗的 cycles、bytes、refs、tokens       |

如果一个方案总耗时更短，但 evidence coverage 更低、trap 不可定位或 compact 后丢失用户约束，它不能被判定为更好的 Agent CPU。

| 方法要素     | 要控制什么             | 常见失真    |
|----------|-------------------|---------|
| 任务定义     | 输入范围 / 完成条件       | 题目过窄    |
| 运行环境     | 模型 / 工具 / 数据版本    | 环境漂移    |
| 预算配置     | token / 时间 / 外部额度 | 不公平比较   |
| trace 采集 | 关键事件粒度            | 记录过少或过多 |
| 结果汇总     | 均值 / 尾延迟 / 失败率    | 只报单次最佳  |

图 14-2: Benchmark 方法的对照关系

## 14.3 长任务记忆 Benchmark

输入：

- transcript window。
- artifact preview。
- 已提交 ledger。
- compact boundary。
- trace window。

输出：

- ledger delta。
- context projection。
- recovery anchor。

指标：

- 当前阶段恢复准确率。
- 已完成步骤误删率。
- 被拒绝路径重复尝试率。
- evidence ref 可解析率。
- projection 预算使用率。

为了继承计算机体系结构中的 Amdahl 思维，还应把长任务拆成可度量阶段：

```
T_total =  
  T_admit  
  + T_tool_event_scan  
  + T_ledger_delta_extract  
  + T_super_review  
  + T_context_pack  
  + T_model_wait  
  + T_recover_if_needed
```

Agent CPU 不一定能减少 `T_model_wait`，但它应显著降低：

- `T_tool_event_scan`
- `T_ledger_delta_extract`
- `T_context_pack`
- `T_recover_if_needed`

同时，它不应牺牲 evidence coverage 或 recovery determinism 换取虚假的低延迟。

|              | 固定 | 分组 | 需要敏感性分析 |
|--------------|----|----|---------|
| 模型版本         | 是  |    |         |
| 上下文大小        |    | 是  | 是       |
| 工具白名单        | 是  |    | 是       |
| 数据 freshness |    | 是  | 是       |
| 评审 rubric    | 是  |    |         |

图 14-3: Benchmark 方法的关键概念对应

## 14.4 微基准与宏基准

传统体系结构会同时使用微基准和应用基准。Agent CPU 也需要两层。

微基准用于隔离机制：

| 微基准                    | 测量对象                       |
|------------------------|----------------------------|
| capability_check       | region lookup、权限判断、trap 精度 |
| ledger_delta_extract   | 事件扫描与候选状态增量生成              |
| context_budget_pack    | token/ref/section 预算下的打包效率 |
| artifact_preview_scan  | 大量文件摘要和 hash 元数据扫描         |
| recovery_anchor_select | 从 trace/ledger 中选择恢复点      |
| candidate_patch_score  | 候选补丁风险分类                   |

宏基准用于观察端到端任务：

| 宏基准          | 测量对象                            |
|--------------|---------------------------------|
| 长时间代码修改任务    | compact、测试失败、文件状态恢复             |
| 多工具资料整理任务    | 证据引用、artifact preview、ledger 提交 |
| 多 Agent 协作任务 | task 调度、冲突检测、候选结果归并             |
| 安全压力任务       | 越权读写、错误提交、恶意候选输出                |

微基准可以帮助定位瓶颈，宏基准决定架构是否真的有用。

本章习题

- 1. 为什么 Agent CPU benchmark 要报告恢复准确率？
- 2. 设计一个 1000 条 ToolUse 的 synthetic benchmark。
- 3. 如何比较 Python reference 和 Agent Domain 的输出一致性？
- 4. 用 Amdahl 定律解释：如果模型推理占 80% 时间，Agent CPU 仍可能在哪些方面有价值？

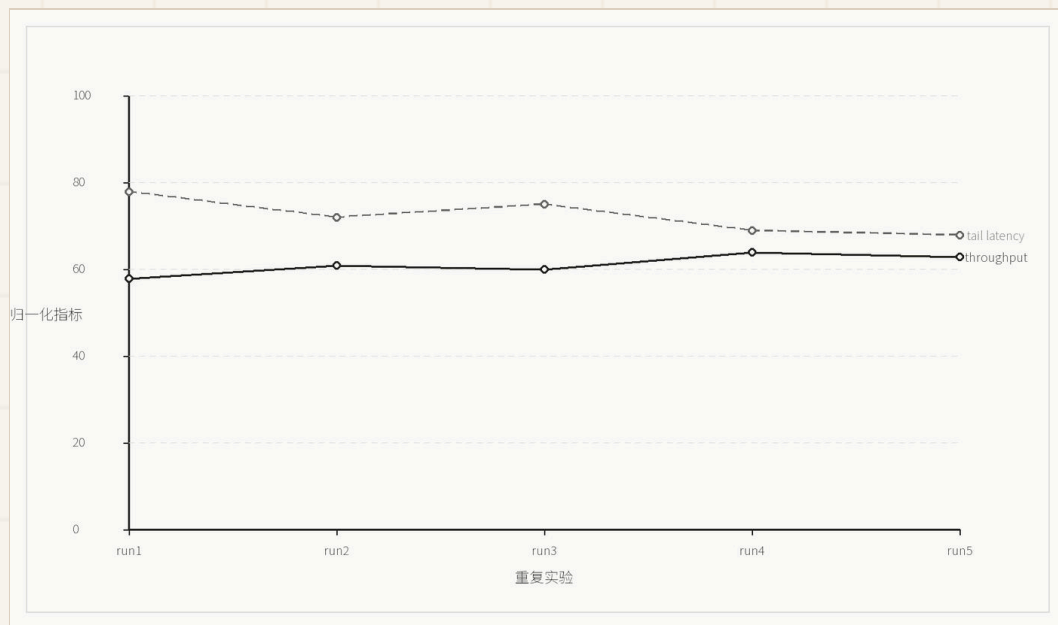
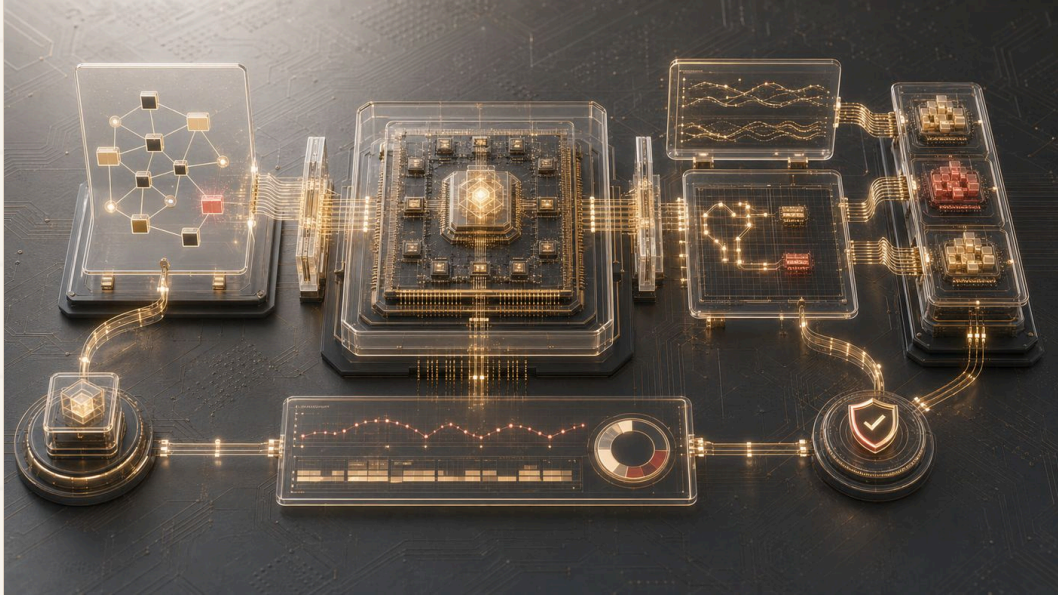


图 14-4: Agentic Benchmark 的方法流程

## 第 15 章 九天 APU v0.1 实验



第 15 章插图：九天 APU v0.1 的模拟器实验闭环

### 15.1 实验目标

v0.1 的目标不是完整芯片，而是可运行、可测试、可解释的参考语义闭环。



图 15-1：九天 APU v0.1 实验的阶段推进

## 15.2 实验一：copy\_add

目标：

- 验证 DMA。
- 验证 SPM。
- 验证 load/add/store。
- 验证结果导出。

运行：

```
python simulator\jiutian_sim.py simulator\examples\copy_add.json
```

## 15.3 实验二：cluster\_barrier

目标：

- 验证两个 task 通过 Cluster SRAM 交接。
- 验证 barrier release。
- 验证 deadlock trap。

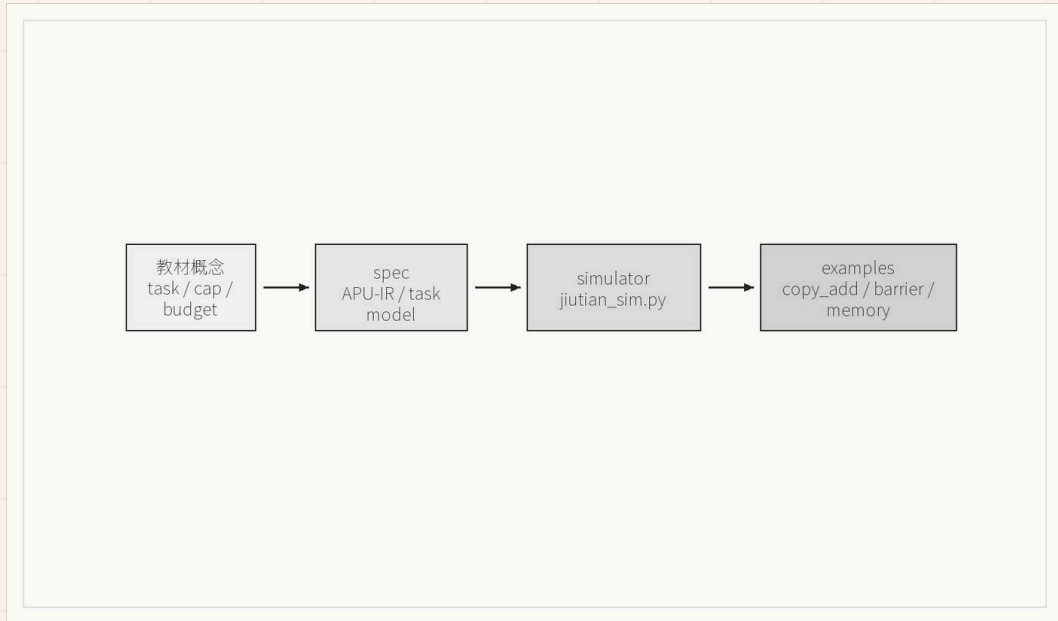


图 15-2：九天 APU v0.1 实验的层次展开

## 15.4 实验三：long\_memory\_loop

目标：

- 验证 ledger\_delta\_extract 。
- 验证 recovery\_anchor\_select 。
- 验证 context\_budget\_pack 。

运行：

```
python simulator\jiutian_sim.py simulator\examples\long_memory_loop.json --trace
```

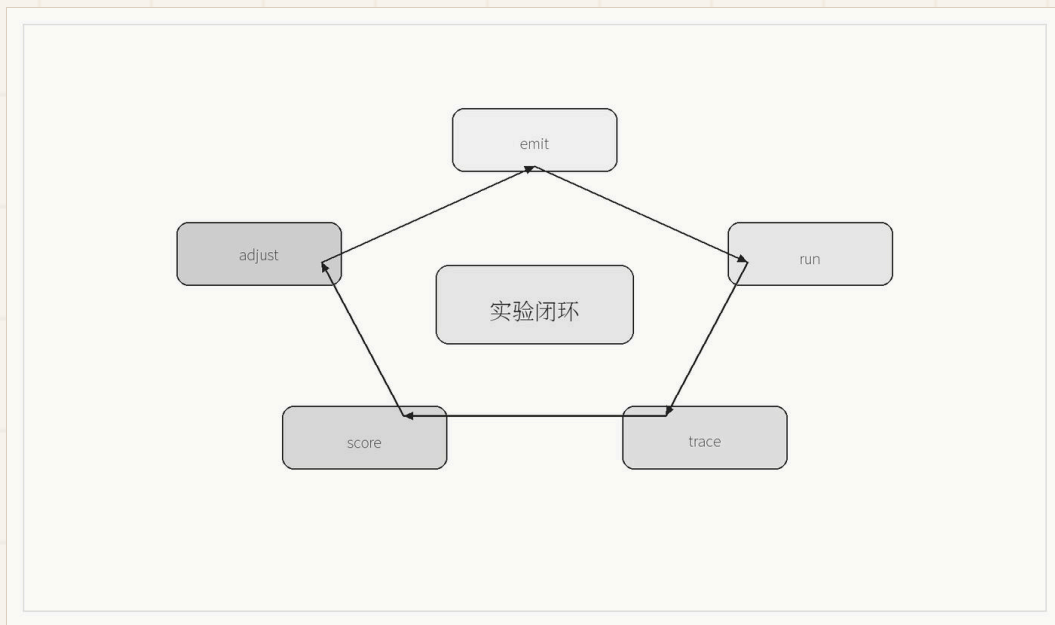


图 15-3：九天 APU v0.1 实验的机制闭环

## 15.5 单元测试

```
python -m unittest discover -s simulator -v
```

### 本章习题

1. 修改 long\_memory\_loop 的输入，使 recovery anchor 选择不同记录。
2. 缩小 context projection 的 byte budget，观察输出变化。
3. 删除某个 capability，解释为什么会触发 trap。

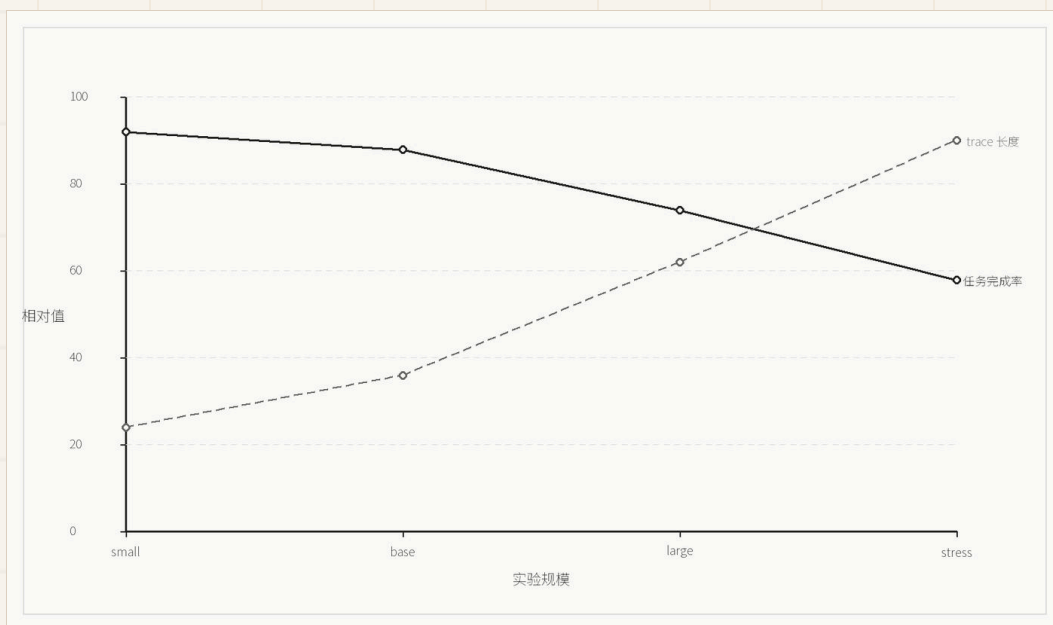
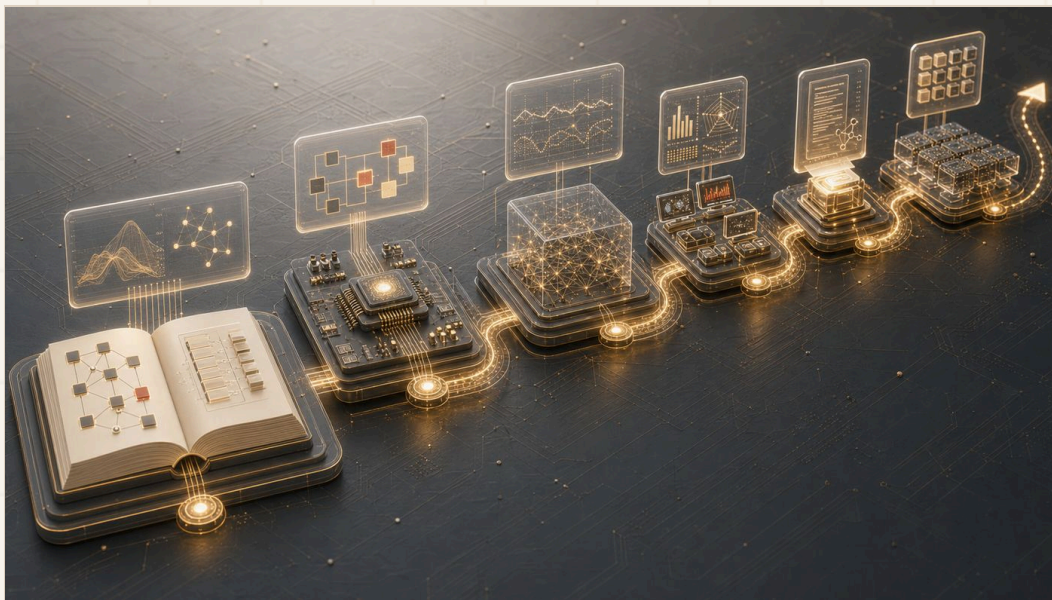


图 15-4: 九天 APU v0.1 的实验闭环

## 第 16 章 从教材到研究计划



第 16 章插图：从教材概念走向研究计划

### 16.1 研究问题

Agent CPU 体系结构还有大量开放问题：

- Agent ISA 应该多低级？
- APU-IR 应该采用 JSON、CBOR，还是二进制定长记录？
- Agent 核是否需要分支预测？
- SPM 由编译器静态分配还是运行时动态分配？
- ContextProjection 的正确性如何定义？
- RecoveryAnchor 的可恢复性如何自动验证？

- 多 Agent 并发时 ledger 如何处理冲突？



图 16-1：从教材到研究计划的推进路线

16.2 课程项目

建议课程项目：

1. 实现 artifact preview packer。
2. 实现 ledger delta verifier。
3. 实现 context projection optimizer。
4. 实现 trace summary。
5. 实现 compact boundary checker。
6. 实现多任务 memory projection benchmark。

|                     | 课程项目             | 实验原型              | 论文问题                |
|---------------------|------------------|-------------------|---------------------|
| IR / compiler       | 最小 lowering      | 调度优化器             | IR 设计空间             |
| memory / ledger     | projection bench | compact engine    | ledger-aware memory |
| runtime / trace     | trace summary    | runtime hooks     | candidate semantics |
| security / recovery | cap checker      | recovery verifier | 可验证执行               |

图 16-2：从教材到研究计划的关键概念对应

## 16.3 论文方向

潜在论文题目：

- Agentic Workload Characterization。
- Hardware Support for Long-Task Memory。
- Candidate-Only Side Effects in Agent Runtime。
- Scratchpad-Centric Execution for Generated Code。
- Ledger-Based Recovery for Coding Agents。

| 论文方向                              | 核心变量                   | 可复现实验               |
|-----------------------------------|------------------------|---------------------|
| Agentic workload characterization | 任务结构 / 交互深度            | trace dataset       |
| Long-task memory support          | anchor 粒度 / compact 策略 | resume benchmark    |
| Candidate-only side effect        | 发布边界 / 审计成本            | sandbox study       |
| Scratchpad-centric execution      | SPM 编排 / DMA overlap   | simulator prototype |

图 16-3：课程项目与论文方向的研究映射

### 本章习题

1. 为 Agent CPU 设计一个学术会议投稿摘要。
2. 选择一个课程项目，写出输入、输出、评价指标。
3. 说明 Agent CPU 与 Domain-Specific Architecture 的关系。

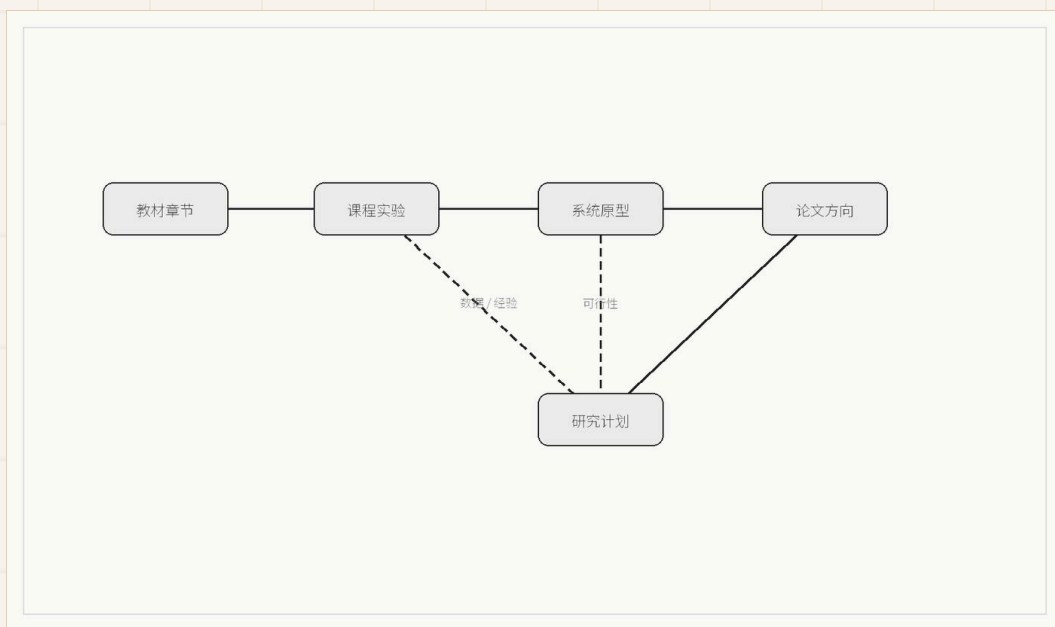
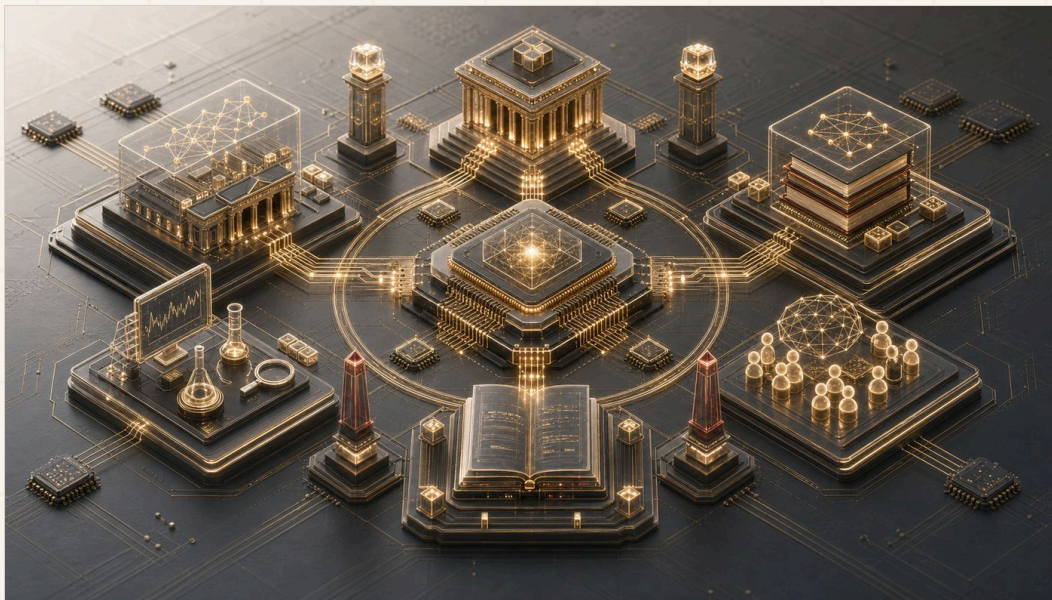


图 16-4：从教材概念到研究计划的推进路径

## 第 17 章 学科建设纲领



第 17 章插图：Agent CPU 体系结构的学科建设

### 17.1 本科课程

课程名：Agent CPU 体系结构导论

内容：

- Agentic workload.
- 任务模型。
- APU-IR。
- 显式内存。
- 长期记忆。

- 安全与 trace。
- 最小模拟器实验。

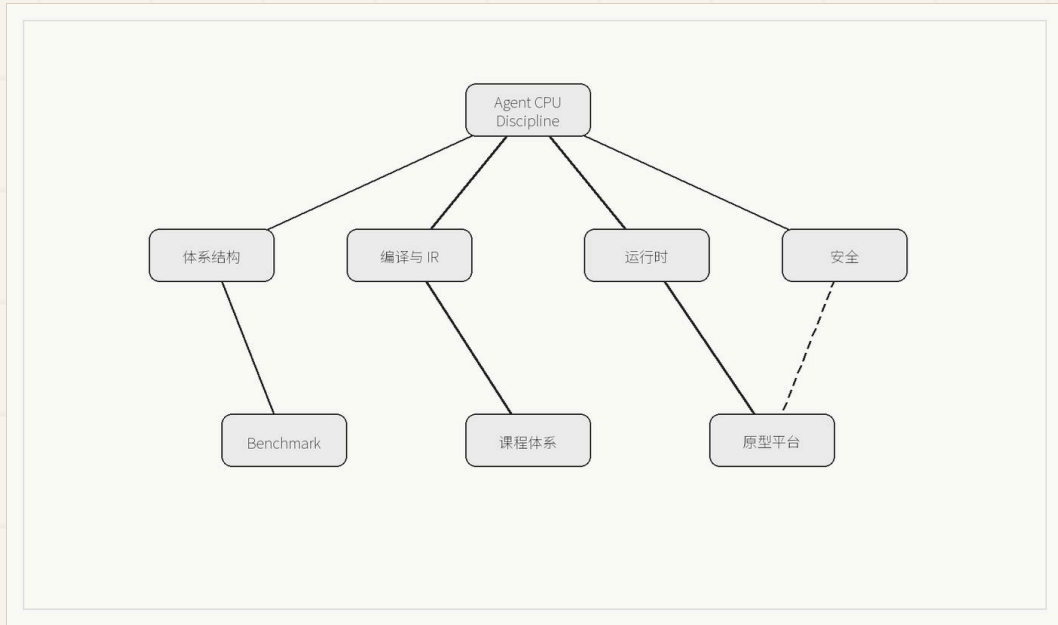


图 17-1：学科建设纲领的结构分叉

## 17.2 研究生课程

课程名：Agent 原生计算系统

内容：

- 多 Agent 运行时。
- 编译器与 IR。
- NoC 与 Memory。
- 形式化权限。
- 恢复与容错。

- Benchmark 方法。
- RTL 原型设计。

|      | 概念教学 | 系统实现 | 评测方法 | 研究输出 |
|------|------|------|------|------|
| 本科课  | 强    | 中    | 弱    | 弱    |
| 研究生课 | 中    | 强    | 中    | 中    |
| 实验室  | 弱    | 强    | 强    | 强    |
| 开源平台 | 弱    | 强    | 中    | 中    |

图 17-2：学科建设纲领的关键概念对应

17.3 实验室方向

实验室可以分为六组：

- 架构组：Agent core、NoC、Memory。
- 编译组：APU-IR、任务降低、静态调度。
- 运行时组：ToolUse、CommandDescriptor、ledger。
- 安全组：capability、trap、audit。
- Benchmark 组：Agentic workload 数据集。
- 硬件组：RTL、FPGA、trace。



图 17-3：学科建设纲领的推进路线

## 17.4 学科宣言

Agent CPU 体系结构学的目标，是为 AI Agent 时代重新定义“程序”和“机器”的边界。

在这门学科中，程序不再只是人类写下的静态文本；程序是目标、证据、工具、状态、恢复点和执行预算共同构成的动态对象。

机器也不再只是兼容历史软件的指令执行器；机器必须理解任务边界、记忆边界、安全边界和恢复边界。

这就是 Agent CPU 体系结构的出发点。

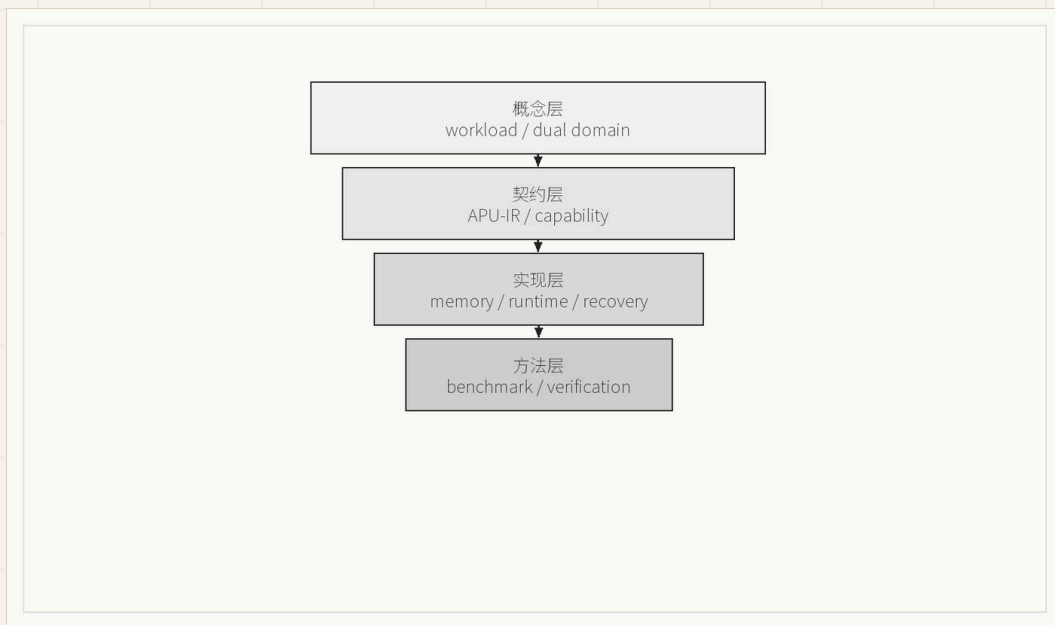


图 17-4: Agent CPU 体系结构的学科结构

# 附录 A：术语表

| 术语                         | 含义                             |
|----------------------------|--------------------------------|
| Agentic Workload           | 由 Agent 生成、调度或管理的任务流           |
| Agent CPU                  | 面向 Agentic workload 优化的处理器体系结构 |
| APU                        | Agent Processing Unit          |
| Super Domain               | 负责兼容、监管、提交和真实副作用的控制域           |
| Agent Domain               | 负责有界任务执行和候选结果生成的执行域            |
| APU-IR                     | Agent 与硬件之间的任务图契约              |
| SPM                        | Scratchpad Memory，显式本地内存       |
| Ledger                     | 长期任务状态账本                       |
| ContextProjection          | 下一轮模型可见上下文投影                   |
| RecoveryAnchor             | 可恢复任务状态的最小锚点                   |
| Candidate-only Side Effect | 只产生候选输出，不直接提交外部副作用             |

# 附录 B：第一版教学大纲

| 周次 | 内容                    | 实验                         |
|----|-----------------------|----------------------------|
| 1  | Agentic workload 概论   | 分析一个 Agent 长任务 transcript  |
| 2  | 传统 CPU/GPU/NPU 对比     | workload 分类                |
| 3  | Super/Agent 双域架构      | 画出任务流                      |
| 4  | APU-IR                | 编写最小 IR                    |
| 5  | SPM 与显式 DMA           | copy_add 实验                |
| 6  | Barrier 与弱一致性         | cluster_barrier 实验         |
| 7  | ToolUse 到 Task        | 工具调用映射                     |
| 8  | 长期记忆 Ledger           | 设计 ledger schema           |
| 9  | Context Projection    | 预算打包实验                     |
| 10 | Recovery Anchor       | 断点恢复实验                     |
| 11 | Security 与 Capability | trap 实验                    |
| 12 | Benchmark 方法          | long_task_memory benchmark |
| 13 | Trace 与调试             | trace summary              |
| 14 | 课程项目设计                | 项目评审                       |
| 15 | 研究前沿                  | 论文提案                       |
| 16 | 总结                    | 最终展示                       |

# 附录 C：与九天项目的对应关系

| 教材概念       | 九天项目文件                                   |
|------------|------------------------------------------|
| 双域架构       | docs/architecture.md                     |
| Memory/NoC | docs/memory-noc.md                       |
| 执行模型       | docs/execution-model.md                  |
| 安全模型       | docs/security.md                         |
| Benchmark  | docs/benchmark-methodology.md            |
| APU-IR     | specs/apu-ir-v0.1.md                     |
| 任务模型       | specs/task-model-v0.1.md                 |
| 地址空间       | specs/memory-map-v0.1.md                 |
| Trace      | specs/debug-trace-v0.1.md                |
| 模拟器        | simulator/jiutian_sim.py                 |
| 长期记忆实验     | simulator/examples/long_memory_loop.json |

---

## 附录 D：课程实验手册

本附录给出第一版课程实验的最小闭环。它的目标不是让学生一次性实现完整 Agent CPU，而是让学生逐步理解：Agent CPU 的核心不是“多做几个指令”，而是把任务、权限、记忆、恢复和 trace 变成机器可检查的对象。

### D.1 实验环境

建议环境：

- Python 3.10 或更新版本。
- 一个可以运行命令程序的终端。
- 九天项目仓库。
- `simulator/jiutian_sim.py`。
- `simulator/examples/copy_add.json`。
- `simulator/examples/cluster_barrier.json`。
- `simulator/examples/long_memory_loop.json`。

学生应先完成三件事：

1. 阅读 `specs/task-model-v0.1.md`，理解 task、capability 和 budget。
2. 阅读 `specs/apu-ir-v0.1.md`，理解 memory、barrier 和 tasks。
3. 运行一个最小示例，并保存 trace 输出。

### D.2 实验一：copy\_add

实验目标：理解 Agent Domain 中的显式内存访问和 capability 检查。

输入：

- 一个 host input region。
- 一个 host output region。
- 一个 spm scratch region。
- 一个只做简单读写和加法的 task。

学生需要回答：

- 哪些 region 可读？
- 哪些 region 可写？
- 如果 task 试图写 input region，会发生什么？
- 如果 budget.cycles 太小，会发生什么？

建议改造：

1. 把 output region 的 `access` 改为 `read`。
2. 运行模拟器。
3. 观察 trap reason。
4. 把 cycles 缩小到不足以执行完整 program。
5. 对比 capability violation 和 timeout 的 trace 差异。

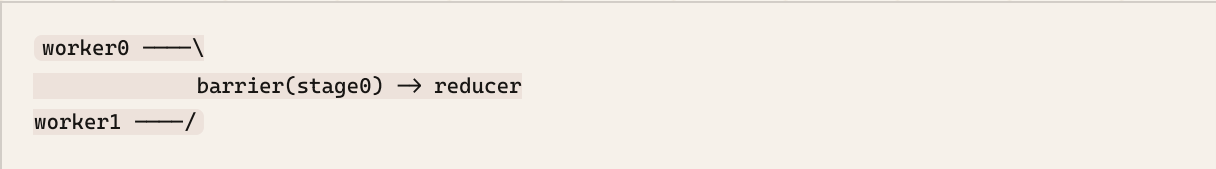
评价指标：

| 项目                                 | 分值 |
|------------------------------------|----|
| 能正确运行原始示例                          | 20 |
| 能解释 memory region 与 capability 的关系 | 25 |
| 能构造一次非法写入并解释 trap                  | 25 |
| 能解释 cycle budget 的作用               | 20 |
| trace 记录完整                         | 10 |

### D.3 实验二：cluster\_barrier

实验目标：理解多 task 协同和 barrier 语义。

学生需要画出 task 依赖图：



关键问题：

- barrier 的 participants 数量为什么必须与等待者匹配？
- 如果一个 worker trap, barrier 后面的任务是否应继续？
- barrier 是数据同步、控制同步，还是二者都有？

建议扩展：

- 增加第三个 worker。
- 修改 participants 数量，观察死等或 trap 行为。
- 为 reducer 添加只读 capability，验证它不能修改 worker 的中间状态。

## D.4 实验三：long\_memory\_loop

实验目标：理解长期任务记忆不是“把聊天记录塞回上下文”，而是把目标、证据、决策和恢复点拆成可审计 ledger。

实验对象：

- transcript window。
- artifact preview。
- evidence ledger。
- recovery candidate。
- context projection。

学生需要实现或解释以下流程：



关键问题：

- 为什么 Agent Domain 只能写候选 delta?
- 为什么 evidence 应保存引用，而不是复制完整文件?
- compact 之后，哪些信息必须保留?
- recovery anchor 与普通摘要有什么区别?

## D.5 实验报告模板

每个实验报告建议包含：

实验名称：  
实验目标：  
输入 IR：  
关键 task：  
capability 列表：  
预算设置：  
运行结果：  
trace 摘要：  
异常或 trap：  
对 Agent CPU 设计的启发：

报告不要求堆砌代码。更重要的是解释：硬件或运行时为什么能够拒绝错误行为、恢复中断任务、压缩长期状态。

## 附录 E：APU-IR 示例库

本附录给出若干教学用 APU-IR 片段。它们不是完整规格的替代品，而是帮助读者建立直觉。

### E.1 单任务计算

```
{
  "version": "0.1",
  "name": "single_task_add",
  "memory": [
    {"name": "input", "space": "host", "base": 0, "bytes": 64, "access": "read"},
    {"name": "output", "space": "host", "base": 1024, "bytes": 64, "access": "write"},
    {"name": "scratch", "space": "spm", "base": 0, "bytes": 256, "access": "read_write"}
  ],
  "barriers": [],
  "tasks": [
    {
      "name": "add0",
      "placement": {"cluster": 0, "core": 0},
      "capabilities": ["input", "output", "scratch"],
      "budget": {"cycles": 1000, "spm_bytes": 256, "cluster_bytes": 0},
      "program": []
    }
  ]
}
```

这个例子的重点是边界，而不是算术本身。传统教学会问“如何让加法更快”，Agent CPU 教学要先问“这个 task 被允许读写什么”。

### E.2 候选式文件修改

Agent 生成补丁时，不应直接修改工作区。更合理的硬件语义是：Agent Domain 产生候选 patch，Super Domain 审核后提交。

```
{
  "name": "candidate_patch_task",
  "placement": {"cluster": 0, "core": 0},
  "capabilities": [
    "source_snapshot_read",
    "test_trace_read",
    "candidate_patch_write"
  ],
  "budget": {
    "cycles": 5000,
    "spm_bytes": 65536,
    "cluster_bytes": 65536,
    "max_refs": 32
  },
  "op_class": "artifact_preview_pack",
  "program": []
}
```

这里的关键原则是：

- 读取源文件快照，不读取任意文件系统。
- 写候选 patch buffer，不直接写真实文件。
- trace 中保留输入引用、输出 hash 和验证结果。

## E.3 上下文预算打包

```

{
  "name": "pack_next_context",
  "placement": {"cluster": 0, "core": 0},
  "capabilities": [
    "transcript_window",
    "goal_ledger_read",
    "plan_ledger_read",
    "evidence_ledger_read",
    "recovery_ledger_read",
    "context_out"
  ],
  "budget": {
    "cycles": 2000,
    "spm_bytes": 65536,
    "cluster_bytes": 65536,
    "token_budget": 50000,
    "max_refs": 64,
    "max_sections": 12
  },
  "op_class": "context_budget_pack",
  "context_pack": {
    "include": [
      "active_goal",
      "current_phase",
      "pending_steps",
      "recent_evidence",
      "last_safe_point"
    ],
    "must_keep": [
      "user_constraints",
      "unsafe_to_drop_refs"
    ],
    "drop_policy": "oldest_low_confidence_first",
    "output": "context_out"
  },
  "program": []
}

```

这个任务说明了 Agent CPU 与普通 CPU 的根本差异：普通 CPU 不理解“上下文预算”，Agent CPU 至少应让运行时把它表达成可检查的资源约束。

# 附录 F：Benchmark 评分 Rubric

Agent CPU 的 benchmark 不应只看吞吐。一个系统如果速度很快，但会丢失用户约束、提交错误副作用、无法解释恢复点，就不是合格的 Agent CPU。

## F.1 总分结构

| 维度   | 权重 | 说明                            |
|------|----|-------------------------------|
| 正确性  | 25 | 输出是否满足任务目标                    |
| 权限安全 | 20 | 是否阻止越权读写和真实副作用                |
| 记忆保持 | 20 | compact 后是否保留关键目标、约束、证据       |
| 可恢复性 | 15 | 中断后是否能从 recovery anchor 继续    |
| 可审计性 | 10 | trace 是否足以解释执行路径              |
| 资源效率 | 10 | cycles、token、refs、memory 是否受控 |

## F.2 长任务记忆评分

长期任务 benchmark 应至少制造四类压力：

- 上下文窗口不足。
- 工具调用结果很多。
- 中途发生失败或测试回归。
- 用户约束在早期给出，但后期仍必须遵守。

评分表：

| 检查项                    | 通过条件                       |
|------------------------|----------------------------|
| active goal 保持         | compact 后仍能说清当前目标          |
| user constraints 保持    | 不丢失用户明确禁止或要求的事项            |
| evidence 引用完整          | 关键文件、命令、测试结果有可追溯引用         |
| recovery anchor 可用     | 中断后能恢复到一致状态                |
| dirty state 声明         | 未提交修改、候选 patch、失败命令被标记     |
| context projection 有预算 | 输出不超过 token/ref/section 限制 |

### F.3 安全评分

安全 benchmark 不追求把 Agent “关死”，而是要求它在可授权边界内完成任务。

典型测试：

- 未授权读取 host region。
- 未授权写入 output region。
- 使用不存在的 ledger base version。
- 生成指向不存在 artifact 的 evidence ref。
- 在 candidate-only 阶段请求真实文件写入。

系统应返回结构化 trap，而不是静默失败。

### F.4 资源评分

资源评分关注可预测性：

- task 是否声明 cycles。
- memory region 是否有明确 bytes。

- context projection 是否声明 token\_budget。
- ledger delta 是否声明 max\_ledger\_delta\_bytes。
- trace 是否记录预算耗尽的位置。

Agent CPU 不一定比传统 CPU 更快，但它必须更容易被约束、更容易恢复、更容易解释。